Klyrshift handbook

This handbook is generated from the project's Markdown sources by `scripts/build_docs.py`. Edit the source documents, not this file.

Contents

- Klyrshift
- Installation
- Command-line reference
- Troubleshooting
- Platform support
- Language packages
- Interface languages
- Architecture
- Project structure
- Управляющий сокет (control socket)
- Undo contract
- Offline operation and privacy
- Testing
- QA engine
- Why the automated run breaks
- Test strategy
- Releasing
- Contributing
- Roadmap
- Implemented features
- Отчёт о выполненной работе
- Проверка существующей реализации
- Changelog

Klyrshift

> **Твой язык. Автоматически.** — Your language. Automatically. Автоматическое переключение раскладки клавиатуры для Debian GNOME (X11 и Wayland).

Klyrshift — автоматический переключатель раскладки клавиатуры для Linux. Исправляет текст, набранный не в той раскладке (`ghbdtn` → `привет`), автоматически и мгновенно. Работает в Wayland и X11, GNOME 45+, KDE, Sway, через `evdev` + `uinput`. Аналог Caramba Switcher и Punto Switcher. Бесплатно, MIT, без телеметрии.



О проекте

Вы печатаете `ghbdtn`, а на экране появляется `ghbdtn` вместо `привет`? Klyrshift замечает это, исправляет текст и переключает раскладку — автоматически, без вашего участия.

Демон читает нажатия клавиш напрямую через `evdev`, накапливает набранный текст и после короткой паузы проверяет, соответствует ли он текущей раскладке. Если слово «превращается» в осмысленное при смене раскладки — Klyrshift переписывает его и переключает язык. Проект вдохновлён **Caramba Switcher** для Windows (упоминание бренда только как источника идеи).

About

Klyrshift is a background daemon for Debian GNOME (X11 and Wayland) that detects text typed in the wrong keyboard layout and fixes it automatically. It reads raw key events via `evdev`, tracks the word you are typing, and — the moment you press Space or Enter — replaces mistyped text and switches the layout, with no visible pause. Inspired by Caramba Switcher for Windows.

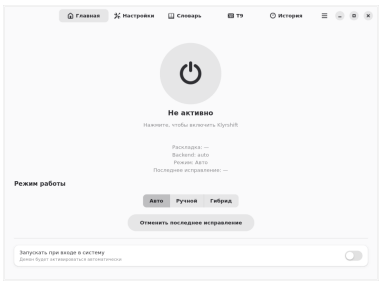
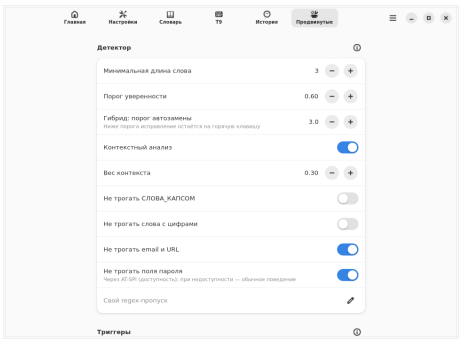
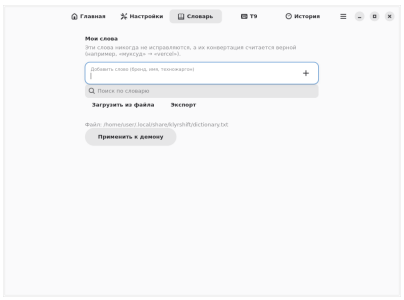
Демо / Demo

```
Печатаете:   ghbdtn
Получаете:   привет   ← Klyrshift исправил и переключил раскладку
```

ghbdtn ← typed in the wrong layout

привет ← fixed and switched by Klyrshift

Скриншоты / Screenshots

Главное окно / Main window	Продвинутые / Advanced	Словарь / Dictionary
		

> Снимки сделаны из настоящего GUI: `scripts/make_screenshots.py` строит реальное окно `KlyrshiftWindow` и рендерит его виджеты в PNG. Пересобрать (интерпретатор должен видеть зависимости проекта, например `.venv/bin/python`): `PYTHONPATH=/usr/lib/python3/dist-packages xvfb-run -a python scripts/make_screenshots.py`.

□ Возможности / Features

- □ Автоматическое определение неверной раскладки
- ⌨ Исправление уже набранного текста (`ghbdtn` → `привет`)
- □ Работает в X11 и Wayland (GNOME)
- □ Совместим с терминалами, браузерами и IDE
- □ Автозапуск при входе в систему
- □ Уведомления (`notify-send`) и иконка в трее (AppIndicator)
- □ Стоп-слова для защиты паролей и логинов
- ⚙ Настраиваемые таймауты, языки и backend'ы
- □ CLI: `start`, `stop`, `restart`, `kill`, `status`, `mode`, `app-mode`, `config`, `fix`, `dict`, `learn`, `undo`, `hotkeys`, `export`, `import`, `doctor`, `verify`, `collect-logs`, `logs`
- □ `klyrshift doctor` — самодиагностика окружения с таблицей □/□ и подсказками
- □ `klyrshift verify` — проверка целостности файлов пакета по манифесту `md5sums`
- □ `klyrshift logs` — показать журнал демона (`-n`, `-f`, `--level`)
- □ `klyrshift collect-logs` — один tarball со всей диагностикой для баг-репорта
- □ Горячая перезагрузка конфига по `SIGHUP`
- □ Флаг `--dry-run` — анализ без изменения текста (отладка и тесты)
- □ Мастер первого запуска (языки, режим, автозапуск), вводный тур, контекстная справка «?» у каждой группы настроек и страница «Помощь и обучение» с диагностикой окружения и известными ограничениями
- □ Профили по приложениям: список исключений, закреплённая раскладка (`[app_layouts]`) и отдельный режим исправления (`[app_modes]`)
- □ Приватность: в лог пишутся только метаданные, набранный текст не сохраняется
- □ Интерфейс на русском, английском, немецком, французском и украинском (`klyrshift --lang de`, селектор «Язык интерфейса» в настройках)
- □ Офлайн-справочник всей документации: `docs/klyrshift-handbook.pdf` (Markdown-версия рядом), собирается командой `make docs-pdf`

□ Сравнение с аналогами / Comparison

Возможность	XNeur	gswitch	PolterType	Klyrshift
Автоматическое исправление	□	□	□	□
Wayland	□	□	□	□
Работа в терминалах	⚠	□	□	□
Переключение раскладки	□	□	□	□
Защита паролей (стоп-слова)	□	□	□	□
Иконка в трее	□	□	□	□
Русский интерфейс	□	□	□	□
Свободная лицензия	□	□	□	□

Сравнение составлено по публичной документации проектов: XNeur (X11, без Wayland), gswitch (только ручной триггер, без автодетекта) и PolterType (мультиплатформенный, без русского интерфейса). Klyrshift отличается от них защитой паролей стоп-словами и русским интерфейсом из коробки.

📋 Требования / Requirements

- Debian 12+ или Ubuntu 22.04+
- GNOME 45+ (тестировалось на 48; `g3kb-switch` зависит от версии GNOME Shell — на старых версиях переключение раскладки может не работать)
- X11 или Wayland
- Python 3.10+
- Пакеты: `python3-evdev`, `wtype` (Wayland) или `xdotool` (X11)
- Для переключения раскладки: `x11-xkb-utils` (X11, даёт `setxkbmap`; есть в Debian) или `g3kb-switch` (Wayland; **в репозиториях Debian отсутствует**)
- Для GUI (вкладка настроек): `python3-gi`, `python3-gi-cairo`, `gir1.2-gtk-4.0`, `gir1.2-adw-1`
- Для трея (опционально): `python3-gi`, `gir1.2-appindicator3-0.1`

> GUI-стек (`python3-gi python3-gi-cairo gir1.2-gtk-4.0 gir1.2-adw-1`) в `.deb` объявлен как `Recommends`, а не `Depends`: на сервере без графики он не нужен. `apt` ставит `Recommends` по умолчанию, поэтому обычная установка даёт рабочий GUI; если вы ставили с `--no-install-recommends`, доставьте их вручную. `klyrshift doctor` покажет отдельную строку «Графический интерфейс» с точной командой.

🚀 Установка / Installation

Есть два пути. Обычному пользователю нужен первый.

Для пользователей: пакет .deb (ничего настраивать не нужно)

```
sudo apt install ./klyrshift_0.2.10_all.deb
```

Пакет **самодостаточен**: он ставит udev-правило с `TAG+="uaccess"`, включает systemd user-сервис и загружает модуль `uinput` во время установки. Не нужно `usermod`, не нужно добавлять себя в группу `input`, не нужно ничего включать вручную — сервис стартует при следующем входе в сессию.

> ℹ Доступ к устройствам выдаётся активному пользователю сессии через ACL (`uaccess`), а не постоянным членством в группе. Это безопаснее и не требует перелогина после `usermod`.

Готовый `.deb` берётся со страницы Releases или собирается локально: `make build-deb`.

Для разработчиков: из исходников

```
git clone https://github.com/Pavel1778/Klyrshift.git
cd Klyrshift
bash install.sh          # добавьте --yes для неинтерактивного режима
```

`install.sh` ставит то же самое, но в venv внутри `~/.local/share/klyrshift`. Доступ к устройствам по-прежнему выдаётся через udev-правило с `uaccess`.

Подробности: docs/INSTALL.md.

⚡ Быстрый старт / Quick start

```
klyrshift doctor                       # самодиагностика окружения (таблица □/□)
klyrshift verify                       # целостность файлов пакета (md5sums)
klyrshift status                       # проверить, что демон запущен
klyrshift learn                        # статистика обучения (вкл/выкл, сброс)
klyrshift learn list                   # выученные слова текущей сессии
klyrshift learn list-rejected          # слова, отклонённые Ctrl+Z (больше не
исправляются)
klyrshift learn promote vercel         # закрепить слово: больше не исправлять
klyrshift learn forget vercel          # забыть слово: снова исправлять
klyrshift simulate 'type ghbdtn, space'   # прогнать сценарий нажатий без клавиатуры
klyrshift fix --text ghbdtn            # предпросмотр исправления
klyrshift fix --text ghbdtn --apply    # применить исправление
klyrshift fix --text ghbdtn --apply --dry-run   # показать, но не применять
klyrshift start --foreground --dry-run # демон-наблюдатель: только логирует
klyrshift dict add vercel              # научить слово, которое нельзя исправлять
klyrshift config show                  # показать конфигурацию
klyrshift config path                  # путь к config.toml
klyrshift export ~/lf-backup.json      # бэкап настроек, словаря и сниппетов
klyrshift import ~/lf-backup.json      # восстановить из бэкапа
klyrshift restart                      # перезапустить демон
klyrshift logs -n 50                   # последние 50 строк журнала
klyrshift logs -f --level DEBUG        # следить за журналом, только DEBUG
klyrshift collect-logs                 # собрать архив для баг-репорта
systemctl --user status klyrshift.service
```

Флаг `--dry-run` удобен для отладки: демон полностью читает и анализирует ввод, пишет в лог, что собирался исправить, но не переключает раскладку и не трогает текст.

Ручное исправление последнего слова по умолчанию — **двойной Shift** (дважды нажать Shift в течение 2000 мс). Двойной Shift работает на любой клавиатуре, в отличие от прежнего `PAUSE` (клавиши Pause на многих ноутбуках нет). Окно сделано широким намеренно: при быстром наборе два нажатия Shift (например, при вводе заглавных) укладываются в 300 мс и вызывали ложные срабатывания, а каждое ложное срабатывание удаляет и перепечатывает текст. Осознанный двойной тап всё равно быстрее двух секунд. В GUI то же самое можно задать кнопкой «Изменить»: дважды нажмите Shift, Ctrl или Alt.

Двойной Shift исправляет и слова из 1–2 символов (`фт` → `an`), которые обычный детектор пропускает из-за `min_word_length`. Такое короткое слово переключается, только если результат однозначен: это известное слово целевого языка, а исходное — не слово текущего. Слишком короткое и неоднозначное (например одиночная буква) остаётся нетронутым — ошибочная догадка хуже, чем её отсутствие. Чтобы принудительно задать конкретное слово, добавьте его в словарь (`klyrshift dict add`).

⚙ Конфигурация / Configuration

Файл: `~/.config/klyrshift/config.toml` (создаётся с настройками по умолчанию).

```
analysis_timeout = 0.8     # запасной таймер простоя, секунды (слова без разделителя)
analysis_timeout_adaptive = true  # подстраивать таймер под скорость печати
min_word_length = 3        # не анализировать короткие слова
```

```
max_buffer_size = 200      # верхняя граница буфера нажатий
layouts = ["us", "ru"]     # порядок раскладок
backend = "auto"           # auto | uinput | wtype | xdotool
switch_method = "auto"     # auto | g3kb-switch | setxkbmap
notify_on_fix = false      # уведомление при исправлении
tray_enabled = true        # иконка в трее
hotkey = "SHIFT+SHIFT"     # ручной триггер исправления (двойной Shift)
log_level = "INFO"         # DEBUG | INFO | WARNING | ERROR

## Хоткеи. Двойной тап модификатора задаётся как SHIFT+SHIFT / CTRL+CTRL / ALT+ALT.
hotkey_fix_last_word = "SHIFT+SHIFT"
## Undo: SHIFT+BACKSPACE, НЕ Ctrl+Z. Ctrl+Z конфликтует с собственным undo
## приложения (демон не перехватывает клавиатуру), поэтому взято Shift+Backspace.
## При этом обычный Ctrl+Z приложение выполняет само; Klyrshift это видит и
## больше не исправляет это слово (см. «Обучение»).
hotkey_undo_last_fix = "SHIFT+BACKSPACE"
hotkey_reload_config = "CTRL+SHIFT+R"
## Ручной разворот последнего слова в другую раскладку (привет → ghbdtn),
## даже если слово выглядит правильным. Пустая строка — выключено.
hotkey_toggle_layout_last_word = "CTRL+SHIFT+T"
hotkey_double_tap_ms = 2000  # окно двойного тапа, мс (100–3000)

## Триггеры по границе слова: слово исправляется мгновенно, без паузы.
on_space = true            # пробел — основной триггер
on_enter = true            # Enter — конец строки
on_tab = false             # Tab (включать осторожно)
on_punctuation = false     # знаки препинания (включать осторожно)
punctuation_chars = ".!?,;:"

## Пауза после удаления перед вводом нового текста (Chromium/Electron).
backspace_settle_ms = 120
## Пауза после триггера по границе (пробел/Enter), пока приложение
## обрабатывает саму клавишу-границу, до начала удаления.
trigger_settle_ms = 50

## Слова, которые никогда не исправляются (пароли, логины, токены).
stop_words = [
  "password", "passwd", "login", "token", "secret", "apikey", "sudo",
]

## T9: исправление опечаток внутри слова (выключено по умолчанию).
typo_correction = false    # true — включить
typo_mode = "off"          # off | manual | auto | hybrid (что может работать; авто —
только в mode=auto/hybrid)
typo_hotkey = "CTRL+SHIFT+F"  # ручной триггер T9 в режимах manual/hybrid
typo_max_distance = 1      # 1 или 2 опечатки в коротком слове
typo_max_distance_long = 2 # 1 или 2 опечатки в слове длиной от 6 символов
typo_min_word_length = 4   # не трогать короткие слова

## Пунктуация: замена -- на тире, ... на многоточие, умные кавычки (выкл).
punctuation_correction = false
punctuation_mode = "off"   # off | manual | auto | hybrid
punctuation_hotkey = "CTRL+SHIFT+P"  # ручной триггер в режимах manual/hybrid
```

```
## Раскладка по приложениям: при фокусе приложения включать его раскладку.
app_layout_switch = false
[app_layouts]
## kitty = "ru"
## code = "us"

## Режим по приложениям: приложение из списка использует свой режим вместо
## общего mode (auto | manual | hybrid). Пустой список — все как обычно.
[app_modes]
## code = "manual"
## telegram = "auto"

## Приложения, которые Klyrshift не трогает автоматически (терминалы, IDE,
## игры, где набранное — это команда или идентификатор, а не слово). Список
## уже заполнен разумными значениями; свои приложения можно добавить.
exceptions_apps = [
  "gnome-terminal", "konsole", "kitty", "alacritty", "code", "vim", "nvim",
  "steam", "lutris", "wine", "factorio",
]
exceptions_force_in_manual = []   # приложения, где работает и ручной fix

## Тихие часы: в этом окне автоматическая коррекция молчит, ручной fix работает.
quiet_hours_enabled = false
quiet_hours_start = "22:00"       # ЧЧ:ММ, начало (включительно)
quiet_hours_end = "08:00"         # ЧЧ:ММ, конец (не включительно)

## История исправлений для вкладки «История» (только метаданные: длина слова,
## языки, время; набранный текст не хранится).
history_size = 20

## Обучение включено всегда: приложение наблюдает, какие слова вы оставляете,
## какие исправляете и какие правки отменяете (Ctrl+Z / Shift+Backspace), и
## подстраивается. Сами слова держатся только в памяти демона; на диск попадают
## лишь счётчики (сколько слов увидено, исправлено, отклонено).
learning_enabled = true
learning_promote_threshold = 3   # сколько раз оставить слово, чтобы запомнить
learning_max_words = 2000        # верхняя граница выученного словаря

## Тема окна настроек: "system" (как в системе), "light" или "dark".
## На исправление текста не влияет — только на внешний вид окна.
theme = "system"
```

Слова исправляются **сразу** при нажатии пробела или Enter — ждать паузы не нужно. `analysis_timeout` — это лишь запасной таймер для слов, набранных без разделителя (например, длинного URL). Слова, которые являются частью URL, e-mail, пути или версии (`github.com`, `test@example.com`, `3.14`), не исправляются.

Клавиши, нажатые в короткое окно между удалением и вставкой текста, не теряются: демон ставит их в очередь и доигрывает после исправления.

Бренд, имя или техножаргон не исправляется

Частотные словари не знают брендов (`vercel`, `муксуд`), имён и жаргона, так что детектор молчит: он не уверен, что `муксуд` — это английское `vercel`. Научите слово — и конвертация станет для детектора «известной хорошей»:

```
klyrshift dict add vercel     # теперь муксуд → vercel срабатывает
klyrshift dict list           # показать слова
klyrshift dict remove vercel  # удалить слово
```

В GUI есть вкладка **«Словарь»**: добавление и удаление слов, поиск, импорт и экспорт файла, кнопка «Применить к демону» (перечитывает словарь без перезапуска). Файл — `~/.local/share/klyrshift/dictionary.txt`, по одному слову на строку. Научаемое слово никогда не переписывается, а его конвертация в другую раскладку считается верной. Демон сам никогда не пишет набранный текст в этот файл.

Расширенные частотные словари

Встроенный словарь ограничен значением `dictionary_size` (по умолчанию 5000 самых частых слов). Чтобы распознавать больше редких слов, подключите дополнительный список частотных слов — он расширяет только состав словаря, не меняя оценку n-грамм, поэтому уже верные слова остаются верными:

```
klyrshift dict download ru          # скачать топ-50k для русского
klyrshift dict download en          # поддерживаются: ru, en, uk, de, fr
klyrshift dict import-file ~/Downloads/ru-50k.txt --lang ru   # вручную
```

Файлы кладутся в `~/.local/share/klyrshift/dictionaries/<lang>.txt` и подхватываются при следующем запуске демона (или по хоткею перезагрузки конфигурации). Источник — FrequencyWords; его код под MIT, а сгенерированные списки слов — контент под CC-BY-SA-4.0. Зеркало списков лежит в репозитории в папке `dictionaries/`; условия указаны в data/copyright.

То же самое можно сделать в GUI: на вкладке **«Настройки»**, в разделе **«Языки»**, у каждого языка есть переключатель (использовать ли его) и кнопки **«Установить»** / корзина. Установка скачивает словарь в фоне (окно не зависает), удаление стирает файл и выключает язык. Единственный включённый язык удалить нельзя — демон должен распознавать хотя бы одну раскладку.

Тематические словари

Кроме базового списка можно подключить профессиональную лексику по разделам: `it`, `medicine`, `legal`, `finance`, `engineering`, `science`, `business`, `electronics`, `media`, `education`, `gaming`, `sport` (русский и английский). Разделы ставятся отдельно и **не** входят в `.deb`:

```
klyrshift dict categories                 # список разделов
klyrshift dict install it --lang ru       # скачать и подключить
klyrshift dict list-installed             # что уже включено
klyrshift dict remove-category it         # отключить
```

Приоритетные разделы (`it`, `medicine`, `science`) содержат **1000+ слов на язык**, остальные — от ~130 до ~330 (точные числа смотрите в `dictionaries/<раздел>/README.md` и на странице `/dictionaries`). Это реальные списки терминов: числа на сайте считаются из самих файлов при сборке и всегда совпадают с тем, что скачивается.

Включённые разделы хранятся в `installed_dict_categories` (по умолчанию `["base"]`). Приоритет при детекции: **пользовательский словарь → тематический → базовый → T9**. Каждый раздел публикуется как ассет релиза `klyrshift-dict-<категория>.tar.gz`; при установке он сначала тянется из релиза, затем из зеркала репозитория, затем из локальной копии — так `install` работает и офлайн. Каталог с разделами — на странице `/dictionaries`.

Ручной разворот раскладки

Если слово выглядит правильным, но набрано не в той раскладке, авто-детектор его не тронет. Хоткей `toggle_layout` (по умолчанию **Ctrl+Shift+T**) принудительно перепечатывает последнее слово в другой раскладке: `привет` → `ghbdtn`. Он работает и после пробела — демон помнит последнее сброшенное слово.

Исправление опечаток (T9)

Отдельно от раскладки Klyrshift умеет исправлять **опечатку внутри слова** (`langauge` → `language`, `teh` → `the`). Функция выключена по умолчанию: неверное исправление хуже, чем его отсутствие. Включается в GUI (вкладка «Дополнительно» → «Исправление опечаток (T9)») или в конфиге `typo_correction = true`.

Режим `manual` — это «только по команде». В нём Klyrshift не делает ничего автоматически: ни переключения раскладки, ни T9, ни пунктуации. Ручной fix (двойной Shift) и ручные хоткеи T9/пунктуации (`CTRL+SHIFT+F` / `CTRL+SHIFT+P`) работают всегда — это ваша команда. Если в каком-то приложении вы хотите автоматику, задайте ему режим через `app_modes` (например `telegram = "auto"`): для этого приложения автоматические проходы включатся. Отдельные переключатели `typo_mode`/`punctuation_mode` отвечают лишь за то, какой проход может работать (`off`/`manual`/`auto`/`hybrid`), но автоматически они срабатывают только в режиме `auto`/`hybrid`.

Правило одно, и оно намеренно строгое: слово заменяется, только если оно

- отсутствует в словаре языка,
- не короче `typo_min_word_length` символов,
- отличается от **ровно одного** слова словаря не более чем на `typo_max_distance` правок (расстояние Дамерау — перестановка соседних букв считается за одну правку).

Для длинных слов правило чуть шире: слово длиной от 6 символов может отличаться на `typo_max_distance_long` правок (по умолчанию 2), но только если лучший вариант заметно частотнее второго — иначе Klyrshift молчит.

Если ближайших вариантов два и более (или слово уже верное) — Klyrshift молчит. Научаемые слова и токены с разделителем (URL, e-mail, путь, `id-с-дефисом`) никогда не «исправляются». T9 работает только там, где раскладка признана верной, поэтому не спорит с авто-переключением.

После правки конфига перезапустите сервис или отправьте `SIGHUP`:

```
systemctl --user restart klyrshift.service
## или
kill -HUP "$(cat ~/.cache/klyrshift/daemon.lock)"
```

Раскладка по приложениям

Приложения, в которых вы почти всегда печатаете на одном языке, можно закрепить за раскладкой. При переходе фокуса на такое приложение Klyrshift включит его раскладку (не чаще, чем раз в ~0.35 с, и не повторно, пока фокус остаётся в нём — чтобы не спорить с ручным переключением).

```
app_layout_switch = true
[app_layouts]
kitty = "ru"
code = "us"
```

В GUI — вкладка **«Дополнительно» → «Раскладка по приложениям»**, с кнопкой «Определить» текущее приложение.

Режим по приложениям

Общий режим один на всю систему, но подходящий режим зависит от задачи: в редакторе кода текст лучше не трогать автоматически, а в мессенджере — наоборот. Список `[app_modes]` закрепляет режим за конкретной программой; приложения вне списка работают с общим `mode`.

```
[app_modes]
code = "manual"      # в редакторе кода — только по хоткею
telegram = "auto"    # в мессенджере — автоматически
```

```
klyrshift app-mode                 # показать список
klyrshift app-mode code manual     # задать режим приложению
klyrshift app-mode code            # какой режим действует для приложения
klyrshift app-mode code --clear    # убрать приложение из списка
```

В GUI — вкладка **«Дополнительно» → «Режим по приложениям»**. Режим выбирается из списка, поэтому недопустимое значение ввести нельзя. Определение активного приложения опирается на D-Bus GNOME Shell и может быть недоступно в других окружениях — тогда приложение работает с общим режимом.

Исключения: где Klyrshift не трогает текст

В терминале, редакторе или игре набранное — это команда, идентификатор или сообщение в чат, а не слово, поэтому «исправлять» его не нужно. Список `exceptions_apps` уже заполнен (терминалы, VS Code и подобные IDE, Steam и Wine); свои приложения можно добавить в GUI (вкладка **«Дополнительно» → «Исключения»**). Ручной fix (двойной

Shift) работает и внутри исключений. Если в каком-то приложении вы хотите и автоматику, и ручной fix, добавьте его в `exceptions_force_in_manual`.

Тихие часы

Чтобы Klyrshift не переключал раскладку в заданное время (например, ночью), включите `quiet_hours_enabled` и задайте `quiet_hours_start`/`quiet_hours_end` в формате `ЧЧ:ММ`. Окно может пересекать полночь (`22:00`–`08:00`). Ручной fix продолжает работать.

Обучение

Klyrshift включён «в режиме обучения» всегда: он незаметно наблюдает за тем, как вы печатаете, и подстраивается под вас.

- **Отмена = отказ.** Если вы отменяете правку (`Ctrl+Z` или хоткей отмены), приложение это видит и больше **не трогает это слово** до конца сессии. Так исправление, которое вам не нужно, не возвращается снова и снова. Отклонённое слово не исправляется автоматически ни в одной раскладке; ручной хоткей (двойной Shift) по-прежнему его исправляет — это ваша команда. Посмотреть список отклонённых слов: `klyrshift learn list-rejected`.
- **Что вы оставляете, то и правильно.** Слово, которое вы не исправляете достаточно часто (по умолчанию `learning_promote_threshold = 3`), попадает в словарь как «правильное» и перестаёт исправляться. Это автоматически учит Klyrshift вашим брендам, именам и жаргону, без ручного `klyrshift dict add`.
- **Приватность.** Сами слова живут только в памяти демона и никогда не пишутся на диск. Сохраняются лишь счётчики (сколько слов увидено, исправлено, отклонено). Посмотреть их: `klyrshift learn`; выключить обучение: `klyrshift learn --off`; сбросить счётчики: `klyrshift learn --reset`.
- **Ручная правка слов.** `klyrshift learn list` покажет, что демон выучил за сессию; `klyrshift learn promote <слово>` закрепит слово сразу, а `klyrshift learn forget <слово>` снова разрешит его исправлять (и снимет отклонение Ctrl+Z). Команды работают через локальный управляющий сокет (права `0600`), потому что сами слова живут только в памяти демона.

История исправлений

Вкладка **«История»** показывает последние исправления и позволяет отменить самое свежее. Хранятся **только метаданные** — длина слова, языки и время; сам набранный текст не сохраняется нигде. Глубина списка — `history_size`.

Сниппеты (текстовые сокращения)

В `~/.config/klyrshift/snippets.toml` можно задать сокращения; они разворачиваются **до** анализа раскладки:

```
[snippets]
"адр" = "ул. Ленина, д. 1"
"дт" = "{date}"
```

Включается через `text_expander_enabled = true` (GUI: **«Дополнительно» → «Сниппеты»**). Поддерживаются подстановки `{date}` и `{time}`.

Исправление уже выделенного текста

Выделите текст и нажмите `CTRL+SHIFT+L` — Klyrshift скопирует его, сменит раскладку и вставит обратно. Хоткей настраивается в GUI; функция включается через `selection_fix_enabled`.

Защита полей паролей

Если AT-SPI сообщает, что фокус в поле пароля, исправление пропускается, даже если слова нет в `stop_words`. Отключается через `password_guard = false`.

Проверка обновлений

Единственная функция, которая ходит в сеть, — и она **выключена по умолчанию**. Если включить `update_check_enabled = true` (GUI: **«Дополнительно» → «Проверять обновления»**), демон раз в сутки запрашивает у GitHub номер последней версии и, если вышла новее, пишет строку в лог и показывает уведомление. Отправляется только обычный `GET` публичного релиза — ни набранный текст, ни конфигурация не передаются.

Troubleshooting

Проблема	Решение
Не исправляет	Запустите `klyrshift doctor` — он проверит права, устройства и backend'ы
`Permission denied: /dev/input/...`	Перезагрузите udev-правило: `sudo udevadm control --reload-rules && sudo udevadm trigger`; проверьте `getfacl /dev/input/event3` — подробнее TROUBLESHOOTING: ACL после .deb
`doctor` из TTY или ssh	Это ожидаемо: тип сессии `unknown` — см. TROUBLESHOOTING: doctor из TTY/ssh
Переключает, но не заменяет текст	Смените `backend` на `uinput`
Исправляет то, что не нужно	Включены проверки `plausibility_check`, `structural_boundaries`, `identifier_guard`; добавьте строку в `custom_skip_regex` — подробнее TROUBLESHOOTING: ложные срабатывания
Нужно вернуть последнее исправление	Хоткей undo (`SHIFT+BACKSPACE`) или кнопка **«Отменить последнее исправление»** в GUI
Хочу посмотреть журнал	`klyrshift logs -f --level DEBUG` — набранный текст в журнал не попадает
Кнопка «включить» показывает битый значок	Обновите тему иконок или переустановите пакет: Klyrshift рисует символ питания сам, если темы нет
Не работает в терминале	Попробуйте `backend = "uinput"`
	`sudo apt install python3-gi gir1.2-gtk-4.0 gir1.2-adw-1` (в `.deb` это `Recommends`; `doctor` покажет точную команду)

Проблема	Решение
`klyrshift gui` пишет «нужны GTK4 и libadwaita»	
В туре не затемняется фон, кнопка включения без значка	`sudo apt install python3-gi-cairo` (без него `set_draw_func` не может получить `cairo.Context`)
Нет иконки в трее	Установите `python3-gi gir1.2-appindicator3-0.1`
Сообщаете о баге	Приложите `klyrshift collect-logs` — в архиве нет набранного текста
Конфликт с IBus/Fcitx	Отключите их, если не используете
Не переключает раскладку в Wayland	Установите `g3kb-switch`

Полностью: docs/TROUBLESHOOTING.md.

FAQ

Работает ли Klyrshift в полях пароля? Нет — Wayland не позволяет определить, что фокус в поле пароля. Используйте стоп-слова (`stop_words`), чтобы защитить конкретные строки.

Почему не сработало в игре? Некоторые игры и Java/Swing-приложения перехватывают ввод и игнорируют синтетические нажатия. Это известное ограничение Wayland.

Как добавить язык? Проще всего — в GUI: вкладка **«Настройки»** → раздел **«Языки»** → кнопка **«Установить»** у нужного языка. Так скачивается частотный словарь и язык включается. Из терминала: `klyrshift dict download <lang>` (например, `de`).

Новый язык с нуля (раскладка + n-граммы) добавляется в код: раскладка в `src/klyrshift/data/layouts.json` и файл `src/klyrshift/data/ngrams_<lang>.json` со словами и биграммами. См. docs/CONTRIBUTING.md.

Почему короткие слова не исправляются? У слов из 1–2 символов слишком мало статистического сигнала. Порог задаётся параметром `min_word_length`.

Как временно остановить Klyrshift? `systemctl --user stop klyrshift.service` или `klyrshift stop`.

Нужны ли root-права? Только при установке (udev-правило и systemd-сервис). Сам демон работает от пользователя.

Как удалить Klyrshift? Для `.deb`: `sudo apt remove --purge klyrshift`. Из исходников: `bash uninstall.sh` (добавьте `--purge`, чтобы удалить конфиг и логи).

Разработка / Development

```
bash scripts/dev_setup.sh     # venv + dev-зависимости
make check                    # ruff + black + mypy --strict + pytest --cov
```

```
make test                       # только тесты с покрытием (>80%)
make build-deb                  # собрать .deb в dist/
```

Команды и опции: docs/CLI.md (та же справка в `man klyrshift`). Архитектура и принятые решения: docs/ARCHITECTURE.md. Структура проекта (что где лежит): docs/PROJECT_STRUCTURE.md. Договор об отмене исправления: docs/UNDO_CONTRACT.md. Поддержка платформ (X11/Wayland, что проверено): docs/PLATFORM_SUPPORT.md. Офлайн-работа и приватность: docs/OFFLINE_AND_PRIVACY.md. Управляющий сокет демона: docs/CONTROL_SOCKET.md. Стратегия тестирования и метрики детектора: docs/TEST_STRATEGY.md. Формат языковых пакетов (версия, контрольная сумма, откат): docs/LANGUAGE_PACKAGES.md. Языки интерфейса (каталоги, fallback, плюрали, как добавить язык): docs/INTERFACE_LANGUAGES.md. Реализованные возможности (что уже готово): docs/FEATURES.md. Отчёт о выполненной работе (что сделано и с каким результатом): docs/WORK_REPORT.md. Проверка существующей реализации (что готово, по факту прогона): docs/VERIFICATION_REPORT.md. План развития: docs/ROADMAP.md. Ручной план тестирования на реальной системе: docs/TESTING.md. Автономный QA-движок (сценарии, инварианты, отчёт, регрессии): docs/QA_ENGINE.md. Почему ломается автопрогон (разбор дефекта QA-движка): docs/AUTOPROGRESS_BREAKAGE.md. Чек-лист выпуска (проверки пакета, словарей и живой клавиатуры): docs/RELEASING.md. Вся документация одним файлом: docs/klyrshift-handbook.pdf (PDF) или docs/klyrshift-handbook.md (Markdown); собирается из этих источников командой `make docs-pdf`.

⚠ Известные ограничения / Known limitations

- Демон не может определить поле пароля (ограничение Wayland) — митигация: стоп-слова.
- Точность на словах из 1–2 символов низкая — митигация: `min_word_length`.
- В некоторых играх и Java-приложениях замена текста может не работать.
- Доступ через `uaccess` применяется при входе в сессию: если пакет установлен в уже активной сессии, права появятся после следующего входа/перезагрузки.
- Переключение раскладки в GNOME Wayland требует `g3kb-switch`.
- Замена текста атомарна только на бэкенде `uinput`. На фолбэках `wtype` и `xdotool` удаление и ввод идут отдельными вызовами, поэтому при аварийном завершении посреди замены текст может остаться обрезанным. Используйте `uinput` (значение по умолчанию в Debian-пакете), где это возможно.
- Демон слушает все доступные клавиатуры; устройство с именем, содержащим `klyrshift` или `uinput`, намеренно пропускается (защита от петли).
- T9 исправляет только слова из встроенного частотного словаря. Бренды, названия компаний и техножаргон (`vercel`, `kubernetes`, имена) в него не входят — добавьте их через `klyrshift dict add <слово>`, иначе детектор промолчит.
- Языки `uk`, `de`, `fr` включаются вручную (`languages`) и проверены меньше, чем `en`/`ru`; по умолчанию активны только `en` и `ru`.
- Адаптивный таймер простоя — эвристика по скорости печати; при редком, неровном темпе он может сработать чуть раньше или позже. Отключается через `analysis_timeout_adaptive = false`.

Contributing

См. docs/CONTRIBUTING.md. Коммиты атомарные: `feat: …`, `fix: …`, `docs: …`.

📄 Лицензия / License

Код — MIT, см. LICENSE. Базовые частотные списки (`dictionaries/*-50k.txt`) — контент из FrequencyWords под CC-BY-SA-4.0; полные условия для обоих случаев — в data/copyright.

История изменений — в CHANGELOG.md. Баги и предложения — в Issues.

🙏 Благодарности / Acknowledgements

- gswitch — ручное исправление текста через `evdev`/`uinput` (идея и подход к замене).
- g3kb-switch — переключение раскладки в GNOME Wayland.
- PolterType — кросс-платформенный детектор раскладки (идеи по UX и per-app исключениям).
- python-evdev — чтение клавиатуры (`evdev`) и ввод текста (`evdev.UInput`).
- Идея вдохновлена Caramba Switcher для Windows.

🌐 Сайт

- **Основной сайт (RU):** https://klyrshift.layero.app
- **Зеркало (EN):** https://klyrshift.vercel.app

Оба сайта индексируются в поисковиках с per-host canonical и hreflang (`ru` / `en` / `x-default`), поэтому они не конкурируют как дубликаты.

Installation

Klyrshift targets Debian 12+ / Ubuntu 22.04+ with GNOME 45+ on either X11 or Wayland. Python 3.10 or newer is required.

Zero-config .deb path (recommended)

This is the path for regular users. Nothing has to be configured by hand:

```
sudo apt install ./klyrshift_0.2.10_all.deb
```

That is the whole procedure. The package:

- installs `/usr/lib/udev/rules.d/71-klyrshift.rules`, which grants access to `/dev/input/event*` and `/dev/uinput` to the active session user with `TAG+="uaccess"` — no `usermod`, no group membership, no relogin;
- loads the `uinput` module so `/dev/uinput` exists;
- enables the systemd user service **globally** (`systemctl --user enable --global klyrshift.service`), so it starts at the next login. It is not started during installation because there may be no graphical session yet;
- ships the man page (`man klyrshift`) and the bash and zsh completions, so `klyrshift <Tab>` completes commands and their options.

After the next login the daemon is running. Verify with:

```
klyrshift doctor
systemctl --user is-enabled klyrshift.service   # -> enabled
```

`uaccess` grants the ACL when the user logs in at the seat. If the package is installed while a session is already open, the ACL appears at the next login or after a reboot; that is expected and needs no manual command. If `doctor` still reports a permission problem after a relogin, see TROUBLESHOOTING: после установки .deb doctor жалуется на права /dev/input или /dev/uinput.

> Why `uaccess` and not the `input` group: the ACL is granted only to the user physically logged in at the seat, is applied dynamically and is revoked when the session ends. Permanent membership of the `input` group would give every member access to every keyboard, always.

Upgrading from the old `linguafix` package

The project was renamed twice — `LinguaFix` → `Lingvix` → `Klyrshift` — and the Debian package with it (`linguafix` → `klyrshift`). Install the new package the same way:

```
sudo apt install ./klyrshift_0.2.10_all.deb
```

apt sees `Conflicts: linguafix, lingvix` and `Replaces: linguafix, lingvix` in the new package and removes the old one first, so the two do not coexist. Do not end up with both: each package ships a systemd user unit, and two enabled units means two daemons reading the keyboard at once.

Verify that only the new package remains:

```
dpkg -l | grep -E 'linguafix|lingvix|klyrshift'   # only 'ii  klyrshift' expected
systemctl --user is-enabled klyrshift.service    # -> enabled
```

The configuration directory **is migrated** on first load. The old package used `~/.config/linguafix` (and, between the renames, `~/.config/lingvix`) while the new one uses `~/.config/klyrshift`; on the first start the new package copies `config.toml` and `snippets.toml` across. The copy happens once, only when `~/.config/klyrshift/config.toml` does not exist yet, and it leaves the old file in place — so an existing current config is never overwritten and a downgrade still finds its settings. To migrate by hand instead (or to check what was copied):

```
ls ~/.config/klyrshift/config.toml
## manual equivalent, if you prefer:
mkdir -p ~/.config/klyrshift
cp ~/.config/linguafix/config.toml ~/.config/klyrshift/config.toml
```

A `dpkg -i` install (rather than `apt install ./…`) also removes the old package, but leaves an empty `linguafix` record in dpkg's database and reports `1 not upgraded`. That is cosmetic — no files, no unit and no daemon remain. Clear it with `sudo apt-get -f install`, or by `sudo dpkg --purge linguafix`.

GNOME version and layout switching

On GNOME Wayland the active layout is changed through `g3kb-switch`, which talks to a GNOME Shell extension and is tied to the shell's D-Bus API. That API has changed between GNOME releases, so the supported range is **GNOME 45+ (tested on 48)**. `install.sh` prints a warning when it detects a GNOME version outside that range. If `g3kb-switch` is missing or incompatible, Klyrshift still replaces the text but cannot change the active layout.

`g3kb-switch` is **not in the Debian repositories** (verified: no candidate in Debian 12 or 13), so it is only a `Recommends`, never a `Depends` — apt would otherwise refuse to install the package. The X11 backend `setxkbmap` ships in `x11-xkb-utils`, which is in Debian and is recommended, so a default Debian install keeps a working layout backend. Check what is available with:

```
klyrshift doctor     # reports the detected layout backend
gnome-shell --version
```

Quick install (developer path)

This path is for contributors who work from a clone. Regular users should use the `.deb` above.

```
git clone https://github.com/Pavel1778/Klyrshift.git
cd Klyrshift
bash install.sh
```

The installer is idempotent and can be re-run safely. Pass `--yes` for a non-interactive run (useful in scripts):

```
bash install.sh --yes
```

To install without starting the daemon at login, pass `--no-autostart`. The daemon still runs for the current session; it is simply not enabled at login. Re-running the installer with this flag also removes an autostart entry a previous install wrote, so the opt-out always holds:

```
bash install.sh --no-autostart
```

Either way the autostart can be toggled later from the GUI switch, or with `klyrshift install-autostart` / `klyrshift uninstall-autostart`.

What the installer does

1. Checks that the host looks like Debian/Ubuntu (warns, but continues, if not).
2. Verifies Python 3.10+.
3. Installs system packages with `apt`: `python3-evdev python3-gi python3-gi-cairo gir1.2-appindicator3-0.1 wtype xdotool libnotify-bin`.
4. Optionally helps you install `g3kb-switch` for GNOME Wayland layout switching.
5. Creates a virtual environment in `~/.local/share/klyrshift/venv/` and installs the package with `pip install -e .`.
6. Installs the udev rule `/etc/udev/rules.d/71-klyrshift.rules` (asks for sudo). The rule grants access with `TAG+="uaccess"`, so no group change is needed.

7. Loads the `uinput` module.
8. Installs the systemd user service `~/.config/systemd/user/klyrshift.service`.
9. Installs the XDG autostart entry `~/.config/autostart/klyrshift.desktop` (skipped with `--no-autostart`).
10. Copies the icon to `~/.local/share/icons/hicolor/scalable/apps/klyrshift.svg`.
11. Installs the man page (`~/.local/share/man/man1/klyrshift.1`) and the bash and zsh completions (`~/.local/share/bash-completion/completions/klyrshift`, `~/.local/share/zsh/site-functions/_klyrshift`).
12. Enables and starts the service (with `--no-autostart`, starts it without enabling it at login).
13. Sends a desktop notification.

After installing (developer path)

The udev rule grants the ACL to the active session user. If the rule was just installed, reload it so the ACL is applied:

```
sudo udevadm control --reload-rules && sudo udevadm trigger
```

A full logout/login also applies it. No `usermod` and no group change is involved.

Verifying the install

```
klyrshift status
systemctl --user status klyrshift.service
journalctl --user -u klyrshift.service -n 50 --no-pager
```

`klyrshift status` should report that the daemon is running, the current layout, and the active injector backend.

Run the built-in self-diagnosis to check permissions, devices, tools, the GNOME version and the resolved backends at once:

```
klyrshift doctor
```

The output is a table with a marker per check (pass, ⚠ warning, failure) and a hint for every check that did not pass. The command exits non-zero when a check fails, so `install.sh` runs it as a final self-test.

To check the installed files themselves, not the environment:

```
klyrshift verify
```

The `.deb` ships an `md5sums` manifest (one digest per installed file), and `verify` compares the files on disk against it. A file that is missing is reported separately from one that was modified, because the fix is the same either way — reinstall:

```
sudo apt install --reinstall klyrshift
```

`dpkg --verify klyrshift` performs the same comparison using dpkg's own copy of the manifest. A source checkout or a `pip install` has no manifest; `verify` says so instead of claiming a pass.

Ubuntu notes

Some package names differ between Debian and Ubuntu. `install.sh` installs the APT packages one by one and skips any name that is not available in the current repository, so a missing package never aborts the install:

- The `uinput` backend uses `evdev.UInput` from `python3-evdev`, so the `python-uinput` package is not required. `python3-evdev` ships in Debian and Ubuntu; `install.sh` reuses it through `--system-site-packages`, so no compiler is needed.
- `wtype` and `g3kb-switch` may need a third-party repository on older Ubuntu releases. Without `wtype`, Klyrshift falls back to `xdotool` (X11) or `uinput`.

Manual install

If you prefer to install by hand:

```
python3 -m venv ~/.local/share/klyrshift/venv
~/.local/share/klyrshift/venv/bin/pip install -e .
mkdir -p ~/.config/systemd/user
cp data/klyrshift.service ~/.config/systemd/user/
systemctl --user daemon-reload
systemctl --user enable --now klyrshift.service
```

Then install the udev rule manually (it uses `uaccess`, so no group change is needed):

```
sudo cp data/71-klyrshift.rules /etc/udev/rules.d/
sudo udevadm control --reload-rules && sudo udevadm trigger
```

Uninstalling

For a `.deb` install:

```
sudo apt remove --purge klyrshift
```

For a source install:

```
bash uninstall.sh
```

Both stop and disable the service, remove the udev rule, the unit, the autostart entry and the icon, and delete the per-user configuration and logs.

Command-line reference

`klyrshift` is one binary with a subcommand for each action. Running it with no subcommand prints an error and the usage summary; `klyrshift --help` lists every command, and `klyrshift <command> --help` documents one.

This page is the prose reference. The man page (`man klyrshift`) and the shell completion are generated from the same argument parser by `scripts/build_cli_docs.py`, so all three describe the same commands; the test suite fails if any of them drifts.

```
klyrshift [--lang LOCALE] <command> [options]
```

Global options

Option	Meaning
`-h`, `--help`	Show the usage summary and exit.
`--version`	Print the version and exit.
`--lang LOCALE`	Language of the output. `auto` (the default) follows the environment; otherwise one of `ru`, `en`, `de`, `fr`, `uk`.

The output language only affects the messages this tool prints. It is separate from the dictionary language and from the interface language of the GUI; see Interface languages.

Daemon control

Command	What it does
`start`	Start the daemon. `--foreground` keeps it attached to the terminal; `--dry-run` analyses without ever switching the layout or replacing text.
`stop`	Stop the daemon (the service is not disabled; it returns at next login).
`kill`	Stop the daemon immediately, without the graceful shutdown `stop` performs.
`restart`	Stop and start again.
`status`	Show whether the daemon is running and a one-line summary of its state.

Correction behaviour

Command	What it does
`mode [auto|manual|hybrid]`	Show or set the global correction mode. `auto` corrects on its own, `manual` only on the hotkey, `hybrid` corrects on its own when it is confident and otherwise waits for the hotkey.
`app-mode [APP] [MODE]`	Show or set the mode for one application. `--clear` removes the override. An application without an override uses the global mode.
`undo`	Revert the last correction. See Undo contract.
`hotkeys`	Print the configured hotkeys.

Command	What it does
`fix [--text TEXT] [--apply] [--dry-run]`	Correct a string. With no `--text` it reads from standard input; with `--apply` it also rewrites the text on screen.
`explain [--text TEXT] [--layout LAYOUT]`	Explain what would be decided for a string and why, without changing anything. `--layout` states the current layout instead of detecting it.
`simulate SCENARIO [--json] [--layout LAYOUT]`	Replay a keystroke scenario through the real daemon with the keyboard and injector faked, and print what the user would see. The scenario is a comma-separated list: `type ghbdtn`, `space`, `enter`, `ctrl z`, `dblshift`, `layout ru`, `mode manual`, `config key=value`, `wait`. See Testing.

Learning

Command	What it does
`learn [list|list-rejected|promote|forget] [WORD]`	Inspect and adjust learned words. `list` shows them; `list-rejected` shows the words the user rejected with `Ctrl+Z` (never corrected again this session); `promote` pins a word so it is never rewritten; `forget` removes it (and clears a rejection).
`learn --on` / `--off`	Enable or disable learning.
`learn --reset`	Reset the learning counters.

Dictionaries

Command	What it does
`dict list`	List the installed dictionaries.
`dict add WORD` / `dict remove WORD`	Add or remove a word in the user dictionary.
`dict download LANG` / `dict import-file PATH`	Fetch or import a base dictionary. `--lang` names the language when the file name does not.
`dict categories`	List the available thematic categories.
`dict install CATEGORY` / `dict remove-category CATEGORY`	Install or remove a thematic dictionary.
`dict list-installed`	List the installed thematic dictionaries.
`dict pack`	Build a versioned language package (manifest + sha256). Options: `--lang`, `--file`, `--name`, `--kind`, `--output`, `--package-version`, `--license`.
`dict verify PATH`	Check a package against its manifest and checksums.
`dict install-file PATH` / `dict uninstall-file PATH`	Install or remove a package from a file.

See Language packages for the package format.

Configuration and data

Command	What it does
`config show` / `edit` / `reset` / `path` / `migrate`	Show, edit, reset or locate the configuration, or migrate it to the current schema. `migrate --dry-run` shows the changes without writing them.
`export [FILE]`	Write a backup (dictionaries and configuration) to `FILE`.
`import SOURCE`	Restore from a backup.

Diagnostics

Command	What it does
`doctor`	Run the self-diagnosis and print what is wrong and how to fix it.
`verify`	Verify the installed files against the package manifest.
`collect-logs [--output DIR]`	Collect a diagnostic archive for a bug report.
`logs [-n N] [-f] [--level LEVEL]`	Show the log. `-f` follows it in real time; `--level` filters to `DEBUG`, `INFO`, `WARNING` or `ERROR`.
`version`	Print the version.

Desktop integration

Command	What it does
`gui`	Open the settings window.
`install-autostart` / `uninstall-autostart`	Start the daemon at login, or stop doing so.

Shell completion

Bash and zsh completions ship with the package. For a manual install they live under `data/completions/`; `install.sh` copies them into `~/.local/share/bash-completion/completions/klyrshift` and `~/.local/share/zsh/site-functions/_klyrshift`.

```
## bash: no configuration needed once the file is in place
klyrshift dic<Tab>          # -> dict
klyrshift dict <Tab>        # -> the dict actions
klyrshift mode <Tab>        # -> auto manual hybrid
klyrshift --lang <Tab>      # -> auto ru en de fr uk
klyrshift --lang=e<Tab>     # -> --lang=en

## zsh: make sure the site-functions directory is on $fpath
fpath=(~/.local/share/zsh/site-functions $fpath)
```

Exit status

`0` on success, `1` when a command failed or the requested action was not possible. A diagnostic command such as `doctor` exits non-zero when it finds a problem, so it can be used in scripts.

Troubleshooting

Start here: `klyrshift doctor`

Before anything else, run the self-diagnosis. It checks the `input` group, read access to `/dev/input/event*`, `/dev/uinput`, the external tools (`g3kb-switch`, `wtype`, `xdotool`, `setxkbmap`), the GNOME version, the session type and the resolved switcher/injector backends, and prints a table with a hint for every failed check.

```
klyrshift doctor
```

The exit code is non-zero when a check fails, so it is safe to use in scripts.

`doctor` also carries a «Целостность пакета» row: for a `.deb` install it compares the installed files against the package's `md5sums` manifest, and for a source checkout it reports that there is nothing to check. To run just that check:

```
klyrshift verify
```

A file of the installed package is missing or was modified

`klyrshift verify` exits non-zero and names each file that no longer matches the `md5sums` manifest the `.deb` shipped. This is what a truncated download, a failed upgrade or an edit by hand looks like. The fix is the same for a missing and for a modified file:

```
sudo apt install --reinstall klyrshift
klyrshift verify     # now exits 0
```

`dpkg --verify klyrshift` performs the same comparison from dpkg's own copy of the manifest; its output uses a single-letter marker per changed attribute. A source checkout or a `pip install` has no manifest — `verify` says so and exits 0 rather than claiming a pass it cannot support.

Some distributions tell dpkg to drop files at install time. Ubuntu, for instance, excludes `/usr/share/man/*` and `/usr/share/doc/*` through `path-exclude` rules in `/etc/dpkg/dpkg.cfg.d/`, so the packaged man page and `CHANGELOG.md` never land on disk. `klyrshift verify` recognises those paths as excluded by policy — not missing — and still exits 0, printing how many files were skipped. `dpkg --verify` lists them as `missing` but also exits 0; a non-zero `klyrshift verify` therefore still means a real problem: reinstall the package as above.

The daemon does not fix anything

1. Check that it is running: `bash klyrshift status`
2. Look at the log: `bash tail -n 100 ~/.local/state/klyrshift/klyrshift.log`
3. Confirm the word-boundary triggers are enabled. Space and Enter flush the word and correct it immediately; Tab and punctuation are opt-in: `toml on_space = true on_enter = true on_tab = false on_punctuation = false` The boundary key is part of the correction: it is deleted and retyped, so it keeps its place. In an application that acts on Enter (a chat that sends on Enter), that means the key may be seen twice -- once by the application and once by the correction. Turn `on_enter = false` in such an application (or list it in

`exceptions_apps`) and let Space or the idle timeout flush the word instead. If you type a word without ever pressing a boundary, the idle fallback `analysis_timeout` (default `0.8` s) flushes it. With `analysis_timeout_adaptive = true` (the default) that timeout follows your typing speed — shorter when you type fast, longer when you type slowly — so you normally do not need to touch it. Set it to `false` to pin the timeout to the `analysis_timeout` value exactly.

4. Make sure the words you type are at least `min_word_length` characters long.
5. Note that Klyrshift deliberately does not rewrite URLs, e-mail addresses, file paths, version numbers or hyphenated identifiers (a token with `.`, `@`, `/`, `\`, `:`, `_` or `-` inside it). `github.com` and `3.14` are left alone.

A string I typed is "corrected" wrongly (false positive)

Symptom: a plausible-looking sequence such as `cb cj,jq?ye;yjn/g/` is rewritten into the other layout even though you meant to type it that way.

Three guards prevent this and are on by default:

```
plausibility_check = true      # don't touch text that already reads as a word
structural_boundaries = true   # skip URLs, e-mails, paths, versions
identifier_guard = true        # skip snake_case, camelCase, x86_64, ...
```

- If the text already looks like a real word **in the layout you typed it in**, it is not rewritten. This is what stops `cb cj,jq?ye;yjn/g/`.
- Tokens with structural markers (a dot, `@`, `/`, `_`, `-`, digits inside) are treated as identifiers and skipped.
- The guards run after the user dictionary, so a word you taught (`klyrshift dict add vercel`) is still fixed even if the source looks plausible.

If a specific string still misfires, add it to `custom_skip_regex`:

```
custom_skip_regex = ["^cb cj"]
```

Set `mode = "manual"` if you want to decide each correction yourself.

The fix went the wrong way and I want it back

Undo the last correction in either of two ways:

- Press the undo hotkey (`SHIFT+BACKSPACE` by default), or
- Open the GUI and click **«Отменить последнее исправление»** on the main page.

`CTRL+Z` is deliberately **not** the undo hotkey. The daemon does not grab the keyboard, so it cannot swallow the chord; if `CTRL+Z` were bound, the application's own undo would also fire and delete the corrected word without the daemon retyping the original — leaving the cursor in an empty spot. `SHIFT+BACKSPACE` is a chord no application uses for undo, so the daemon's replacement is the only one that runs. You can bind another key in the GUI if you prefer.

The GUI button asks the running daemon to reverse its own last fix, so it works even when the application you typed in groups undo history differently. It requires the daemon to be running.

doctor complains about /dev/input or /dev/uinput permissions

Symptom in the log:

```
Permission denied: '/dev/input/event3'
```

The packaged install grants access with a udev rule that uses `TAG+="uaccess"` (an ACL for the active session user), so the fix is almost always to make udev re-apply the rule, not to change group membership.

1. Check that the rule is installed and uses `uaccess`:

`bash cat /usr/lib/udev/rules.d/71-klyrshift.rules # or /etc/udev/rules.d/ # expected: KERNEL=="event*", TAG+="uaccess" # KERNEL=="uinput", TAG+="uaccess", OPTIONS+="static_node=uinput"`

1. Re-apply it and re-check the ACL:

`bash sudo udevadm control --reload-rules && sudo udevadm trigger getfacl /dev/input/event3 # expect a user:<you>:rw- line getfacl /dev/uinput`

If the ACL is missing, log out and back in: `uaccess` is granted when the user logs in at the seat.

1. Make sure `/dev/uinput` exists:

`bash sudo modprobe uinput && ls -l /dev/uinput`

1. If the rule is not there at all, reinstall the package: `sudo apt install --reinstall klyrshift`.

On the developer path (`install.sh`) the rule lives in `/etc/udev/rules.d/71-klyrshift.rules` and the same steps apply. `klyrshift doctor` accepts either mechanism — a `uaccess` rule or membership of the `input` group — and only fails when neither is present.

После установки .deb doctor жалуется на права /dev/input или /dev/uinput

Причина: правило с `TAG+="uaccess"` выдаёт ACL **в момент создания seat-сессии** (`systemd-logind`). Если пользователь поставил `.deb`, уже находясь в графической сессии, ACL для него ещё не создан — он появится при следующем входе. Это штатное поведение `systemd-logind`, а не баг пакета.

Диагностика:

```
getfacl /dev/input/event3    # ожидается строка  user:$USER:rw-
getfacl /dev/uinput          # ожидается строка  user:$USER:rw-
ls -l /usr/lib/udev/rules.d/71-klyrshift.rules
grep uaccess /usr/lib/udev/rules.d/71-klyrshift.rules
```

Если строки `user:$USER:rw-` нет — это и есть причина, применяем решение ниже.

Решение (от простого к сложному):

1. Выйти из GNOME и войти заново, затем проверить `klyrshift doctor`.

2. Если не помогло — перезагрузить правило udev и снова перелогиниться:

```
bash sudo udevadm control --reload-rules && sudo udevadm trigger
```

1. Если ACL всё равно нет — убедиться, что пользователь в **активной** seat-сессии, а не подключён по ssh:

```
bash loginctl show-user "$USER" | grep State # ожидается State=active
```

При `State=online` (ssh, cron, TTY без входа в GNOME) ACL не выдаётся по дизайну: доступ получает только пользователь, физически сидящий за машиной.

doctor запущен из TTY или ssh

Поведение: `klyrshift doctor` не падает, но строка «Тип сессии» становится предупреждением:

```
⚠  Тип сессии                    unknown (XDG_SESSION_TYPE не задан)
                                ↳ запустите doctor из графической сессии GNOME;
                                  из TTY/ssh переключение раскладки и замена текста
недоступны
```

Проверки раскладки и ввода при этом показывают `backend=none session=unknown`, потому что без сессии нет ни `g3kb-switch` / `setxkbmap` в нужном окружении, ни доступа к устройствам. Это ожидаемо.

Что делать: запускать `klyrshift doctor` из терминала внутри GNOME (не по ssh и не из TTY), либо заранее задать тип сессии:

```
XDG_SESSION_TYPE=wayland klyrshift doctor    # или x11
```

Для сбора диагностики с удалённой машины используйте `klyrshift collect-logs` — он фиксирует окружение и не требует графической сессии.

It switches the layout but does not replace the text

The injector backend is probably not able to synthesise input. Check which backend is active:

```
klyrshift status
```

- If it says `backend=none`, install `wtype` (Wayland) or `xdotool` (X11), or make sure `python3-evdev` is available and `/dev/uinput` is writable.
- If it says `backend=wtype` but nothing happens, the focused application may ignore synthetic Wayland input. This affects some games and Java/Swing apps.

Force a specific backend in `~/.config/klyrshift/config.toml`:

```
backend = "xdotool"
```

Nothing is corrected in the terminal

Terminals often use their own key handling. Try forcing the `uinput` backend, which goes through the kernel like a real keyboard:

```
backend = "uinput"
```

The tray icon is missing

The tray is optional and requires PyGObject with AppIndicator support:

```
sudo apt install python3-gi gir1.2-appindicator3-0.1
```

If GNOME does not show AppIndicator icons, install the "AppIndicator and KStatusNotifierItem Support" GNOME extension. If PyGObject is missing Klyrshift logs an INFO message and keeps running without a tray icon.

The on/off switch flips back to OFF

Symptom: you flip the big switch to ON, the spinner runs for a moment, and the switch returns to OFF without an error.

Cause: the switch starts a systemd **user** service. Older units set `Nice=-10` and `CPUSchedulingPolicy=rr`; a user service has no `CAP_SYS_NICE`, so systemd cannot apply them, treats the failure as fatal and the unit never starts. The switch correctly reports "not running" and flips back.

Check it:

```
systemctl --user status klyrshift.service
journalctl --user -u klyrshift.service -n 20
```

If the log mentions "Failed to set nice level" or "Operation not permitted", update to a build whose unit no longer requests nice/RT scheduling (v0.2.1+). If the unit is fine but still fails, the switch now shows the real reason in the toast; you can also start the daemon directly to see it:

```
klyrshift start --foreground
```

A daemon that exits with "No keyboard devices found" (wrong permissions, a container without `/dev/input`) is reported as a start failure, not as running.

The app icon is missing

Symptom: Klyrshift runs, but its launcher in the app grid shows a blank or generic icon.

Cause: the icon is an SVG. GNOME renders it through gdk-pixbuf, which needs the SVG loader from `librsvg2-common`. On a minimal install that loader is absent and the icon cannot be rasterised.

```
sudo apt install librsvg2-common hicolor-icon-theme
gtk-update-icon-cache -f -t ~/.local/share/icons/hicolor 2>/dev/null || true
```

Then log out and back in so the shell re-reads the icon. The `.deb` declares both packages as dependencies from v0.2.1 onward, so a normal install pulls them in automatically.

Conflict with IBus / Fcitx

IBus and Fcitx also intercept and transform keyboard input. Running them next to Klyrshift can lead to double conversion or missed corrections. If you do not use them for an input method, disable them; otherwise prefer a single layout source.

```
## IBus
ibus exit
## Fcitx
fcitx5-remote -e
```

The layout does not switch on Wayland

Klyrshift uses `g3kb-switch` on GNOME Wayland. Install it and verify:

```
g3kb-switch -p     # prints the current layout index
g3kb-switch -s 1   # switches to the second layout
```

If `g3kb-switch` is missing, Klyrshift can still replace text but cannot change the active layout.

A stray first letter remains after a correction (`phello`)

Symptom: you typed `hello` while the Russian layout was active, so the keys produced `руддщ`. After the correction the screen shows `phello` — the first character survived and the rest were replaced.

Cause: the deletion and the new text raced. The compositor (Chromium and Electron applications in particular) processes Backspace asynchronously, so the replacement was typed before the leading Backspace had been applied.

Klyrshift avoids this in three ways, all on by default:

- The daemon counts physical keys, not characters, so the number of Backspaces always equals the number of keys pressed.
- When the trigger is a word-boundary key (Space, Enter, Tab or punctuation), that key is already on screen after the word: the daemon deletes it too and retypes it after the corrected word, so it is never stranded in front of the text. (The deletion counts characters back from the caret, so a boundary that was left out of the count would eat the word's last character: `ghbdtn` + Enter -> `gпривет`. The `uinput` backend can type Space, Enter, Tab and punctuation back; a backend that cannot type the boundary leaves it in place, which is the safe failure.)
- The `uinput` backend flushes the backspaces, waits `backspace_settle_ms`, then types the replacement. Keys you press during that pause are queued and replayed afterwards, so nothing is lost.

If you still see a stray character, the application needs a longer pause. Two windows are involved: `trigger_settle_ms` (the pause after the Space/Enter that triggered the fix) and `backspace_settle_ms` (the pause between deleting and typing). Raise them:

```
trigger_settle_ms = 100     # 50 by default; pause after the boundary key
backspace_settle_ms = 100   # 80 by default; try 100-120 for slow Electron apps
```

A very large value only adds latency; it cannot corrupt the text, because the backspace batch itself is flushed atomically.

A word I type on purpose keeps being "corrected"

Jargon, names and abbreviations often look like a wrong-layout word. Instead of disabling the daemon, teach it the word:

```
klyrshift dict add котопёс        # adds to ~/.local/share/klyrshift/dictionary.txt
```

or edit `~/.local/share/klyrshift/dictionary.txt`, one word per line. A taught word is never rewritten, in any layout. `klyrshift dict list` shows the current words and `klyrshift dict remove <word>` deletes one. The GUI has a **Dictionary** tab with add/remove, search and file import/export; its "Apply to daemon" button re-reads the dictionary without a restart. Reload with a hotkey or restart so the daemon picks the file up.

Alternatively:

- `stop_words = ["котопёс"]` — a plain substring match, applied before detection.
- `ignore_all_caps = true` — never touch `ALL CAPS` tokens.
- `ignore_with_digits = true` — never touch a token containing a digit.
- `custom_skip_regex = "^https?://"` — a regular expression that disables a correction when it matches.

A brand or name is not corrected (`муксуд` stays `муксуд`)

Symptom: you typed a brand, name or technical term on the wrong layout (for example `муксуд` intending `vercel`) and Klyrshift did nothing, even though it corrects ordinary words.

Cause: the bundled frequency corpora contain common words only. A brand such as `vercel` is not one of them, so the detector cannot see that `муксуд` is the wrong-layout rendering of a real word and stays silent — that silence is the desired behaviour for unknown tokens (it protects names and jargon from false corrections).

Fix: teach the word. A taught word makes the conversion a known-good result:

```
klyrshift dict add vercel     # then typing муксуд  ->  vercel
```

A taught word is also never rewritten itself, so adding the brand does not make Klyrshift "correct" it away. The GUI **Dictionary** tab does the same without a terminal, and its "Apply to daemon" button reloads the daemon in place.

If you want to see why a word was skipped, run the daemon in the foreground with `log_level = "DEBUG"` and watch for the `detect(...)` lines; they report the scores, the chosen layout and the reason for skipping.

A one- or two-letter token is not corrected by the fix hotkey (`ϕт` stays `ϕт`)

Symptom: you typed a short token on the wrong layout (`ϕт`, meaning `an`) and a double Shift did nothing.

Cause: short tokens are only forced when the conversion is unambiguous — the result must be a known word of the target language and the token must not already be a word of the current one. A lone letter such as `a` is therefore left alone on purpose: a wrong one-character guess is worse than no guess.

Fix: teach the exact target, then the hotkey resolves it regardless of length:

```
klyrshift dict add an     # then ϕт + double Shift -> an
```

For a single character, teach the target too (`klyrshift dict add f` makes `a` + double Shift produce `f`).

A typo is not corrected (`langauge` stays `langauge`)

Typo correction (T9) is **off by default** — a wrong correction is worse than none. Turn it on in the GUI (**Advanced → Typo correction (T9)**) or in the config:

```
typo_correction = true
```

Even when enabled, it deliberately stays quiet unless the fix is unambiguous:

- the word must be at least `typo_min_word_length` characters (default 4);
- it must be within `typo_max_distance` edits (default 1) of a vocabulary word — if two words are equally close, nothing changes;
- a word at least 6 characters long may use `typo_max_distance_long` edits (default 2), but only when the best candidate is clearly more frequent than the runner-up;
- a word you taught (`klyrshift dict add ...`) or a token with a separator (`id-with-dash`, `a.b`, `x@y`) is never touched;
- a word whose layout is wrong is handled by the layout switcher first; T9 only runs when the layout is already correct.

If a specific typo is missed, either add the intended word to the dictionary (`klyrshift dict add language`) or raise `typo_max_distance` (or `typo_max_distance_long` for long words) to `2`. Raising it increases the chance of a wrong correction, so prefer the dictionary when you can.

Correct words keep getting rewritten ("T9-спам")

If T9 rewrites words that are already correct — and it repeats every few seconds — the automatic pass is running when you do not want it to. Two settings are involved:

- the working `mode` (`auto` / `manual` / `hybrid`) decides **whether** anything runs automatically. `manual` means "only when I ask": it silences the layout pass and the T9/punctuation passes;
- `typo_mode` (`off` / `manual` / `auto` / `hybrid`) decides **which** T9 pass may run:

```
typo_mode = "manual"   # off | manual | auto | hybrid
```

- `off` — T9 never runs;
- `manual` — T9 runs only on the explicit hotkey (default `CTRL+SHIFT+F`);
- `auto` / `hybrid` — T9 runs on a word boundary (Space / Enter), but only when the working `mode` is also `auto` or `hybrid`.

So a user who wants automatic layout switching but hotkey-only typo fixing sets `typo_mode = "manual"` and leaves `mode = "auto"`. A user who wants a fully quiet app sets `mode = "manual"`: nothing is rewritten until an explicit hotkey, whatever `typo_mode` says. On the **T9** tab of `klyrshift gui` each feature has its own mode combo.

If T9 still rewrites a specific correct word, add it to the dictionary (`klyrshift dict add загадка`) — a word known in any loaded language is never touched, so this both silences the false positive and teaches the word.

Correction does not trigger for a newer language (uk/de/fr)

By default only `en` and `ru` are active. The Latin layouts (`us`, `de`, `fr`) share physical positions, so loading all corpora at once makes them compete. Enable the languages you actually type:

```
languages = ["en", "ru", "uk"]
```

A language is only a conversion target when its corpus is bundled (`ngrams_<lang>.json`) and its layout is in `layouts.json`; a missing corpus is skipped rather than treated as an empty model.

Bigger vocabulary for a language

The bundled word list is the `dictionary_size` most frequent words (5000 by default). If real words in your language are still not recognised, add the extended top-50k list — it widens the vocabulary without changing the n-gram scoring, so nothing that already worked stops working:

```
klyrshift dict download ru      # ru, en, uk, de, fr
klyrshift dict download uk
```

The file lands in `~/.local/share/klyrshift/dictionaries/<lang>.txt`. Restart the daemon (or press the reload-config hotkey) to load it. You can also import a list you already have: `klyrshift dict import-file ~/Downloads/uk-50k.txt --lang uk`.

I type on purpose in two languages and the daemon fights me

Turn on context analysis (on by default) and give it a known preceding word; or switch the daemon to `mode = "manual"` and drive it with the fix hotkey. See the modes and hotkeys section above.

The daemon corrects in a specific application I do not want touched

Add the application to the exception list:

```
exceptions_apps = ["code", "gnome-terminal"]
```

Listed applications are skipped in every mode. To keep everything automatic except a few applications while in `mode = "manual"`, use `exceptions_force_in_manual` instead. The focused application is resolved best-effort; when it cannot be determined the daemon falls back to automatic behaviour.

The list is pre-filled with terminals, IDEs and games. If a word you type on purpose in one of them should still be correctable with the manual hotkey, that already works: the manual fix (`SHIFT+SHIFT`) ignores the exception list.

Automatic correction is silent for a while (quiet hours)

If automatic correction stops working at a regular time of day, check `quiet_hours_enabled` and the `quiet_hours_start` / `quiet_hours_end` window (`ЧЧ:ММ`, may wrap midnight). Inside it the daemon skips automatic fixes; the manual fix still works. The GUI home tab shows a line explaining the pause.

The "История" tab is empty

The history is refreshed on demand: while the tab is visible the GUI asks the daemon (SIGUSR2) to write `~/.cache/klyrshift/history.json` and then reads it. The poll is paused while the tab is hidden, so a background window does not signal the daemon every few seconds. An empty tab means no fix has been made yet, or the daemon is not running (the last snapshot is shown otherwise). The file holds metadata only — word length, layouts, timestamp — never the typed text.

Text looks deformed while typing fast

Symptom: characters are duplicated, dropped or reordered while you type quickly (for example `нет` becomes `ннет`, or `базу и` becomes `byar b`).

This is a race between the synthetic Backspace batch and the replacement text: the application must apply the deletion before the new characters arrive, and a layout correction must actually move to another layout.

What Klyrshift does about it:

- `backspace_settle_ms` (default 120 ms) is the pause between the Backspace batch and the retyped text; raise it if your application is slow.

- A correction that stays in the same layout is refused — it is never a real fix, only a source of deformation.
- Keys pressed during a replacement are queued and replayed afterwards, in order.

To diagnose, run the daemon in the foreground with `log_level = "DEBUG"` and watch the `Fixing buffer ... reason=...` lines, then report what you see with `klyrshift collect-logs`.

A letter is left behind (`phello` instead of `hello`)

Symptom: after a correction the first (or last) character of the old word is still on screen — `руддщ` becomes `phello` instead of `hello`, or a stray letter survives at the end of a fix.

There are two distinct causes and Klyrshift addresses both.

1. The deletion count and the injection raced. The Backspace batch and the replacement text are written as two separate flushes: the Backspaces go out with one `syn`, then `backspace_settle_ms` later the replacement follows with a second `syn`. The daemon counts physical keys (scancodes), not characters, so the number of Backspaces always equals the number of keys that produced the word — even if the compositor has not yet painted them. If your application still applies the deletion late, raise `backspace_settle_ms` (default 120 ms) in **Advanced**.

2. The buffer mixed two scripts inside one word. If you switch layout mid-word you produce a token like `ghbdtnпривет`. Converting that token as a unit would damage the half that is already correct (`ghйпривет`). Klyrshift therefore refuses to rewrite a token that mixes Latin and Cyrillic; the space (or Enter) that ends the token flushes each half separately. If you see a half-converted token it means the two scripts were typed as one word — add the separator and each half is corrected on its own boundary.

Both cases are pinned by tests: `tests/test_boundary_flush.py` for the deletion count and `tests/test_mixed_script_word.py` for the mixed-script rule.

Backspace does not delete in the terminal (or deletes too much)

Some terminals handle synthetic Backspace differently from real Backspace. Symptoms: the old text stays, or the correction eats characters it should not.

- Force the kernel path, which behaves like a physical keyboard: `toml backend = "uinput"`
- If you use `xdotool`, `--clearmodifiers` is already passed; a stuck modifier from the surrounding session can still interfere. Press and release both Shift keys once, then retry.
- In `vim`/`nano`, the correction happens in normal mode. Klyrshift only rewrites what it believes is an input buffer; in a full-screen editor that assumption may not hold. Use `min_word_length` to reduce false triggers, or add the surrounding words to `stop_words`.

Permission denied on /dev/uinput

```
Could not create uinput device
PermissionError: [Errno 13] Permission denied: '/dev/uinput'
```

The `uinput` backend needs write access to `/dev/uinput`. Check and fix:

```
ls -l /dev/uinput
sudo modprobe uinput                      # load the module if missing
ls -l /etc/udev/rules.d/71-klyrshift.rules # udev rule installed?
sudo udevadm control --reload-rules && sudo udevadm trigger
```

The udev rule sets `GROUP="input", MODE="0660"`. Make sure you are in the `input` group and that you have logged out completely since being added. When `uinput` is unavailable, Klyrshift falls back to `wtype` or `xdotool`; set `backend = "auto"` to allow that.

uinput created the device but no text appears

The kernel accepted the virtual keyboard but the compositor did not route its events to the focused window. This happens when the virtual device is created after the window grabbed the keyboard, and in some games that read the device directly. Switch the window focus away and back, or force `wtype` / `xdotool`:

```
backend = "wtype"
```

IBus / Fcitx conflict in detail

IBus and Fcitx sit between the kernel and the application, so a keystroke can be consumed twice: once by Klyrshift (reading `/dev/input`) and once by the input method. Symptoms: characters appear twice, the layout flips twice, or nothing is corrected.

Diagnose:

```
echo "$GTK_IM_MODULE $QT_IM_MODULE $XMODIFIERS"
pgrep -a ibus; pgrep -a fcitx
```

If you do not need the input method (for example, you only switch between `us` and `ru` with GNOME's own layout switcher), disable it:

```
ibus exit                 # IBus
fcitx5-remote -e          # Fcitx5
```

For a permanent change, unset `GTK_IM_MODULE` / `QT_IM_MODULE` in your session environment. Do not run Klyrshift and an input method that performs the same correction at the same time.

Several keyboards: text typed on the "other" one is not corrected

The daemon listens to every keyboard it can find, not just the first one. If a keyboard is present but its events never reach the daemon, check:

- The device is not filtered out. Devices whose name contains `klyrshift` or `uinput` are skipped on purpose to avoid a feedback loop; a keyboard with such a name is ignored.
- You have read access to that device's `/dev/input/event*` node (see the permission section above). A keyboard that appears after login may need the session to be refreshed (`loginctl terminate-session $XDG_SESSION_ID`) for the `uaccess` ACL to apply.
- Run `klyrshift doctor` and check the "Устройства /dev/input/event*" row.

The daemon stopped after unplugging a USB keyboard

Expected and handled. When the last keyboard disappears the daemon logs `Device read error (device removed?)`, drops that device and re-runs discovery every 2 seconds until a keyboard is back. If recovery ever fails the systemd unit has `Restart=always`, so the service is restarted with a fresh device list (the path is rediscovered, never cached). To confirm:

```
systemctl --user status klyrshift.service
journalctl --user -u klyrshift.service -n 50 --no-pager
```

`klyrshift stop` does not stop the daemon

`stop` sends `SIGTERM` and waits up to 2 seconds. If the process is wedged (for example blocked in a subprocess), `stop` escalates to `SIGKILL` automatically and prints a warning. To skip the graceful attempt entirely:

```
klyrshift kill
```

Corrections trigger while typing a password

On Wayland the daemon cannot see which window has focus, so it cannot tell a password field from a text field. Mitigations:

- Keep the default `stop_words` (`password`, `login`, `token`, `secret`, ...).
- Add the words that appear around your passwords to `stop_words`.
- Raise `min_word_length` so short passwords are ignored.
- Disable Klyrshift while entering credentials: `klyrshift stop` and `klyrshift start` afterwards.

Privacy: what Klyrshift does and does not store

Klyrshift never writes the text you type to disk. The log records only metadata (buffer length, layout names). The tests in `tests/test_privacy_audit.py` enforce this: they run a session with a distinctive secret string and then search the log, cache, state directory, config file, systemd journal and every subprocess argv for any trace of it.

Two consequences worth knowing:

- The desktop notification is a fixed label ("Раскладка исправлена"), not the corrected text, so it never appears in the notification history.
- If a fix fails, the exception traceback is redacted of the buffer before it is logged.
- The correction history (`~/.cache/klyrshift/history.json`) stores only the word length, the layouts, a timestamp and an `undone` flag — never the typed text.

Collecting diagnostics

The fastest way is one command that gathers everything:

```
klyrshift collect-logs
```

It writes `~/klyrshift-logs-YYYYMMDD-HHMMSS.tar.gz` containing the `doctor` output, the configuration, the log tail, the user journal, environment details, device permissions and package versions. Attach it to the GitHub issue. The daemon logs only metadata, so the archive never contains the text you typed.

Manual equivalent:

```
klyrshift status
systemctl --user status klyrshift.service
journalctl --user -u klyrshift.service -n 100 --no-pager
tail -n 100 ~/.local/state/klyrshift/klyrshift.log
```

Platform support

What Klyrshift supports today, on which platform, and what it does not. The table separates **implemented and tested** from **implemented but unverified on real hardware** from **not implemented**. Nothing here claims support that has not been exercised.

Verification status legend

Mark	Meaning
✅	Implemented; covered by automated tests
🧪	Implemented; automated tests only, needs a real session/keyboard to confirm
❌	Not implemented
⛔	Technically not possible on this platform as designed

Linux

Capability	X11	Wayland (GNOME)	Notes
Wrong-layout detection	✅	✅	Pure logic; platform-independent
Automatic text replacement	🧪 `xdotool` / `uinput`	🧪 `uinput` / `wtype`	Needs `/dev/input` + `/dev/uinput`
Atomic replacement	🧪 `uinput` only	🧪 `uinput` only	`wtype`/`xdotool` delete and retype separately
Layout switching	🧪 `setxkbmap`	🧪 `g3kb-switch`	`g3kb-switch` is not in the Debian repos
Password-field detection	✅	✅	Both hide the focused widget from unprivileged clients
Focused-application exceptions	✅	✅	Best-effort; depends on the compositor
System tray	✅	✅	Optional AppIndicator
GUI settings (GTK4 + libadwaita)	✅	✅	Runs under either session

Capability	X11	Wayland (GNOME)	Notes
Autostart	✅	✅	systemd user service

The daemon does not grab the keyboard on either session. It reads `evdev` events and injects through `uinput`; this is why it never swallows a chord (see UNDO_CONTRACT.md).

Distribution formats

Format	Status	Notes
`.deb`	✅ Built and released	Self-contained: udev rule, systemd user service, `uinput` module load
Source install (`install.sh`)	✅	Installs into a venv under `~/.local/share/klyrshift`
Flatpak	❌ Not implemented	Investigated as a future option; needs a portal-based input path
AppImage	❌ Not implemented	Needs the same input path as Flatpak

A distribution format is only listed as supported once it has been exercised on a real image; see `docs/RELEASING.md` for the checks that gate a release.

Languages

Language	Layout conversion	Frequency dictionary	Detector	Status
`en`	✅	✅ `en-50k`	✅	Default
`ru`	✅	✅ `ru-50k`	✅	Default
`uk`	✅	✅ `uk-50k`	✅	Manual enable; less tested than `en` / `ru`
`de`	✅	✅ `de-50k`	✅	Manual enable; less tested
`fr`	✅	✅ `fr-50k`	✅	Manual enable; less tested

Adding a language is a data change: a word list, an n-gram model, and an entry in the layout map. See the extension points in ARCHITECTURE.md.

Other operating systems

Platform	Status	What it would need
Windows	❌ Not implemented	A `WH_KEYBOARD_LL` hook and `SendInput` adapter; the detector core is reusable
macOS	❌ Not implemented	Accessibility permissions and a CGEvent tap; the detector core is reusable
Android	❌ Not implemented	A separate IME built on `InputMethodService`; the detector core is reusable

Platform	Status	What it would need
ChromeOS / BSD	□ Not implemented	Not investigated

The detection core (`detector.py`, `converter.py`, `typo.py`, `dictionary.py`) has no GTK, evdev, or session dependency and is the part worth reusing. The input adapters (`injector.py`, `switcher.py`, `app_focus.py`) are platform-specific by nature. No claim of cross-platform support is made: the core compiling elsewhere is not the same as the product working elsewhere.

Environment this matrix was written against

- Debian 13 (trixie) — the development container runs this.
- The container has **no GNOME, no display, no `/dev/input`, no `/dev/uinput`**, so every □ row is unverified here. The □ marks are honest, not optimistic: they mean "the code and its tests exist, a real session has not confirmed it".

What a real Debian 13 container does verify

The package layer is checked against real `debian:13` containers (Docker), not the dev container alone. These run in CI and were re-run for this revision:

- `scripts/smoke_test.sh` with `SMOKE_IMAGE=debian:13` — the `.deb` installs, the daemon starts, `doctor` reports `backend=setxkbmap` with no `g3kb-switch` present, and the data files load. **Passed.**
- `scripts/upgrade_test.sh` (image `debian:13`) — the pre-rename `linguafix` package is displaced, the old unit file is removed, exactly one daemon unit is enabled, and `mode=manual` survives the rename. **Passed.**
- `scripts/deb_selfsufficiency_test.sh` with `SMOKE_IMAGE=debian:13` — no manual step is needed (no group change), reinstall is idempotent, and `apt remove --purge` leaves nothing behind. **Passed.**

What these still cannot reach is the kernel input path and the GNOME session: `/dev/uinput`, `/dev/input/event*`, the live layout switch and the real focus tracking need a seat. Those stay □ and are what `scripts/acceptance_check.sh` is for on a physical machine.

Language packages

A language package is how a dictionary or an n-gram model travels between machines. It is a single `.tar.gz` carrying one or more data files plus a `manifest.json` that describes them and records a SHA-256 for each. The format gives the dictionary layer the four properties the roadmap asks for: integrity, provenance, version compatibility, and an atomic install that rolls back.

Nothing in a package is typed text. A package is a curated word list or a built n-gram model, exactly like the lists the bundled dictionaries already ship. See OFFLINE_AND_PRIVACY.md.

Layout

```
ru-50k.tar.gz
├── manifest.json
└── ru-50k.txt
```

`manifest.json`:

```
{
  "format": 1,
  "name": "ru-50k",
  "kind": "frequency",
  "language": "ru",
  "version": "1.0.0",
  "license": "MIT",
  "source": "klyrshift dict pack",
  "created": "2026-10-19T00:00:00Z",
  "files": [
    { "name": "ru-50k.txt", "sha256": "…", "bytes": 512345, "words": 50000 }
  ]
}
```

Field	Meaning
`format`	The package-format version. An installer refuses a package whose `format` is newer than it understands.
`kind`	`frequency`, `thematic` or `ngrams`. It decides the install directory.
`language`	The language the data is for.
`version`	The package's own version, for provenance and updates.
`license`	SPDX id or a short licence note. A package without one can still be installed, but the field is where provenance lives.
`source`	Where the data came from.
`files[].sha256`	The member's SHA-256, checked before anything is written.
`files[].name`	A plain file name. A name containing a path separator is rejected, so a package cannot write outside its directory.

Install locations

`kind` decides where a member lands, relative to the extended-dictionary directory (`extended_dictionary_dir`, default `$XDG_DATA_HOME/klyrshift/dictionaries`):

`kind`	Directory
`frequency`	`<dir>/` (for example `<dir>/ru-50k.txt`)
`thematic`	`<dir>/thematic/<lang>/`
`ngrams`	`<dir>/ngrams/<lang>/`

Commands

```
## Build a package from one or more word lists.
klyrshift dict pack ru-50k.txt --name ru-50k --lang ru --license MIT \
    --output ru-50k.tar.gz

## Check a package before trusting it.
klyrshift dict verify ru-50k.tar.gz

## Install it (verified, atomic, with rollback).
klyrshift dict install-file ru-50k.tar.gz

## Remove the files a package installed.
klyrshift dict uninstall-file ru-50k.tar.gz
```

`dict pack` accepts extra files with `--file <path>`, a package version with `--package-version`, and a kind with `--kind`.

What is enforced

- **Integrity.** Every member is hashed and compared to the manifest before a single byte is written. A truncated download or a tampered archive is refused with a non-zero exit code and nothing is changed on disk.
- **No undeclared members.** An archive carrying a file the manifest does not describe is refused: an unexpected member would otherwise be written unverified.
- **No path traversal.** A member name must be a plain name; `../etc/passwd` is rejected at manifest-parse time.
- **Bounded size.** A member larger than 64 MiB, or a manifest listing more than 256 files, is refused, so a crafted archive cannot exhaust the disk.
- **Atomic install with rollback.** Each member is staged next to its destination and swapped in one at a time; the previous file is moved aside first. If any later step fails, every file already swapped is restored and the staging files are removed. A failed update leaves the previous dictionaries exactly as they were — this is the "rollback on a failed update" the roadmap asks for.
- **Reproducible archives.** A fixed input produces a byte-identical archive (fixed mtime, no embedded file name, sorted members), so a checksum can be published and verified.

Relationship to `dict download` / `dict install`

The older commands fetch a single word list from the project mirror, the upstream source, or the bundled copy, and are unchanged. `dict pack` / `verify` / `install-file` are the versioned transport: they add the manifest, the checksums, the provenance fields and the atomic rollback. A future release can publish its bundled lists as packages without changing the download order, because a package installs into the same paths.

Tests

`tests/test_langpack.py` pins the format: round-trip build/read/verify/install, reproducible output, a tampered member refused before any write, an undeclared member refused, rollback restoring the previous file when a later commit fails, a newer `format` refused, a path-traversal name

refused, and the module never importing a network client. `tests/test_cli.py` covers the four commands end-to-end.

Interface languages

Klyrshift detects text typed in the wrong keyboard layout. That is a problem that has nothing to do with the language a menu is written in, so the interface language is a separate setting from the languages the detector knows. This document explains how the interface is translated, which languages ship, and how to add one.

What is translated, and what is not

Only the interface is translated: menu items, setting labels, explanations, notifications and CLI output. Three things are deliberately left alone.

- **The Russian source strings are the keys.** Every user-facing string in the source is written in Russian, and the catalogues map that Russian text to a translation. There is no invented message id, so a reader of the source sees the sentence that is actually shown, and a missing catalogue entry degrades to Russian rather than to a placeholder like `settings.title`.
- **Words the detector processes are never translated.** A dictionary entry, a stop word or a snippet is data, not interface. Translating it would change what the daemon corrects.
- **Diagnostic output that a bug report depends on** stays in one form. The layout tables, the `doctor` check identifiers and the manifest fields are matched literally by tests and by support, so they are not localised.

The languages that ship

Code	Language	Catalogue	Plural forms
`ru`	Русский	source (no file)	3
`en`	English	`data/locales/en.json`	2
`de`	Deutsch	`data/locales/de.json`	2
`fr`	Français	`data/locales/fr.json`	2
`uk`	Українська	`data/locales/uk.json`	3

The list lives in one place, `klyrshift.i18n.SUPPORTED_LOCALES`. The config validator (`config.VALID_UI_LOCALES`) and the GUI selector (`gui.settings_page.UI_LOCALE_ORDER`) both derive their lists from it, so a new catalogue cannot appear in one and be rejected by another. A test asserts this.

How complete the catalogues are

A new catalogue is not expected to be complete on the first commit. German, French and Ukrainian each carry 381 of the 675 source strings (56%), which covers everything the settings window, the CLI and the first-run tour and context help render. The remainder — the tray labels and some daemon messages — is not translated yet and falls back to English. The English

catalogue is the complete one and a test fails the build if a source string is missing from it, so a gap can never be a missing key, only an untranslated one.

Area	Covered in `de` / `fr` / `uk`
Settings window (all pages and widgets)	yes
CLI output (`hotkeys`, `status`, `doctor`, ...)	mostly
First-run tour and context help	yes
Tray, some daemon messages	no (falls back to English)

Choosing a language

There are three ways, in order of precedence.

1. **`--lang <code>`** on any CLI invocation localises that one run: `klyrshift --lang de hotkeys`. This also works for `--help`, which is built before the flag is parsed.
2. **The `ui_locale` setting** (`[general] ui_locale` in `config.toml`, or the "Interface language" combo on the settings page). The value `auto` follows the desktop; any other value forces a shipped catalogue. It is applied at startup without a restart.
3. **`auto`** reads the environment, honouring `LANGUAGE`, `LC_ALL`, `LC_MESSAGES` and `LANG` in that order. A value like `ru_RU.UTF-8` or `de_AT@euro` is reduced to its language part; an unsupported one falls back to Russian.

The combo lists every language **in its own script** (`Deutsch`, `Français`, `Українська`) rather than in the currently active language. A user who cannot read the current interface language is exactly the user who needs the selector, and "German" written in Russian does not help them.

Falling back

A catalogue is translated incrementally, so it may not cover every string. When an entry is missing the lookup walks a chain before giving up:

```
active locale  →  en  →  Russian source
```

German, French and Ukrainian therefore fall back to **English**, not to Russian. An incomplete German build shows an English label here and there; it never shows a German speaker a Cyrillic sentence. `en` and `ru` are terminals: the chain is empty and the source text is used. `i18n.LOCALE_FALLBACKS` holds the chains and a test forbids a self-referential entry.

The same principle governs plurals. `i18n.plural_form(count, locale)` returns a form index for a list-valued entry. Russian and Ukrainian use three forms (1, 2–4, 5+); English and German use two; French uses two but treats zero as singular ("0 mot"), which is why it is not folded into the default branch.

Adding a language

1. Add the code to `SUPPORTED_LOCALES` in `src/klyrshift/i18n.py`.
2. Add its native name to `LOCALE_NAMES` and its fallback chain to `LOCALE_FALLBACKS`.
3. Add `data/locales/<code>.json`. Start from `en.json` and translate the strings the interface renders; leave the rest out and the chain fills them.

4. Run `tests/test_i18n.py`. It fails the build when a catalogue entry is orphaned (a key that is not a source string), when a plural helper mis-selects a form, and when the config or GUI list drifts from the registry.

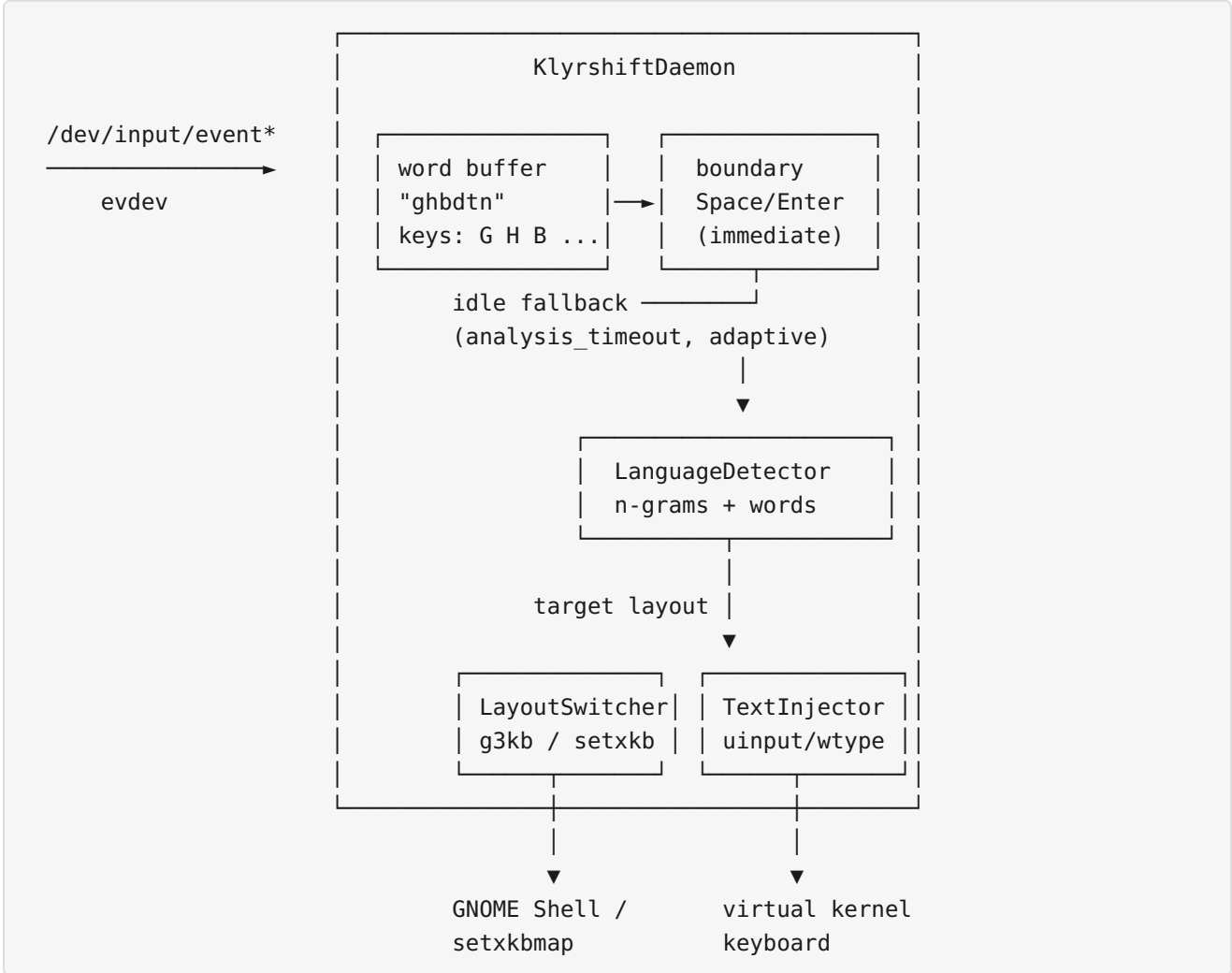
Nothing else changes: the combo, the validator and `--lang` all read the registry. A catalogue that is valid JSON but wrong is a cosmetic defect, never a crash — every lookup falls back to the source string.

Architecture

This document explains how Klyrshift is put together, why the individual components were chosen, and where the boundaries of the design are.

Overview

Klyrshift is a background daemon with no main window. It observes raw keyboard events, keeps the word currently being typed as a sequence of physical keys, and — the moment the user presses a word boundary (Space or Enter) — decides whether the word was typed in the wrong layout. When it is confident, it rewrites the word and switches the active layout, all within that one keystroke.



Data flow

1. **Capture.** The daemon opens every keyboard-like device under `/dev/input/event*` and waits in a `selectors` event loop. Only `EV_KEY` events are considered; modifiers and `EV_MSC` (scancode) events are ignored.
2. **Buffer.** Each printable key appends both its character (for detection) and its evdev code (for deletion) to the word buffer. Backspace removes the last character and its key. The buffer tracks the word, so it is flushed at every word boundary.
3. **Trigger.** Space and Enter flush the buffer immediately (Tab and punctuation are opt-in). The correction therefore happens inside the same keystroke that ends the word — there is no visible pause. An idle fallback (`analysis_timeout`, default 0.8 s) catches words typed without a separator, such as a long URL. With `analysis_timeout_adaptive` (on by default) that timeout follows the user's typing speed — shorter for a fast typist, longer for a slow one — clamped to 0.3–2.0 s.
4. **Analyse.** The buffer is handed to the detector, which tokenises it, scores each word against the Russian and English n-gram/word corpora and picks the most likely language. Tokens that are not words — a URL, an e-mail address, a path, a version number or a hyphenated identifier — are recognised by an internal separator (`.` `@` `/` `\` `:` `_` `-`) and left alone.
5. **Decide.** If the detected language differs from the active layout, the converter produces the "other-layout" rendering of the buffer. If that rendering looks more like a real word, the daemon proceeds.
6. **Act.** The switcher changes the active layout and the injector deletes the word with Backspace and retypes it in the new layout.

Scancode replay and the `phello` bug

The replacement sends one Backspace per physical key the user pressed, not per character in a string. The buffer keeps the evdev codes alongside the characters, so the deletion can never drift from what is on screen: if a key was processed late, its code is still in the list, and the count is still exact. A free-form character buffer can desynchronise (a key the daemon did not process in time) and leave the first character behind — the `phello` symptom, where `руддщ` was deleted but the leading `р` survived.

The uinput backend also flushes the deletion and the replacement as two separate `syn` batches, with an optional pause (`backspace_settle_ms`, default 120 ms) between them. Chromium and Electron applications process Backspace asynchronously, so typing into the same batch can race the deletion. The pause lets the compositor apply the deletion before the new text arrives. The backspace batch is still flushed with a single `syn`, so a crash before that point leaves the text untouched instead of half-deleted.

When the flush is triggered by a word-boundary key (Space/Enter/Tab/punctuation), the daemon waits an additional `trigger_settle_ms` (default 50 ms) before the deletion. The boundary key that ended the word is still being processed by the compositor at that instant; deleting immediately races it and leaves the first character behind (`руддщ` -> `phello`). The boundary key is then part of the deletion and is retyped with the corrected word, so it keeps its place. The idle fallback and an explicit hotkey skip this pause, because no boundary key is in flight.

A word-boundary trigger is the practical limit of "real time": a word cannot be corrected before it is finished, because until then the detector does not know what the word is. Pressing Space is that moment, and the correction costs no extra keystroke.

Why evdev + uinput

- `evdev` reads events straight from the kernel, so Klyrshift sees the physical keys the user pressed regardless of the current layout or the toolkit the focused application uses. This is what makes detection possible at all: the application only ever sees the translated characters.
- A virtual keyboard (created with `evdev.UInput`, which speaks the kernel `uinput` interface) lets the daemon synthesise Backspace and the corrected characters. Because the input goes through the same kernel path as a real keyboard it works in almost every application, including those that ignore synthetic X11/Wayland events.
- The combination works identically on X11 and Wayland, which is why it was chosen over toolkit-specific hooks.

The device is created with `evdev.UInput`, not with the `python-uinput` package. `python-uinput` changed its public API between Debian releases (`Device`/`emit` versus `UInput`/`write`) and its `KEY_*` constants are `(event_type, code)` tuples, so `int(uinput.KEY_A)` raises `TypeError` on Debian 13. Key codes come from `evdev.ecodes` (plain ints), which removes the dependency and the version skew in one move.

Device access: uaccess instead of the input group

Reading `/dev/input/event*` and writing `/dev/uinput` both need elevated device permissions. There are two common ways to grant them, and Klyrshift deliberately uses the first.

- **`TAG+="uaccess"` in a udev rule (chosen).** `systemd-logind` grants a POSIX ACL to the user who is physically logged in at the seat. The ACL is applied when that user logs in, is granted only to them, and is removed when the session ends. It requires no `usermod` and no relogin after installing, which is what makes the `.deb` zero-config.
- **Membership of the `input` group (rejected).** This is permanent: every member can read every keyboard, at every seat, whether or not they are the active user. It also only takes effect after a full logout/login.

The security difference matters: `uaccess` follows the session, so a background process belonging to a logged-out user does not keep keyboard access. The packaged rule is:

```
ACTION!="remove", KERNEL=="event*", TAG+="uaccess"
ACTION!="remove", KERNEL=="uinput", TAG+="uaccess", OPTIONS+="static_node=uinput"
```

`OPTIONS+="static_node=uinput"` makes udev create `/dev/uinput` at boot once the module is loaded, so the node exists before the daemon starts.

Two details are easy to get wrong and both are load-bearing:

- The file is named `71-klyrshift.rules`, not `99-*`. On systemd 257+ (Debian 13) the rule that turns `uaccess` into an ACL is `73-seat-late.rules`. udev applies a tag in filename order, so a `99-*` rule runs after `73-seat-late.rules` and the ACL is never applied. Installing the rule before 73 makes it effective.

- Every rule carries `ACTION!="remove"`. `73-seat-late.rules` only processes rules that guard against the `remove` event, so a rule without it is skipped.

`klyrshift doctor` accepts either mechanism so the developer path (which may still use the `input` group) is not reported as broken, but the packaged path never touches group membership.

Why g3kb-switch

On GNOME Wayland there is no portable CLI to change the layout. `g3kb-switch` talks to GNOME Shell over D-Bus and is the de-facto standard tool for this. On X11 the daemon falls back to `setxkbmap`. The `LayoutSwitcher` abstracts the two and caches the current layout for a short period to avoid spawning a subprocess on every keystroke.

Wayland limitations

Wayland deliberately hides the notion of "which widget has focus" from unprivileged clients. Klyrshift therefore cannot know whether the caret is in a password field, a terminal, or a game. The consequences are documented and mitigated rather than solved:

- **Password fields.** The daemon cannot detect them. Mitigation: a stop-word list (`password`, `login`, `token`, …) suppresses corrections for those words.
- **Short words.** One- and two-character words carry almost no statistical signal. Mitigation: `min_word_length` (default 3) skips them.
- **Some applications.** A few games and Java/Swing applications intercept input in ways that defeat synthetic keystrokes. This is a known limitation.

Logging and privacy

Klyrshift reads everything you type, so it must never persist that text. The log records only metadata: the buffer length, the source and target layout, and the action taken. The typed string itself is never written to the log file, which is rotated at 5 MB with three backups (`~/.local/state/klyrshift/klyrshift.log`). The `--dry-run` flag analyses and logs the same metadata without switching the layout or touching the text, which makes it safe to run while investigating behaviour.

The same rule applies to every other surface:

- **Notifications.** The desktop notification is a fixed label, never the corrected text, so nothing typed reaches the notification history.
- **Subprocesses.** No argv ever contains the typed text. This matters because argv is world-readable through `/proc` and is echoed into the journal.
- **Tracebacks.** If a fix raises, the traceback is formatted and the buffer is replaced with `<redacted>` before it is logged, so an exception message that happens to embed the text cannot leak it.

`tests/test_privacy_audit.py` enforces all of this: it runs a session with a distinctive secret string and then searches the log, cache, state directory, config file, user journal and every subprocess argv for any trace of it.

Failure handling

The daemon is designed to survive the failures that a long-running, keystroke- reading process will meet in practice:

- **Atomic deletion.** The `uinput` backend writes every backspace and flushes them with a single `syn`, then types the replacement as a second flush after `backspace_settle_ms`. A crash or `SIGKILL` before the first `syn` leaves the text untouched rather than half-deleted, and the settle pause keeps the replacement from racing an asynchronous compositor. A boundary-triggered flush also waits `trigger_settle_ms` before the deletion, so the Space/Enter that ended the word is processed first. The `wtype`/`xdotool` backends cannot be atomic; that residual window is documented in the troubleshooting guide.
- **Exact deletion count.** The daemon counts physical keys, not characters, so the number of Backspaces always matches what is on screen even under very fast typing.
- **No mixed-script rewrites.** A single token that mixes Latin and Cyrillic (`ghbdtnпривет`, `приветhello`) cannot be attributed to one language: only one half is on the wrong layout, so converting the token as a unit would damage the half that is already correct (`ghйпривет`). `target_layout` and `detect` return `None` for such a token (`_has_mixed_script_word`) and the T9 pass skips it too; the separator between the two scripts flushes each half on its own boundary. Scripts that occupy separate words (`привет hello`) are still converted as a whole, which is what makes the guard safe.
- **No same-layout rewrites.** A layout correction must move to a different layout. If `target_layout` ever returns the current layout, the daemon refuses the replacement (and logs a warning): deleting and retyping the word in place would be a pure deformation, not a fix. The `Fixing buffer ... reason=...` log line records whether the replacement was a layout fix, a typo fix or a punctuation clean-up.
- **Races.** The buffer is snapshotted and cleared under a lock, but the lock is released before the slow switch/inject, so keys pressed during a fix are buffered for the next pass instead of being dropped.
- **Bounded memory.** The buffer is capped at `max_buffer_size` (default 200). A held key or a paste-like burst is analysed and cleared early rather than growing without bound.
- **Shutdown.** `SIGTERM`/`SIGINT` set a flag. A fix is skipped if shutdown was requested before it started, and always finishes once started. `klyrshift stop` (and `kill`) stop the systemd user unit first when it is the manager, because the unit carries `Restart=always` and would otherwise respawn a process killed out from under systemd; `stop` then escalates to `SIGKILL` if the daemon does not exit within 3 seconds, and `kill` sends `SIGKILL` at once.
- **Device loss.** When a keyboard disappears the device is dropped and discovery is retried in place; `Restart=always` in the unit is the backstop.
- **Every event and every buffer is guarded.** An exception in the detector, converter, switcher or injector is logged (redacted) and the loop continues.

Languages, context and exceptions

Recognition is language-first, not script-first. Every layout is mapped to a language (`us` → `en`, `ru` → `ru`, `uk` → `uk`, `fr` → `fr`, `de` → `de`) and a word is scored against the corpora whose layout could have produced it.

- **Bundled languages.** `en`, `ru`, `uk`, `de` and `fr` all ship corpora under `src/klyrshift/data/`. Only `en` and `ru` are loaded by default: the Latin layouts (`us`, `de`, `fr`) share physical positions, so loading every corpus at once makes them compete and blurs borderline

words. Enable the rest with `languages = ["en", "ru", "uk", "fr"]` in the config. This is why `LanguageDetector()` without arguments keeps behaving exactly as before.

- **Context analysis** (`context_analysis`, `context_weight`). The previous word is treated as weak evidence: when it has an unambiguous language, a candidate in that language gains `context_weight`. It is additive only, so it can tip a borderline word but can never talk the detector out of a clear correction. Disable it with `context_analysis = false` or a weight of `0.0`.
- **Per-app exceptions** (`exceptions_apps`). A focused application on this list is never touched, in any mode. The list ships pre-filled with terminals, IDEs, editors and games (`DEFAULT_EXCEPTION_APPS` in `config.py`) because there the other layout is not more plausible — the text is a command or an identifier. `exceptions_force_in_manual` lists the applications that stay automatic while `mode = "manual"`. The focused application is resolved best-effort; the daemon degrades to auto behaviour when it cannot tell, and never reads window contents. A manual fix (`SHIFT+SHIFT`) bypasses the exception list, so it still works inside a terminal.
- **Per-app mode** (`app_modes`). The global `mode` is one value, but the right mode depends on the task: code should not be rewritten automatically while a chat window benefits from it. `app_modes` maps an application to its own mode (`auto`/`manual`/`hybrid`); an application absent from the map keeps `mode`. `KlyrshiftDaemon._effective_mode` resolves this, and both policy gates consult it — `_should_fix_buffer` (does an automatic fix happen at all) and `_layout_target` (is a low-confidence hybrid fix applied). The focused application is probed once per word through `_focused_app`, which caches the D-Bus answer for `APP_PROBE_INTERVAL` so the two gates do not each pay for it.
- **Quiet hours** (`quiet_hours_enabled`, `quiet_hours_start`, `quiet_hours_end`). Inside the daily window the daemon skips automatic correction but keeps the manual fix. The window may wrap midnight (`22:00`–`08:00`). The check is cached per wall-clock minute and consulted in `_should_fix_buffer`.
- **Correction history** (`history_size`). A bounded ring of recent fixes backs the GUI "История" tab. An entry stores only the word length, the source and target layouts, a monotonic timestamp and an `undone` flag — never the typed text. The daemon rewrites the atomically-written `history.json` whenever the history changes (a fix or an undo); the GUI only reads it through `daemon_control.read_history`, without signalling the daemon, so a background tab can poll it without any log spam.
- **Skip rules** (`ignore_all_caps`, `ignore_with_digits`, `ignore_emails_urls`, `custom_skip_regex`). These short-circuit a buffer before detection. Digital tokens, `ALL CAPS` and e-mail/URL/path-like tokens (an internal separator such as `. @ / \ : _ -`) are left untouched.
- **False-positive guards** (`plausibility_check`, `structural_boundaries`, `identifier_guard`, all on by default). Three layers keep the detector from rewriting text that already reads as real words:
- Plausibility: the typed text is scored in the layout it was typed in; if it already looks like a real word there, no correction is made. This suppresses the `cb cj,jq?ye;yjn/g/` class of false positives.
- Structural boundaries: URLs, e-mail addresses, paths and version numbers are recognised and skipped before scoring.
- Identifier guard: `snake_case`, `camelCase`, `x86_64` and similar code tokens are treated as identifiers, not prose. The guards run **after** the user-dictionary check, so a taught word (`муксуд` → `vercel`) is still fixed even when the source looks plausible.

- **User dictionary** (`dictionary_size`, `dictionary_custom_path`). A newline-separated list of words the user taught the daemon. It is a deliberate, explicit file (`~/.local/share/klyrshift/dictionary.txt` by default) — the daemon never writes typed text to it on its own. The list is consulted in two directions. A taught word is an absolute override: when the buffer equals a taught word, `target_layout` returns `None` for it in every layout. The converted form is also checked: if the wrong-layout rendering of the buffer is a taught word (`муксуд` → `vercel`), that rendering is accepted as valid and the correction fires, even though the brand is absent from the frequency corpora. Without this, an unknown brand makes the detector stay silent. `dictionary_size` caps the bundled vocabulary (larger = more recall, more RAM).
- **Extended and thematic dictionaries** (`extended_dictionary_dir`, `installed_dict_categories`). Two opt-in layers on top of the bundled corpora. The base extended list (`dict download`/`import-file`) is one `<lang>.txt` file per language. The thematic lists (`dict install <category>`) are professional vocabularies — `it`, `medicine`, `legal`, `finance`, `engineering`, `science`, `business`, `electronics`, `media`, `education`, `gaming`, `sport` — stored under `<extended_dictionary_dir>/thematic/<lang>/<slug>.txt` and enabled by slug in `installed_dict_categories` (`base` is always on). Both layers widen vocabulary **membership only**; n-gram scoring is unchanged, so detection behaviour for already-known words stays identical while rarer words are recognised. Priority is **user dictionary → thematic → base → T9**: the user dictionary is checked first and is an absolute override, a thematic word is accepted before the general corpus is consulted. The detector is given the categories at construction and rebuilds its vocabularies when `set_thematic_categories` is called on `reload_config`. Thematic files are never bundled in the `.deb`; they are fetched from the GitHub Release (`klyrshift-dict-<slug>.tar.gz`, falling back to the older `lingvix-dict-<slug>.tar.gz` and `linguafix-dict-<slug>.tar.gz` that the live release ships) or the offline repository copy.
- **Typo correction (T9)** (`typo_mode`, `typo_max_distance`, `typo_max_distance_long`, `typo_min_word_length`). A second, layout-neutral correction path, off by default. Its own `typo_mode` (`off` / `manual` / `auto` / `hybrid`) selects which T9 pass may run, but the working `mode` is the master gate: `manual` silences the layout-neutral passes too, so nothing is rewritten without an explicit hotkey. Within that, `manual` fires only on `CTRL+SHIFT+F`, `auto`/`hybrid` on a word boundary, `off` never. The legacy boolean `typo_correction` is kept mirrored to `typo_mode != "off"` for compatibility (and for the GUI switch). A legacy file is migrated on load: `typo_correction = true` with `mode = "manual"` becomes `typo_mode = "manual"`, otherwise the global mode is used. It runs only when `target_layout` returned `None` — that is, when the layout is already right — so it can never fight the layout switcher. It uses the same vocabulary the detector scores against (`LanguageDetector.ordered_vocabulary`, which preserves the corpus frequency order that `vocabulary` loses in a set) and accepts a replacement only when the word is absent from the vocabulary and within `typo_max_distance` edits of a word (optimal string alignment / Damerau-Levenshtein, so a transposition counts as one edit). A word at least 6 characters long may use `typo_max_distance_long` edits (default 2), but only when the best candidate is clearly more frequent than the runner-up. Two equally close candidates mean no change. Taught words and tokens with an internal separator are never corrected. `TypoCorrector` instances are built lazily per language and dropped on `reload_config`, because the vocabulary can change with `dictionary_size` and `languages`.

Modes and hotkeys

`mode` is the master gate: `auto` acts on every word, `manual` acts only on an explicit hotkey, and `hybrid` acts automatically but withholds an uncertain layout fix for the manual hotkey. `manual` silences every automatic pass — layout, T9 and punctuation — so a user who chose "only when I ask" is never rewritten without a command (the reported "the app fixes me without my command" bug).

Each optional feature then has its own mode on top of that (`typo_mode`, `punctuation_mode`): `off`, `manual` (explicit hotkey only), `auto` (on a word boundary) or `hybrid`. It selects which pass may run; the working `mode` still decides whether it runs automatically. This lets a user keep automatic layout switching while making T9 hotkey-only (`typo_mode = "manual"`, `mode = "auto"`), while a fully quiet app is `mode = "manual"`.

Hotkeys are matched in `daemon._parse_hotkey`. A chord such as `CTRL+F12` requires the modifier to be held; a **double tap** such as `SHIFT+SHIFT`, `CTRL+CTRL` or `ALT+ALT` fires when the same modifier is pressed twice within `hotkey_double_tap_ms` (default 2000 ms, accepted range 100-3000 ms) with nothing else in between. A generous default is deliberate: at typing speed two Shift presses made while capitalising can fall inside a short window, and each false match deletes and retypes text. The default fix hotkey is `SHIFT+SHIFT` because a laptop has no Pause key. Double taps are tracked separately from chords (`_double_tap_hotkeys`): the first tap arms the family, a matching second tap within the window fires the action and clears the state, and any other key cancels the arm. A single Shift press never triggers a fix, so normal capitalisation is unaffected. Hotkeys are re-parsed on `reload_config`, so editing `config.toml` and sending `SIGHUP` rebinds them live.

The default **undo** hotkey is `SHIFT+BACKSPACE`, not `CTRL+Z`. The daemon does not grab the keyboard (`EVIOCGRAB` is never issued), so it cannot swallow a chord: a bound `CTRL+Z` would fire the application's own undo as well, deleting the corrected word while the daemon's retype raced it. `hotkey_swallow` in the config is inert for the same reason and is kept only for forward compatibility. The daemon logs the effective hotkeys at startup so the binding is discoverable.

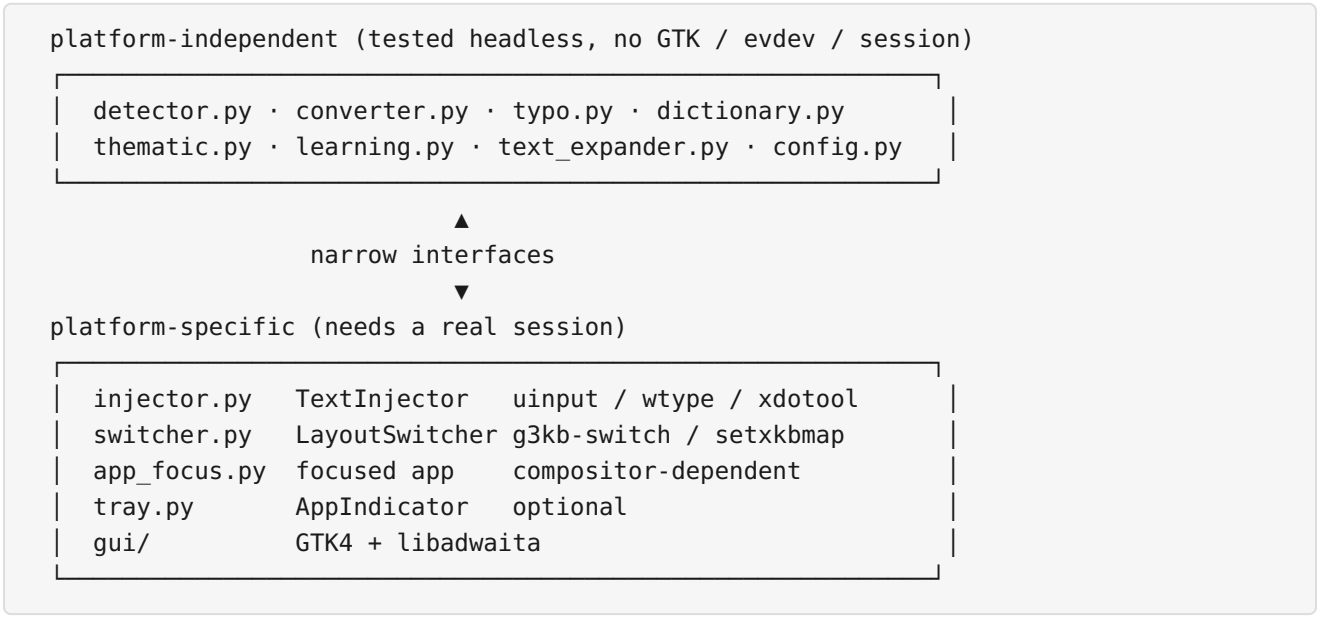
The **toggle-layout** hotkey (default `CTRL+SHIFT+T`, configurable as `hotkey_toggle_layout_last_word`) force-retypes the last word in the other layout, the manual counterpart of an automatic correction. It deliberately bypasses the detector — a word that looks correct in its current layout is still converted when the user asks — and falls back to `_last_flushed_word` when the live buffer is empty because a Space boundary already consumed the word. A word already corrected by the boundary is remembered in `_last_flushed_corrected`, so a later double Shift does not revert it.

Hybrid mode offers what it will not apply. The confidence gate in `_layout_target` withholds a conversion below `hybrid_confidence` instead of rewriting the word. The withheld offer is kept in `_pending_suggestion` (typed text, target layout, confidence) and can be applied deliberately through the tray's "Apply suggestion" entry, the optional `hotkey_accept_suggestion`, or the same double Shift that fixes any last word. The offer is dropped when the next word is flushed, so a stale suggestion is never applied to text the user has moved past. The text is held in memory only: neither the offer nor the optional notification ever logs or passes it to a subprocess argv.

Target architecture: the system-integration boundary

The daemon is layered so that the platform-specific parts sit behind narrow interfaces and the decision logic has no OS dependency. That boundary is what makes a Debian 13 + GNOME target

concrete rather than aspirational: everything above the line below is testable headless, everything below it is what a real session must confirm.



Integration points a Debian 13 + GNOME install depends on:

Point	Mechanism	Failure if missing
Reading keystrokes	`evdev` on `/dev/input/event*`	No detection at all
Device permission	udev `uaccess` tag (not the `input` group)	`doctor` reports no device access
Typing the replacement	`/dev/uinput` (module loaded by the package)	No automatic replacement
Switching layout (Wayland)	`g3kb-switch` over D-Bus	Manual mode only
Switching layout (X11)	`setxkbmap`	Manual mode only
Autostart	systemd user service	Daemon does not start at login

Each row is a check in `klyrshift doctor`. See PLATFORM_SUPPORT.md for which rows are verified.

Extension points

- `LanguageDetector` — add a language by dropping `ngrams_<lang>.json` and a word list into `src/klyrshift/data/` and extending the layout map. Then add the language to `SUPPORTED_LANGUAGES` in `config.py`.
- `LayoutConverter` — layouts live in `src/klyrshift/data/layouts.json`; adding a pair is a data-only change.
- `TextInjector` — new backends implement `replace_text` and register in the backend-selection logic.
- `LayoutSwitcher` — new session types implement `get_current_layout` and `switch_to`.
- `tray.py` — optional; the daemon runs fine without PyGObject installed.

Module map

Module	Responsibility
`config.py`	`Config` dataclass, XDG paths, TOML load/save
`logging_setup.py`	Rotating file logger under `~/.local/state/klyrshift/`
`converter.py`	Character maps between layouts (`us`, `ru`, `uk`, `de`, `fr`)
`detector.py`	Language detection, context, user dictionary, `target_layout`
`dictionary.py`	User dictionary file, plus extended frequency dictionaries (`import-file`/`download`)
`thematic.py`	Thematic (professional) dictionaries: categories, release/mirror download, offline fallback
`typo.py`	T9: bounded Damerau-Levenshtein + `TypoCorrector` (opt-in)
`text_expander.py`	Snippet expansion (trigger -> text)
`selection_fix.py`	Convert the layout of already-selected text
`app_focus.py`	Best-effort focused-application detection for exceptions
`app_layouts.py`	`AppLayoutManager`: switch layout when a mapped app gains focus
`switcher.py`	`g3kb-switch` / `setxkbmap` layout control
`injector.py`	Text replacement via `uinput` / `wtype` / `xdotool`
`daemon.py`	`KlyrshiftDaemon`: event loop, buffering, orchestration
`daemon_control.py`	Start/stop/reload/undo a daemon however it was launched
`backup.py`	Export/import a JSON bundle of config + dictionary + snippets
`update_check.py`	Opt-in GitHub release check (daily, off by default)
`doctor.py`	`klyrshift doctor`: environment and permission diagnosis
`collect_logs.py`	`klyrshift collect-logs`: bundle logs for a bug report
`onboarding.py`	Tour steps and help texts as plain data (no GTK import)
`tray.py`	Optional AppIndicator icon
`cli.py`	`argparse` command-line interface
`gui/`	Optional GTK4 + libadwaita preferences window

Project structure

A map of the Klyrshift repository: what every top-level directory is for, which module does what, where the tests, the documentation and the packaging live, and which files are generated rather than edited. Read this first if you are new to the codebase; the design rationale behind the modules is in ARCHITECTURE.md, and the repository conventions an agent or a contributor must follow are in `AGENTS.md`.

Top level

Path	What it holds
`src/klyrshift/`	The Python package (PEP 621 layout, declared in `pyproject.toml`).
`tests/`	The pytest suite; replaces the OS boundary with fakes.
`docs/`	The documentation sources, the generated handbook and the screenshots.
`scripts/`	Build, packaging, verification and documentation-generation scripts.
`data/`	System integration files used by `install.sh` and the `.deb` build.
`src/klyrshift/data/`	Data files **shipped inside the package** (what an install reads).
`dictionaries/`	Frequency word lists and the thematic dictionary categories.
`site/`	The Astro landing page (separate Node project, own lockfile).
`.github/`	CI/release workflows, issue and PR templates, Dependabot config.
`.openhands/`	Session memory notes for agents (not shipped, not part of the build).
`README.md`	The user-facing front page and the entry point to the docs index.
`CHANGELOG.md`	Keep-a-Changelog history; the release date is derived from it.
`AGENTS.md`	Repository conventions and the invariants that have bitten before.
`AUDIT.md`	The daemon audit: baseline numbers, defects found, fixes.
`pyproject.toml`	Packaging metadata, dependencies and the tool configuration.
`requirements.txt`	The same runtime dependency floors, for the venv installer.
`Makefile`	The task entry points (`make check`, `make build-deb`, ...).
`install.sh` / `uninstall.sh`	The source installer and its uninstaller.
`MANIFEST.in`	Extra files the sdist carries (dictionaries, man page, completions).
`coverage-gui.rc`	The coverage config the GUI CI job uses (GUI modules included).
`LICENSE`, `.editorconfig`, `.gitignore`	Licence and editor/tooling defaults.

The Python package — `src/klyrshift/`

The package is split into small, single-responsibility modules. The daemon is the centre; everything else is a capability it composes or a boundary it talks to.

Core runtime

Module	Responsibility
`__init__.py`	The package version (`__version__`) and `__all__`.
`__main__.py`	`python -m klyrshift` entry point.

Module	Responsibility
`cli.py`	The `argparse` command line: every subcommand and option.
`config.py`	The `Config` dataclass, XDG paths, TOML load/save, validation.
`daemon.py`	`KlyrshiftDaemon`: the event loop, buffering and orchestration.
`daemon_control.py`	Start/stop/reload/undo a daemon however it was launched.
`detector.py`	Language detection, context, user dictionary, `target_layout`.
`converter.py`	Character maps between layouts (`us`, `ru`, `uk`, `de`, `fr`).
`switcher.py`	Layout switching via `g3kb-switch` / `setxkbmap`.
`injector.py`	Text replacement via `uinput` / `wtype` / `xdotool`.
`dictionary.py`	The user dictionary file and the extended frequency dictionaries.
`thematic.py`	Thematic (professional) dictionaries: categories, download, offline fallback.
`typo.py`	T9: bounded Damerau-Levenshtein and `TypoCorrector` (opt-in).
`punctuation.py`	Opt-in punctuation cleanup for a finished word.
`text_expander.py`	Snippet expansion (trigger -> text).
`selection_fix.py`	Convert the layout of already-selected text.
`learning.py`	Always-on, privacy-preserving learning (counters on disk, words in memory).
`app_focus.py`	Best-effort focused-application detection for exceptions.
`app_layouts.py`	`AppLayoutManager`: switch layout when a mapped app gains focus.
`control.py`	The local control socket for the running daemon.
`integrity.py`	Post-install integrity check (`klyrshift verify`).
`langpack.py`	Versioned, checksummed language packages.
`langstore.py`	Install/remove language dictionaries (the GUI path).
`doctor.py`	`klyrshift doctor`: environment and permission diagnosis.
`collect_logs.py`	`klyrshift collect-logs`: bundle logs for a bug report.
`logs.py`	Read the daemon log for `klyrshift logs`.
`logging_setup.py`	Rotating file logger under `~/.local/state/klyrshift/`.
`backup.py`	Export/import a JSON bundle of config + dictionary + snippets.
`update_check.py`	Opt-in GitHub release check (daily, off by default).
`onboarding.py`	Tour steps and help texts as plain data (no GTK import).
`tray.py`	Optional AppIndicator tray icon.
`i18n.py`	Interface localisation: the Russian source string is the key.
`repo.py`	Single source of truth for the project's GitHub URLs.
`simulate.py`	The headless behavioural simulator (fakes keyboard/switcher/injector).

GUI — `src/klyrshift/gui/`

An optional GTK4 + libadwaita window. It is imported lazily, so the package works without PyGObject installed.

Path	Responsibility
`app.py`	The `Adw.Application`.
`window.py`	The main window: home, settings and advanced tabs.
`home_page.py`	Home page: the big on/off toggle and at-a-glance status.
`settings_page.py`	Basic settings page (always visible).
`advanced_page.py`	Advanced settings page (hidden behind a button).
`typo_page.py`	T9 and punctuation cleanup, each with its own mode.
`dictionary_page.py`	The user-dictionary page.
`history_page.py`	The last corrections, shown as metadata only.
`help_page.py`	Tour replay, settings help and diagnostics.
`about_page.py`	The About dialog (carries the dictionary-data credit).
`prefs_base.py`	Shared base for the settings pages.
`state.py`	Shared GUI state: config access and daemon status helpers.
`style.py` / `style.css`	Load the bundled stylesheet.
`theme.py`	Apply the configured colour scheme.
`async_utils.py`	Run blocking probes off the GTK main thread.
`widgets/`	The reusable widgets (each in its own module):
`widgets/big_toggle.py`	The VPN-style big on/off button.
`widgets/mode_switcher.py`	The segmented auto/manual/hybrid switch.
`widgets/status_row.py`	The compact status line under the big toggle.
`widgets/dictionary_list.py`	The editable user-dictionary list.
`widgets/app_exceptions_list.py`	The editable per-application exceptions list.
`widgets/app_layout_map.py`	The app-name -> preferred-layout mapping.
`widgets/app_mode_map.py`	The app-name -> correction-mode mapping.
`widgets/languages_group.py`	Enable a language and install/remove its dictionary.
`widgets/hotkey_row.py`	A row that records a keyboard shortcut.
`widgets/backup_group.py`	Export/import the whole configuration.
`widgets/first_run_wizard.py`	The paginated first-run setup dialog.
`widgets/tour_overlay.py`	The guided-tour overlay.
`widgets/help_button.py`	The context-help `?` button and its dialog.
`widgets/shortcuts_dialog.py`	The keyboard-shortcut reference.

Autonomous QA engine — `src/klyrshift/qa/`

A testing layer on top of the pytest suite, driven by the simulator. The corpus is data (`src/klyrshift/data/qa/scenarios.json`), not Python literals.

Module	Responsibility
`scenario.py`	The scenario model and its JSON format.
`curated.py`	The curated scenario library, loaded from bundled JSON.
`invariants.py`	Independent correctness checks over a simulation result.
`executor.py`	A safe executor: run a scenario through `klyrshift.simulate`.
`generator.py`	Deterministic (seeded) generation, with a pluggable LLM hook.
`analyzer.py`	Classify a failure and minimise it by delta-debugging.
`store.py`	The append-only regression store.
`runner.py` / `report.py`	Orchestration and measured-only reports.

Shipped data — `src/klyrshift/data/`

Path	Contents
`layouts.json`	The physical-position maps for every supported layout.
`ngrams_<lang>.json`	Frequency word lists and bigram maps per language.
`stop_words.txt`	Tokens that are never corrected (one per line).
`locales/<locale>.json`	Interface translation catalogues (`en`, `de`, `fr`, `uk`).
`qa/scenarios.json`	The curated QA scenario corpus.
`klyrshift.service`, `klyrshift.desktop`, `klyrshift-autostart.desktop`, `klyrshift.svg`	Copies of the system integration files (see below).

System integration — `data/` and `install.sh`

`data/` is the source the source installer (`install.sh`) and the `.deb` build read; `src/klyrshift/data/` is what an installed package ships and what the CLI reads. The service, desktop, autostart and icon files exist in **both** places and must stay byte-identical — `tests/test_data_sync.py` enforces this.

Path	Contents
`data/71-klyrshift.rules`	The udev rule granting device access via `uaccess` (named below `73-`).
`data/klyrshift.service`	The systemd **user** unit (enabled globally by the `.deb`).
	The application and XDG-autostart entries.

Path	Contents
`data/klyrshift.desktop`, `data/klyrshift-autostart.desktop`	
`data/klyrshift.svg`	The application icon.
`data/klyrshift.1`	The man page (**generated** by `make cli-docs`).
`data/completions/`	The bash/zsh completions (**generated** by `make cli-docs`).
`data/copyright`	The DEP-5 copyright, shipped to `/usr/share/doc/<pkg>/copyright`.

Tests — `tests/`

88 modules covering logic, config, detector, daemon policy, GUI, packaging, documentation consistency and privacy. The suite replaces the OS boundary (evdev, `uinput`, GTK) with fakes and never mocks Klyrshift's own modules, so it proves logic but **not** packaging — `scripts/smoke_test.sh` installs the real `.deb` for that. `tests/conftest.py` holds the shared fixtures. See TESTING.md and TEST_STRATEGY.md.

The GUI tests need a display: run them under `xvfb-run -a` with `PYTHONPATH=/usr/lib/python3/dist-packages` so `gi` (GTK4/libadwaita) imports.

Documentation — `docs/`

Every guide is a Markdown source in `docs/` and is linked from the README. The offline handbook (`docs/klyrshift-handbook.md` and `.pdf`) is **generated** by `scripts/build_docs.py` from the sources listed in its `SOURCES` tuple — a new guide must be added there or `tests/test_docs_handbook.py` fails.

Document	Covers
INSTALL.md	Install from the `.deb` or from source; migrate from the old names.
CLI.md	The command-line reference (generated; same as `man klyrshift`).
TROUBLESHOOTING.md	Symptom-to-fix guide for real systems.
PLATFORM_SUPPORT.md	X11/Wayland support and what is actually verified.
LANGUAGE_PACKAGES.md	The language-package format: version, checksum, rollback.
INTERFACE_LANGUAGES.md	UI catalogues, the fallback chain and how to add a locale.
ARCHITECTURE.md	Design decisions, data flow and the module map.
PROJECT_STRUCTURE.md	This file: the repository map.
CONTROL_SOCKET.md	The daemon control-socket protocol.
UNDO_CONTRACT.md	The contract for undoing a correction.
OFFLINE_AND_PRIVACY.md	Offline operation and the privacy invariants.
TESTING.md	The three test levels and the manual desktop plan.
QA_ENGINE.md	The autonomous QA engine.
AUTOPROGRESS_BREAKAGE.md	Why the automated run breaks: the defect, its root cause and the fix.

Document	Covers
TEST_STRATEGY.md	Quality metrics and regression thresholds.
RELEASING.md	The release checklist.
CONTRIBUTING.md	Dev setup, code standards and commit conventions.
ROADMAP.md	Where the project is and what comes next.
FEATURES.md	The implemented-feature inventory, grouped by area.
WORK_REPORT.md	The work report: what is done and the results achieved.
VERIFICATION_REPORT.md	The verification report: what is already built, measured on the current tree.
screenshots/	The README screenshots and how they are rendered.

Scripts — `scripts/`

Script	Purpose
`build_deb.sh`	Build the reproducible `.deb` with `dpkg-deb` (no `dh_make`).
`build_dict_archives.sh`	One `.tar.gz` per thematic dictionary category.
`smoke_test.sh`	Install the `.deb` into a clean container and exercise the CLI.
`deb_selfsufficiency_test.sh`	Prove the `.deb` needs no manual step (udev, enable).
`upgrade_test.sh`	Install over the pre-rename `linguafix` package.
`patch_upgrade_test.sh`	Install the current patch over the previous one.
`acceptance_check.sh`	The real-hardware acceptance check (Debian 13 + GNOME).
`verify_v028.sh`	Post-install verification for v0.2.8.
`build_docs.py`	Render the Markdown handbook and the PDF.
`build_cli_docs.py`	Generate the man page and completions from the parser.
`build_corpora.py`	Build `ngrams_<lang>.json` from FrequencyWords lists.
`build_demo_model.py`	Generate the landing page's `demo-model.json`.
`make_screenshots.py`	Render the README screenshots from the real GUI.
`make_og_image.py`	Render the social-preview card.
`measure_detector.py`	Reproducible detector-quality measurement.
`measure_latency.py`	Reproducible latency and memory measurement.
`merge_dict_terms.py`	Merge terms into a thematic dictionary file.
`refresh_dict_readmes.py`	Rewrite the word count in the thematic READMEs.
`release_consistency.py`	Check the tree (and, with `--latest`, the release) for consistency.
`dev_setup.sh`	Create the dev venv and install the hooks.

Landing page — `site/`

A separate Astro 7 + Tailwind CSS 4 project with its own `package.json` and lockfile. It carries a port of the detector's plausibility test (the demo), which must stay in step with the Python detector. See `site/README.md` and the `site/` notes in `AGENTS.md`.

Path	Contents
`site/src/pages/`	The pages (`index.astro`, `dictionaries.astro`, `robots.txt.ts`).
`site/src/components/`	The page sections (hero, features, comparison, demo, ...).
`site/src/layouts/Base.astro`	The shared HTML shell (canonical, hreflang, JSON-LD).
`site/src/lib/`	Shared helpers (site constants, theme, dictionary palette).
`site/src/data/demo-model.json`	The demo model (**generated** by `build_demo_model.py`).
`site/public/`	Static assets (logo, favicons, the generated `og-image.png`).
`site/astro.config.mjs`, `site/layero.json`, `site/vercel.json`	Build and deploy config.

Dictionaries — `dictionaries/`

The base frequency lists (`en-50k.txt`, `ru-50k.txt`, `uk-50k.txt`, `de-50k.txt`, `fr-50k.txt`) are CC-BY-SA-4.0 content from FrequencyWords. The thematic categories (`business/`, `code/`, `medicine/`, ...) are hand-curated and MIT. `dictionaries/README.md` documents the layout; see also LANGUAGE_PACKAGES.md and the licensing notes in `AGENTS.md`.

Build and verify

```
bash scripts/dev_setup.sh        # venv + dev dependencies
make check                       # ruff + isort + black + mypy --strict + pytest --cov
make build-deb                   # build the .deb into dist/
make smoke-test                  # install it into a clean Debian container
make docs-pdf                    # regenerate the handbook (Markdown + PDF)
make cli-docs                    # regenerate the man page and completions
```

Generated files that are committed and must be regenerated, never hand-edited: `docs/klyrshift-handbook.md`/`.pdf`, `data/klyrshift.1`, `data/completions/*`, `site/src/data/demo-model.json`, `site/public/og-image.png` and the README screenshots. `AGENTS.md` lists the invariants (duplicated data files, version strings in five places, reproducible `.deb`) that the tests pin.

Управляющий сокет (control socket)

Демон Klyrshift держит в памяти слова, которые выучил, наблюдая за набором («закреплённые» слова — их он больше не исправляет). Эти слова **никогда не пишутся на диск**: сохраняются только счётчики. Значит, одноразовая CLI-команда не может прочитать их из файла — она должна спросить работающий демон.

Для этого есть локальный управляющий сокет.

Что это

Unix-сокет `SOCKET_STREAM`, который демон создаёт при старте и слушает в том же `select()`-цикле, что и клавиатуры. Запрос — одна строка JSON, ответ — одна строка JSON. Сокет:

- лежит в `$XDG_RUNTIME_DIR/klyrshift/control.sock` (а если `XDG_RUNTIME_DIR` не задан — в кэш-каталоге);
- создаётся с правами `0600`: читать и писать может только владелец;
- это файл в локальной ФС, а не сетевой порт: снаружи машины он недоступен;
- удаляется при остановке демона, так что «залипший» сокет не мешает следующему запуску.

Размер запроса ограничен (`MAX_REQUEST_BYTES`); слишком большой запрос отклоняется, а не копится в памяти. Битый JSON не роняет демон: клиент получает `{"error": "malformed request"}`, и цикл продолжает работу.

Команды

Команда	Ответ	Смысл
`{"cmd": "learn.list"}`	`{"words": [...]}`	список выученных слов сессии
`{"cmd": "learn.rejected"}`	`{"words": [...]}`	слова, отклонённые Ctrl+Z (не исправляются в этой сессии)
`{"cmd": "learn.promote", "word": "vercel"}`	`{"ok": true, "changed": true}`	закрепить слово
`{"cmd": "learn.forget", "word": "vercel"}`	`{"ok": true, "changed": true}`	забыть слово (и снять подавление)

`changed` показывает, изменилось ли что-то: повторное закрепление уже закреплённого слова вернёт `false`.

Как этим пользоваться

Из CLI, который сам всё делает:

```
klyrshift learn list              # что демон выучил за сессию
klyrshift learn list-rejected     # слова, отклонённые Ctrl+Z
klyrshift learn promote vercel    # закрепить слово вручную
klyrshift learn forget vercel     # снова разрешить исправлять
```

Если демон не запущен, команда честно сообщает об этом и возвращает код 1: читать нечего и менять нечего.

Почему сокет, а не сигнал или файл

- **Сигнал** не несёт полезной нагрузки: нельзя передать слово, которое надо закрепить.
- **Файл** нарушил бы главное обещание приватности — слова на диске.
- **Сокет** позволяет демону остаться единственным владельцем словаря, а CLI — просто спросить. Слова по-прежнему не покидают память процесса.

Безопасность

Сокет доступен только владельцу (`0600`) и только локально. Демон не доверяет содержимому запроса: неизвестная команда возвращает `{"error": "unknown command"}`, отсутствующее слово — `{"error": "missing word"}`, а исключение внутри обработчика превращается в `{"error": ...}`, а не в падение цикла.

Undo contract

This document states what "undo a correction" guarantees in Klyrshift, what it deliberately refuses to do, and which guarantees are impossible on which platforms. It is the contract the daemon's undo path and its tests are written against.

The one rule

A correction is undoable only while its word is still under the caret.

A restore works by deleting a fixed number of characters and retyping the original. That count is only meaningful if the text at the caret is still the text the daemon produced. The moment the user types anything else, moves the caret, or switches application, the count no longer describes what is on screen. Klyrshift then drops the recorded fix and the undo hotkey becomes a no-op.

A refused undo is always preferable to a corrupted word. This is the same principle the detector uses for a wrong guess ("a wrong guess is worse than no guess").

How undo is performed

Klyrshift never grabs the keyboard (`EVIOCGRAB` is never issued). It cannot swallow a chord, so it cannot intercept the application's own `Ctrl+Z`. Two separate mechanisms follow from that:

1. **The application's own undo (`Ctrl+Z`).** The daemon lets the chord through. The application reverts the daemon's synthetic rewrite exactly as it reverts any other edit, restoring the text the user had typed. Klyrshift only observes the chord, records the word as rejected, and drops its own history so it does not retype the original over text that is already back.
2. **The undo hotkey (`SHIFT+BACKSPACE`, configurable).** The daemon deletes the fixed character count and retypes the original, then switches the layout back if the fix had switched it.

`Ctrl+Z` is not the default undo hotkey for the same reason: binding it would fire the application's undo and the daemon's restore, and the two would race over the same characters.

Guarantees

Guarantee	Mechanism
A fix stays undoable for a bounded time and depth	`undo_window_seconds` (default 10 s), `undo_history_depth` (default 3)
A stale fix is never restored	History is cleared on the first keystroke or caret move after a fix
A stale GUI undo request is refused	The caller passes the history id; only the newest id is accepted
The undo hotkey never re-fixes the restored word	The restored word is recorded as rejected
The typed text never reaches disk	The original lives only in the in-memory undo tuple; the history snapshot is metadata-only
A dry run never touches the text	`--dry-run` reports the undo without replacing anything

What undo does not do

- **It does not undo more than the last fix.** Only the newest entry is restorable; the depth bound exists to age entries out, not to walk back a chain of edits.
- **It does not undo other applications' edits.** Only Klyrshift's own rewrite is restored.
- **It does not run when the daemon could not type the replacement** (an untypable character, a missing backend): nothing was changed, so nothing is recorded.
- **It does not roll back what Klyrshift learned.** Undo restores the screen; the learned vocabulary is a separate, forward-only signal. Undo means "I mistyped, let me retry", not "I reject this word" -- rejection is the separate `Ctrl+Z`-as-a-rejection path that suppresses the word. Un-learning a word on a routine correction would be surprising, so the two are deliberately independent.

Platform limits

Situation	Behaviour	Why
Wayland, no `uinput` access	The fix is refused rather than applied	Without a backend there is no atomic replacement to undo
Fallback backends (`wtype`, `xdotool`)	Undo works, but the original replacement was not atomic	Deletion and retyping are separate calls; a crash mid-replacement can leave text truncated
A password field	Never fixed, so never undone	Wayland hides the focused widget; a stop-word list is the mitigation
An application that eats synthetic keys (some games, Java/Swing)	The fix may not land; undo has nothing to restore	Known limitation

Tests

`tests/test_undo_suppression.py` enforces the contract:

- a bare `Ctrl+Z` suppresses the word and is not re-fixed;
- the word is not fixed again after the chord;
- `Ctrl+Shift+Z` (redo) is not treated as a rejection;
- the undo hotkey restores the last fix while its word is still under the caret;
- a fix is **not** restored once new text was typed or the caret moved;
- `Ctrl+Z` followed by the undo hotkey does not retype the already-restored word;
- the undo history is dropped by a caret move.

Offline operation and privacy

Klyrshift reads everything you type, so it is built to work with no network and to keep the typed text off disk. This document states the guarantees, the two places where a network connection is possible, and how the guarantees are enforced.

Offline guarantee

No core function needs a network connection. Correction, detection, learning, dictionaries, the GUI, diagnostics, history, backup and restore all run locally. The daemon never opens a socket during normal operation.

Function	Network needed?	Fallback when offline
Layout detection	No	—
Text replacement	No	—
Learning (kept / corrected / rejected words)	No	—
User dictionary	No	—
Bundled frequency dictionaries	No	Ship inside the package
Thematic (professional) dictionaries	No, once installed	Bundled copy is used
GUI, diagnostics, history	No	—
Backup / restore	No	Local JSON file
Update check	Optional, **off by default**	Nothing happens
`dict download`	Optional	Falls back to the bundled copy, then to upstream

The two network touch points

Both are explicit and both are bounded.

1. Update check — opt-in, off by default

`update_check_enabled` defaults to `False`. When a user turns it on, the daemon asks the GitHub releases API for the latest tag at most once per day and shows a desktop notification if a newer version exists. It sends no user data — the request is an unauthenticated `GET` of a public URL. The fetch refuses any non-`https` scheme before opening a socket, so a caller-supplied URL cannot make it read `file:` or `ftp:`.

2. Dictionary download — explicit command

`klyrshift dict download` and the thematic-dictionary install fetch word lists. They try, in order: the project mirror, the upstream FrequencyWords source, then the copy bundled in the package. The command works with no network at all because of the bundled copy.

Data locations

All paths honour the XDG base-directory variables.

Data	Path	Contains typed text?
Configuration	`$XDG_CONFIG_HOME/klyrshift/config.toml`	No
User dictionary	`$XDG_CONFIG_HOME/klyrshift/`	Words the user added deliberately
Learning profile	`$XDG_STATE_HOME/klyrshift/`	**Counters only, never a word**
Correction history	`$XDG_STATE_HOME/klyrshift/history.json`	Metadata only: length, layouts, timestamp
Log	`$XDG_STATE_HOME/klyrshift/klyrshift.log`	Metadata only
Cache	`$XDG_CACHE_HOME/klyrshift/`	Downloaded lists

What is never persisted

- **The word being typed.** It lives in memory as a buffer of physical keys and is discarded at the word boundary or on the idle timeout.
- **The original text of a correction.** It is kept only in the in-memory undo tuple, so it can be restored while its word is under the caret, and is never written to the history file or the log.
- **Learned words.** The learning profile stores counters (kept / corrected / rejected), never the words themselves. Learned words live in the daemon's memory for the session only.

The log records the buffer length, the source and target layout, and the action taken. It rotates at 5 MB with three backups.

Other surfaces

- **Notifications** carry a fixed label, never the corrected text.
- **Subprocess arguments** never contain the typed text (argv is world-readable through `/proc`).

- **Tracebacks** replace the buffer with `<redacted>` before formatting, so an exception message that embeds the text cannot leak it.

How this is enforced

`tests/test_privacy_audit.py` runs a session with a distinctive secret string, then searches the log, cache, state directory, config file, the user journal and every subprocess argv for any trace of it. A regression that leaks the text into any of those surfaces fails the test.

`tests/test_learning.py` asserts the learning profile holds counters only and no typed word.

What would break the guarantee

Adding any of the following would violate this document and must not be merged:

- telemetry or analytics;
- cloud learning or an account system;
- a required online API for correction, dictionaries or diagnostics;
- persisting typed text, or a word, anywhere on disk.

Testing

Testing happens on three levels, from cheapest to most realistic. A layer does not replace the ones above it — a green pytest run does **not** mean the daemon works on a real desktop.

Level	What it runs	How to run	Proves
1. Unit / integration	Real code, faked OS boundary	`make test`	Logic, parsing, config, conversion
1b. Headless scenario	Real daemon, faked keyboard and injector	`klyrshift simulate '...'`	End-to-end keystroke behaviour without a keyboard
2. Package smoke test	The built `.deb`, installed into a clean `debian:12` container	`make smoke-test`	The package installs, all runtime imports resolve, the CLI starts
3. Manual desktop test	A real GNOME session (Wayland and X11)	This document	Permissions, D-Bus, `uinput`, real applications

Level 1b: the headless scenario simulator

The daemon reads a keyboard and injects synthetic keys, so exercising it end to end normally needs a desktop. `klyrshift simulate` replaces the keyboard, the layout switcher and the text injector with in-process fakes and drives the **real** daemon with a scripted scenario, then prints what the user would see. It is what makes the correction logic testable without a keyboard — by a person or by an automated agent:

```
klyrshift simulate 'type ghbdtn, space'
## Экран: привет
## Режим: auto · раскладка: ru
## замена −7 +'привет ' @ru
```

```
klyrshift simulate 'mode manual, type langauge, space'      # nothing happens
klyrshift simulate 'type ghbdtn, space, ctrl z, type ghbdtn, space'  # stays ghbdtn
klyrshift simulate --json 'type ghbdtn, space'              # machine-readable
```

Steps are comma-separated: `type <text>`, a named `key`/`press` (letters, digits, punctuation by name — `dot`, `comma`, `minus`, `slash`, … — and `space`/`enter`/`tab`/`backspace`/`esc`/arrows/`f1`..`f24`), a chord (`ctrl+z`, `shift+f12`), `dblshift`/`dblctrl`/`dblalt`, `layout <name>`, `mode <auto|manual|hybrid>`, `app <name>` (pretend that application is focused, for the per-app mode/exception rules), `config key=value; ...` (bools, ints, `[a, b]` lists and `{k: v}` dicts) and `wait` (the idle flush). `type` presses the physical keys that produce the text on a US keyboard, so the active layout decides the glyphs on screen — that is how the wrong-layout case is reproduced. The simulator also renders the screen by applying each replacement, so a scenario asserts what the user would see, not just the raw replacement calls.

Level 2: package smoke test

`scripts/smoke_test.sh` builds nothing itself; it installs `dist/*.deb` into a throwaway `debian:12` container and checks that `klyrshift version`, `--help`, `status` and `doctor` run, and that the daemon starts in `--dry-run`. It catches the class of bug that pytest cannot: a missing runtime dependency or a broken entry point in the installed package. It does **not** need a GUI, so it also runs in CI (see `release.yml`).

```
make build-deb
make smoke-test
```

The container has no keyboard devices, so `doctor` is expected to report failures there; the smoke test only asserts that the commands execute.

`scripts/deb_selfsufficiency_test.sh` installs the `.deb` with no manual step and checks that a purge leaves nothing behind. `scripts/upgrade_test.sh` installs a stand-in for the pre-rename `linguafix` package and then the new `.deb` over it, and checks that the old package is removed, only one daemon unit is enabled and the user's config survived the rename. Neither needs a desktop.

Level 3: manual test plan

The automated suite runs headless and replaces the operating-system boundary with fakes. It cannot verify permissions, D-Bus behaviour or how real applications react to synthetic input. This plan covers what must be checked by hand on a real desktop.

`scripts/acceptance_check.sh` automates the parts of this plan a script can check on a real machine (installed package, no leftover pre-rename package, session type, layout backend, text injector, device access, the daemon, the autostart toggle). Run it first, inside the graphical session:

```
scripts/acceptance_check.sh          # diagnose
scripts/acceptance_check.sh --fix    # also load the uinput module
```

It exits non-zero on a hard failure and prints the manual steps that still need a human (a real correction, the hotkeys, the undo, the log). A container cannot run it meaningfully: there is no seat and no `/dev/uinput`.

Quick start (copy-paste)

Run this block first. It installs Klyrshift, prints a diagnosis and starts watching the log. It is safe to paste as one piece. It exercises the **zero-config `.deb` path** — the one a regular user takes.

```
## 1. Install the package. No usermod, no manual enable.
sudo apt install ./klyrshift_0.2.10_all.deb

## 2. Verify that nothing else is required.
systemctl --user is-enabled klyrshift.service   # -> enabled
cat /usr/lib/udev/rules.d/71-klyrshift.rules    # -> TAG+="uaccess", no GROUP="input"

## 3. If a session was already open during install, log out and back in once.
## uaccess is granted when the user logs in at the seat.

## 4. After (re)login — verify the environment.
getfacl /dev/input/event3 2>/dev/null | grep "user:$(id -un)" && echo "OK: ACL present"
klyrshift doctor
klyrshift status

## 5. Watch the daemon while you test (Ctrl-C to stop).
journalctl --user -u klyrshift.service -f
```

For the **developer path** instead, replace step 1 with:

```
git clone https://github.com/Pavel1778/Klyrshift.git
cd Klyrshift
bash install.sh --yes
sudo udevadm control --reload-rules && sudo udevadm trigger
```

If `klyrshift doctor` shows no □, continue with the scenarios below. If it shows a □, stop and attach the output to issue #5 — that is already a useful result.

Environments

Test on both, in a virtual machine:

#	Distribution	Session	GNOME
1	Debian 12 (netinst, GNOME)	Wayland (default)	43
2	Ubuntu 22.04 LTS (GNOME)	Wayland (default)	42

Then repeat the core scenarios switching to **X11** (choose "GNOME on Xorg" in GDM).

A. Installation

`.deb` path (primary):

1. `sudo apt install ./klyrshift_0.2.10_all.deb` — note every prompt and any failure.
2. Confirm the udev rule: `cat /usr/lib/udev/rules.d/71-klyrshift.rules` — uses `TAG+="uaccess"`, no `GROUP="input"`.
3. `systemctl --user is-enabled klyrshift.service` — prints `enabled`, without you running `systemctl --user enable` by hand.
4. Log out completely and back in (only needed if a session was already open).
5. `getfacl /dev/input/event3` — shows a `user:<you>:rw-` ACL entry.
6. `klyrshift status` — daemon running, backends reported.
7. `ls -l /dev/input/event*` — readable by your user.

Developer path:

1. Run `bash install.sh` interactively. Note every prompt and any failure.
2. Confirm the apt packages: `dpkg -l | grep -E 'evdev|uinput|wtype|xdotool|appindicator'`.
3. Confirm the udev rule: `cat /etc/udev/rules.d/71-klyrshift.rules` — `uaccess`.
4. `sudo udevadm control --reload-rules && sudo udevadm trigger`.
5. `klyrshift doctor` — no .

B. Daemon startup

1. `systemctl --user status klyrshift.service` — active, no restart loop.
2. `journalctl --user -u klyrshift.service -n 50 --no-pager` — no tracebacks.
3. `klyrshift start --foreground --dry-run` — the daemon runs and logs "Dry run" instead of touching the text.

C. Correction matrix

In each application, switch to the **wrong** layout, type `ghbdtn` (meaning "привет"), and record whether Klyrshift corrected it:

Application	Wayland	X11	Notes
gnome-terminal			
Firefox (address bar + text area)			
Chrome / Chromium			
VS Code			
Gedit / Text Editor			
SuperTuxKart (chat)			

Also test:

- A Russian word typed on the US layout and vice versa.
- Uppercase (`Ghbdtn`).
- Punctuation (`ghbdtn?`, `;b`).

- A password field: the correction must **not** fire for stop words, but note whether it fires for non-stop-word content in a password field (known limitation).

D. Layout switching

1. With `g3kb-switch` installed: `g3kb-switch -p` before and after a correction.
2. Without `g3kb-switch` on Wayland: text is replaced but layout is unchanged (document the behaviour).
3. On X11: confirm the `setxkbmap` fallback works.

E. Tray and notifications

1. Tray icon appears (requires the AppIndicator GNOME extension).
2. "Показать статус" shows a notification with the current layout.
3. "Исправить сейчас" triggers a correction.
4. `notify_on_fix = true` shows a notification after each fix.

F. Privacy and logs

1. Type a word, trigger a correction, then inspect `~/.local/state/klyrshift/klyrshift.log`.
2. The log must contain only metadata (length, layouts) — never the typed text.
3. Confirm rotation config: 5 MB × 3 files.

G. Conflicts

1. Start IBus or Fcitx and check for double conversions or missed fixes.
2. Document the outcome.

H. GUI

Run `klyrshift gui` and check the window stays up.

1. **Big on/off toggle** — turns on and off, and the state agrees with `klyrshift status` in both directions, several times in a row.
2. **Autostart switch** — toggles, and the setting survives a GUI restart. Verify with `systemctl --user is-enabled klyrshift.service` (or the presence of `~/.config/autostart/klyrshift.desktop`): turning it off must remove both.
3. **Mode switcher** Auto/Manual/Hybrid — agrees with `klyrshift mode`.
4. **Dictionary tab** — add `vercel`, confirm `klyrshift dict list` shows it, then remove it and confirm it disappears. Type `муксуд` and expect `vercel`.
5. **Advanced tab** — every group expands, and a change is saved to `config.toml` and survives `klyrshift restart`.
6. **Export/import** — export to JSON, reset, import; the settings come back (`klyrshift export` / `klyrshift import` work too).
7. **Guided tour** — on a fresh config the first-run offer appears; starting it walks the steps, each highlighting the widget it describes. Skipping, finishing and the "Повторить вводный тур" button on the help page all work, and a restart after finishing shows no offer.

8. **Context help** — the "?" on a settings group opens the matching topic; the "Помощь и обучение" page lists every topic, runs the environment check and shows the known limitations.

Reporting

File one issue per problem found, using the bug report template (`.github/ISSUE_TEMPLATE/bug_report.md`). Apply the `needs-testing` label while triaging.

Attach a diagnostic bundle — one command collects everything a maintainer needs (the `doctor` output, configuration, log tail, user journal, environment, device permissions and package versions):

```
klyrshift collect-logs
## -> ~/klyrshift-logs-YYYYMMDD-HHMMSS.tar.gz
```

The daemon logs only metadata, so the bundle never contains the text you typed. See `docs/TROUBLESHOOTING.md` for details.

QA engine

An autonomous testing layer on top of the existing suite. It drives the real daemon through the headless simulator (`klyrshift simulate`), checks independent invariants, classifies failures, minimises them and keeps a regression store.

It is **not** a replacement for the pytest suite and it does **not** talk to `evdev`/`uinput` or a compositor. It tests the logic and observable behaviour of the daemon, exactly like the simulator does; the real device stack still needs a desktop (see TESTING.md).

Run it

```
klyrshift qa run                        # the curated corpus, deterministic
klyrshift qa run --generate 50 --seed 7 # + 50 generated scenarios
klyrshift qa run --changed src/klyrshift/daemon.py   # focus on what a change touched
klyrshift qa run --report qa/out        # write qa-report.json and qa-report.md
klyrshift qa run --json                 # the whole report on stdout
klyrshift qa list                       # the regression store
klyrshift qa regress                    # re-check every open regression
```

The default run is offline and deterministic: the same seed and corpus produce the same report with no network and no API key. The exit code is `0` only when nothing failed and no scenario broke the harness.

What it checks

Each scenario names the invariants to check. They are pure functions of the screen, the replacements and what the user pressed (an independent oracle -- the typed counts come from the steps, not from the simulator):

Invariant	What it proves
`no-lost-characters`	No Space/Enter/Tab the user pressed disappeared
`space-preserved` / `enter-preserved` / `tab-preserved`	The boundary key survived its own fix
`punctuation-preserved`	A `.` / `,` is not dropped
`manual-mode-silent`	`mode = manual` rewrites nothing without a hotkey
`ctrl-z-suppressed`	A `Ctrl+Z` rejection is recorded
`no-mixed-script-replacement`	A fix never emits a token mixing Latin and Cyrillic
`layout-switched`	A fix left the active layout at its target
`no-over-deletion`	A replacement never deletes more than was pressed
`screen-matches-expectation`	The screen equals the scenario's `expect.screen`
`layout-is-known`	The final layout is one the converter knows

An invariant never mutates anything, and the engine never changes an assertion to make a failure go away: a failing invariant is reported, classified and recorded.

How a failure is classified

Class	Meaning
`confirmed-bug`	A **structural** invariant failed and it reproduces
`suspicion`	Only a hand-written `expect` failed; the expectation may itself be wrong
`flaky`	The failure did not reproduce on a second run
`harness-error`	The simulator rejected a step (a broken scenario, not a product bug)

A confirmed bug is minimised by delta-debugging: the engine drops steps (and trims `type` steps) while the failure signature stays identical, so the minimal reproduction is the shortest step list that still fails the same way.

The scenario format

A scenario is JSON data, never code:

```
{
  "id": "layout/wrong-layout-space",
  "title": "A wrong-layout word is fixed on Space",
  "category": "layout",
  "difficulty": "basic",
  "tags": ["layout", "boundary"],
  "steps": ["type ghbdtn", "space"],
  "config": {},
  "layout": "us",
  "expect": {"screen": "привет "},
```

```
  "invariants": ["no-lost-characters", "screen-matches-expectation"],
  "seed": 0,
  "notes": "",
  "origin": "manual"
}
```

The bundled corpus lives in `src/klyrshift/data/qa/scenarios.json`; pass your own with `--scenarios <file-or-dir>`. A scenario is only ever fed to the simulator's step parser -- no step is executed as a command.

Generation and the LLM hook

`klyrshift.qa.generator` produces deterministic scenarios from a seeded grammar and expands around the curated corpus and the known bug classes. It also maps a changed source file to the functional areas it affects (`categories_for_files`), so a run can focus on what a change touched.

An LLM provider is an **optional** input: an object with a `propose(focus, seed)` method returning step lists. The generator validates every proposal through the scenario model and drops anything malformed; nothing it returns is executed as code, and the deterministic run never depends on it.

The regression store

Confirmed bugs are recorded in `qa/regressions.json`: the original scenario, the minimal reproduction, the violated invariant and a status. The store is only ever appended to; a fixed entry is marked `fixed` (by `klyrshift qa regress`) and kept, never deleted.

Limits

- It tests the same surface the simulator does: the daemon's logic with a faked keyboard, switcher and injector. It cannot prove the real `evdev`/`uinput` path or a live compositor.
- The generated scenarios are deterministic, not exhaustive; a green run is evidence, not proof.
- The report shows only measured numbers. There is no coverage percentage.

Tests

`tests/test_qa_engine.py` pins each piece and the end-to-end run, including that the engine records a structural failure and never invents one on a green corpus.

Why the automated run breaks

An investigation into "почему автопрогон ломается" (why the auto-run breaks), performed with the project's own tooling: the headless simulator (`klyrshift simulate`) and the autonomous QA engine (`klyrshift qa run`).

This document records what was measured, the defect that reproduces, its root cause, the blast radius, and the recommendations. It changes no product code; the fix is proposed, not applied.

0. What "автопрогон" is in this repository

The word is ambiguous, so the investigation started by pinning it down. In Klyrshift the automated (unattended) run is the autonomous QA engine:

- `klyrshift qa run` — the documented "Autonomous QA" engine (QA_ENGINE.md); it drives the real daemon through the simulator, checks independent invariants, classifies failures, minimises them and records regressions.
- the CI job `Autonomous QA (deterministic, offline)` (`.github/workflows/ci.yml`) runs exactly `klyrshift qa run --generate 60 --seed 12345 --no-record`.

(The literal phrase "Автономный прогон" is the NEXUS orchestrator's term for one of its runs against this repository; that machinery lives in another project and is out of scope here. The only automated run that is part of Klyrshift is the QA engine above, and it is where the breakage reproduces.)

A run is "broken" when it reports a `harness-error` (a scenario the simulator rejects) and the CLI exits non-zero, which fails the CI job. That is what happens below.

1. Summary

Symptom	A QA run aborts with `harness-error: scenario rejected by the simulator: unknown key '...'` and exits `1` (CI red).
Where	`src/klyrshift/qa/executor.py` (`build_script`) against `src/klyrshift/simulate.py` (`run_scenario`).
Trigger	Any scenario whose `config` has a **list or dict with more than one element**.
Root cause	The script builder joins steps with `", "`, but the simulator splits the script on `","`. The commas inside a rendered `[a, b]` list or `{k: v}` dict are indistinguishable from step separators, so one step is shredded into several bogus steps.
Status	Reproducible on the current `main` (HEAD `2a296d0`). Not covered by any test.

2. Reproduction

The smallest reproduction uses the engine's own public surface.

```
from klyrshift.qa.executor import build_script
from klyrshift.qa.scenario import Scenario
from klyrshift.simulate import run_scenario, ScenarioError

s = Scenario(id="per-app/two", steps=["type ghbdtn", "space"],
             config={"exceptions_apps": ["kitty", "code"]})
script = build_script(s)
## -> 'config exceptions_apps=[kitty, code], type ghbdtn, space'

script.split(",")
## -> ['config exceptions_apps=[kitty', 'code]', 'type ghbdtn', 'space']

run_scenario(script)
## ScenarioError: unknown key 'code]'
```

The same happens from the CLI with a scenario file:

```
$ klyrshift qa run --scenarios two-apps.json --no-record
QA: проверено 37, успешно 36, провалов 0, ошибок стенда 1
$ echo $?
1
```

and directly through the simulator, whose docstring advertises the list form (`config key=value; ... [a, b] lists and {k: v} dicts are parsed`):

```
$ klyrshift simulate 'config exceptions_apps=[kitty, code], space'
ScenarioError: unknown key 'code]'
```

Which shapes break

config value	Rendered step	Result
`{"exceptions_apps": ["kitty"]}`	`config exceptions_apps=[kitty]`	ok
`{"exceptions_apps": ["kitty", "code"]}`	`config exceptions_apps=[kitty, code]`	**harness-error**
`{"app_modes": {"code": "manual"}}`	`config app_modes={code: manual}`	ok
`{"app_modes": {"code": "manual", "chat": "auto"}}`	`config app_modes={code: manual, chat: auto}`	**harness-error**
`{"app_layouts": {"code": "us", "chat": "ru"}}`	`config app_layouts={code: us, chat: ru}`	**harness-error**

A single-element list or dict happens to survive; **any two-or-more-element list/dict does not**. The failure is silent in the sense that the engine classifies it as a `harness-error` (a "broken scenario"), not as a product bug — so the run does not invent a false bug, but it does fail, and the scenarios that exercise multi-entry per-app configuration can never run at all.

3. Root cause

Two functions disagree about the separator:

- `src/klyrshift/qa/executor.py`:

```python def _render_value(value): ... if isinstance(value, list): return "[" + ", ".join(str(item) for item in value) + "]" if isinstance(value, dict): return "{" + ", ".join(f"{k}: {v}" for k, v in value.items()) + "}" return str(value)

def build_script(scenario): parts = _config_steps(scenario) + list(scenario.steps) return ", ".join(parts) ```

It uses `", "` both as the step separator and as the intra-list/dict separator.

- `src/klyrshift/simulate.py`:

`python for raw_step in scenario.split(","):`

It splits on a bare `","` with no notion of nesting.
```

Because the two separators are the same character, a rendered `[kitty, code]` becomes two steps, `config exceptions_apps=[kitty` and `code]`, and the second is rejected as an unknown key.

The design intent is visible and inconsistent: `_coerce` in the simulator explicitly parses `[a, b]` and `{k: v}`, and `test_qa_engine.py` asserts the builder emits `[a, b]` and `{code: manual}` — but nothing ever feeds that output back through the parser, so the mismatch was never exercised.

4. Why the test suite does not catch it

`tests/test_qa_engine.py::test_build_script_renders_lists_and_dicts` checks the string the builder produces, never that the simulator can run it:

```
script = build_script(Scenario(id="x", steps=["space"],
    config={"stop_words": ["a", "b"], "app_modes": {"code": "manual"}}))
assert "[a, b]" in script
assert "{code: manual}" in script
```

That test passes while the very input it builds is unrunnable. It gives false confidence: the renderer is "tested", the parser is "tested", but the seam between them is not.

The curated corpus does not trigger it either: 36 scenarios, exactly one with a non-empty `config`, and that one has a **single-element** list (`per-app/exception-applies-immediately` → `{"exceptions_apps": ["myapp"]}`), which survives by luck. The deterministic generator only ever emits scalar config (`on_tab=True`, `on_punctuation=True`), so `--generate N` never reaches the broken shape. The default run is therefore green; the defect only appears once a scenario (hand-written, LLM-proposed, or a future curated one) carries a two-element list or dict.

5. Blast radius

- **The QA engine (`klyrshift qa run`).** Any scenario with a multi-entry per-app config (`exceptions_apps`, `app_modes`, `app_layouts`) is a `harness-error`; the CLI exits `1`. The `qa` CI job fails for such a run.
- **`klyrshift simulate` (the CLI and `python -m klyrshift.simulate`).** The documented `[a, b]` / `{k: v}` config form is unusable with more than one entry.
- **The `--scenarios <file>` input path.** A user-supplied corpus (or an LLM-proposed one, which the generator validates as data and then hands to the same builder) hits it immediately.
- **Not affected:** the curated corpus as shipped, the deterministic generator, and the default CI invocation. That is precisely why it has gone unnoticed.

A related, lower-severity sharp edge sits on the same seam: a literal comma inside a `type` step is also a step separator, so `type hello, world` is parsed as `type hello` + a bogus `world` step (`ScenarioError: unknown key 'world'`), and `type a,b` silently types only `ab`. This is a property of the comma-separated DSL, not of the config renderer, but it is the same class of "the separator is overloaded" defect and is worth noting when the DSL is touched.

6. Recommendations

Ordered by value; the first is the actual fix.

1. **Make the script separator unambiguous (primary fix).** Teach the simulator's splitter to ignore separators inside `[ ... ]` / `{ ... }`, so a rendered list/dict stays one step. This is a small, self-contained change in `simulate.run_scenario` (track bracket depth while splitting) and fixes both the QA engine and the `simulate` CLI at once, with no change to the scenario format. Verified locally: with a bracket-aware split, the three broken shapes above all run to `привет` and a plain `type ghbdtn, space` is unchanged.

Alternative: give the config renderer a separator the DSL does not use (e.g. `|` or `;` inside lists/dicts) and teach `_coerce` to accept it. This touches two places and changes the on-the-wire script text; the splitter fix is smaller and keeps the documented `[a, b]` syntax.

1. **Close the test gap at the seam (do this with the fix).** Replace the string-only assertion with an end-to-end one: build the script for a scenario with a two-element list and a two-element dict, run it through `run_scenario`, and assert the screen (not the string). Add at least one curated scenario with a multi-entry `exceptions_apps` so the shipped corpus exercises the path. The existing test should stay, but it must not be the only guard.

2. **Reject an unrunnable step at build time, not at run time.** `build_script` could round-trip its own output through the simulator's parser and raise a `ScenarioFormatError` naming the scenario if a step does not survive. That turns a `harness-error` in the middle of a run into a clear, early failure.

3. **Consider the `RunConfig.repeats` dead knob (unrelated, found while reading).** `RunConfig.repeats` is documented as "how many times a failing scenario must reproduce", but nothing reads it: `runner.run` calls `analyse(...)` which calls `reproduces(scenario)` with the hard-coded default `runs=2`. Either wire the knob through or delete it; a knob that silently does nothing is a trap for the next reader.

4. **Note the comma-in-`type` limitation in the DSL docs** (`simulate` module docstring and QA_ENGINE.md) so a scenario author knows a literal comma cannot appear in typed text, or extend the DSL to escape it.

7. What was checked and is clean

- `pytest tests/` — 1776 passed, 111 skipped (the defect is real but uncovered).
- `ruff check .`, `black --check .`, `isort --check-only .` — clean.
- `klyrshift qa run --generate 60 --seed 12345` — 96/96 (the default path).
- 30 seeds × `--generate 120` — 0 failures, 0 harness errors (the generator never emits the broken shape).
- The curated corpus — all 36 scenarios' built scripts run without a harness error.
- QA determinism — two runs of the same seed agree on every field except the wall-clock `duration_s`.

Test strategy

How Klyrshift is tested, which numbers are tracked, and how a change is judged. This complements TESTING.md, which describes the three levels and the manual desktop plan; this document covers the quality metrics and the regression thresholds.

Layers

Layer	Tool	Runs in CI	Proves
Lint / types	`ruff`, `isort`, `black --check`, `mypy --strict`	Yes	Style, imports, type soundness
Unit / integration	`pytest`, coverage floor 80 %	Yes	Logic, parsing, config, conversion, daemon policy
Detector quality	`scripts/measure_detector.py`	Manually	Recall, false positives, corruption, speed on fixed corpora
Detector cost	`scripts/measure_latency.py`	In CI (`latency-budget`)	ms/word and peak memory on fixed corpora, with a budget
Package smoke	`scripts/smoke_test.sh`	Yes	The built `.deb` installs and its entry points run
Manual desktop	`docs/TESTING.md` level 3	No	Permissions, D-Bus, `uinput`, real applications

Tests replace the OS boundary (evdev, GTK, `uinput`) with fakes but always run real project code. There are no mocks of Klyrshift's own modules.

Detector metrics

`scripts/measure_detector.py` measures four numbers on a fixed slice of the shipped corpora. **Only identical slices with identical conditions are comparable**; a top-1000 run and a top-3000 run are different measurements.

Metric	Definition	Want
Recall	share of target-language words typed on the wrong layout that are sent back	high
Missed	the complement of recall	low
False positives	real, correctly-typed words that would still be moved (a wrong fix)	**0**
Corruption	conversions where a character is lost or mangled	**0**
ms/word	processing time per word	flat or lower

```
PYTHONPATH=src python scripts/measure_detector.py --limit 1000
PYTHONPATH=src python scripts/measure_detector.py --limit 3000
```

Recorded baseline

Measured on the current tree, Debian 13, Python 3.13:

Slice	Direction	Recall	False pos.	Corruption	ms/word
top 1000	ru on us	92.8 %	0.0 %	0	0.008
top 1000	en on ru	95.1 %	0.0 %	0	0.008
top 3000	ru on us	93.6 %	0.0 %	0	0.008
top 3000	en on ru	97.3 %	0.0 %	0	0.008

The residual miss rate on short words (1–2 characters) is expected: they carry almost no statistical signal and are skipped by `min_word_length`.

Latency and memory baseline

`scripts/measure_latency.py` measures the two costs a user feels and a CI budget could guard. It is kept separate from the quality script because `tracemalloc` must be switched off before timing: it hooks every allocation, so a loop measured under it reports the tracer's cost, not the detector's.

```
PYTHONPATH=src python scripts/measure_latency.py --limit 3000 --repeats 3
```

Pass `--budget-ms` and `--budget-mib` to turn the run into a check: the process exits non-zero when the slowest direction or the peak memory exceeds the budget. The `latency-budget` CI job runs exactly this with loose thresholds (0.5 ms/word, 64 MiB) — the point is to catch an order-of-magnitude regression on a noisy shared runner, not to police a few percent of noise.

Slice	Direction	ms/word	Peak traced memory
top 3000	ru on us	0.007	—
top 3000	en on ru	0.007	—
build + first word	—	—	4.3 MiB

Peak memory covers building the detector and deciding one word, which is what a long-running daemon holds; it is not per-word growth.

Regression thresholds

A change must not:

- introduce a false positive or a corruption (both must stay at **0**);
- lower recall on a recorded slice by more than 0.5 percentage points;
- raise `ms/word` by more than 20 % on a recorded slice;
- raise peak traced memory by more than 20 % on the recorded slice;
- drop coverage below the 80 % CI floor;
- turn a passing test into a skip.

The "both layouts spell a real word" class is covered separately by `tests/test_ambiguous_words.py`, which sweeps the shipped corpora for cross-layout collisions and pins

that neither spelling is ever moved. It is a test rather than a script because it must fail the build, not be read by eye.

Improving one metric at the cost of another must be stated explicitly in the PR. An "improvement" that trades a false positive for recall is a regression.

Adding a regression test

Every defect fixed gets a test that fails before the fix and passes after. The test is named for the behaviour, not the change, and it asserts on the same surface the user saw (the replacement list, the buffer, the history), not on an internal counter that could drift.

Example: the stale-undo defect (a fix restored after the user typed new text) is covered by `tests/test_undo_suppression.py`, which asserts the second replacement never happens.

What the automated suite cannot prove

- That real applications accept the synthetic keystrokes.
- That `uinput` and `/dev/input` are reachable under the user's session.
- That layout switching works through `g3kb-switch` on the installed GNOME.
- That the GTK4 GUI renders (needs a display).

These require `docs/TESTING.md` level 3 on a real machine and are the reason the platform matrix in PLATFORM_SUPPORT.md marks those rows as unverified rather than supported.

Releasing

This is the checklist to run before publishing a release. It is written for `v0.2.9`, which adds always-on learning and hardens undo, but the same steps apply to any version.

Nothing here is executed automatically. A release is published by pushing a tag (`git push origin v0.2.9`), which triggers `.github/workflows/release.yml`. Read the whole list before pushing the tag: the tag is the point of no return.

0. Decide what the version is

- [ ] The version is **one** value, in five places that a test compares: `pyproject.toml`, `src/klyrshift/__init__.py`, `site/src/consts.ts`, `site/package.json` and the landing page JSON-LD (which reads `VERSION` from `consts.ts`). `site/package-lock.json` carries it too and is bumped with it. `python3 scripts/release_consistency.py --source` must print `ok`.
- [ ] The `CHANGELOG.md` section for this version exists and is not still under `## [Unreleased]`. A released version must have its own `## [x.y.z] - date` heading, or the tree claims a version the changelog does not describe. **The dated heading is also what makes the `.deb` reproducible**: the build takes its timestamp from that line (not from git), so a release PR must add it before the `.deb` is built. Without it the build falls back to the commit time and stops being byte-identical across hosts. See `scripts/build_deb.sh`.
- [ ] The version is not already used. `git ls-remote --tags origin` must not list it. `0.2.8.2` has a **changelog heading from 2026-10-08** that was never tagged; `0.2.8.3` and `0.2.8.4` are tagged and published. `0.2.9` is the next number and does not reuse either.

1. Local checks

- [ ] `make check` — lint (ruff, isort, black), `mypy --strict`, and the test suite with coverage. The suite must stay above **80 %**.
- [ ] `bash scripts/build_deb.sh` builds `dist/klyrshift_<version>_all.deb`.
- [ ] `lintian --fail-on error dist/klyrshift_<version>_all.deb` is clean. The `package` CI job runs this, together with a check that the man page and both completions are inside the `.deb` and that the man page is compressed.
- [ ] `python3 scripts/release_consistency.py --source` — the tree agrees with itself and the asset names match what the workflow uploads.
- [ ] `python3 scripts/release_consistency.py --latest` — the newest published release matches the tree. **This is expected to fail while the version is bumped but not yet released**; it must pass once the tag is published.

2. The package on clean Debian images

These need Docker. Run both on **Debian 12 and Debian 13**, since the layout backend and the `tomli`/`tomllib` split differ between them.

- [ ] `bash scripts/smoke_test.sh` — the installed CLI starts and answers `version`, `--help`, `status` and `doctor`.
- [ ] `bash scripts/deb_selfsufficiency_test.sh` — `apt install ./…` needs no `usermod`, no re-login and no manual `systemctl enable`; `apt remove --purge` leaves nothing behind.
- [ ] `SMOKE_IMAGE=debian:13 bash scripts/deb_selfsufficiency_test.sh` — same on Debian 13.

3. Upgrade from the old `linguafix` package

`bash scripts/upgrade_test.sh` automates this on a clean `debian:13` image (`UPGRADE_IMAGE=debian:12` for the other release): it installs a stand-in for the old package, installs the new `.deb` over it with apt, and checks the three conditions below plus the config migration. The manual steps, for a real machine:

- [ ] Install the previous `linguafix_0.2.8_all.deb`, then install the new `klyrshift_<version>_all.deb` over it. After the upgrade:
- `dpkg -l | grep -E 'linguafix|lingvix|klyrshift'` shows only `ii klyrshift`;
- `ls /etc/systemd/user/default.target.wants/` lists only `klyrshift.service`;
- `pgrep -af 'klyrshift start' | wc -l` is `1` — two daemons must never run.
- [ ] The configuration **is** migrated on first load: `~/.config/lingvix` (or `~/.config/linguafix`) is copied into `~/.config/klyrshift`, and the old file is left in place. Confirm the manual copy still works (see INSTALL.md).

4. On a real keyboard (this cannot be done in CI)

The daemon reads `/dev/input/event*` through `evdev` and injects through `uinput`. No container in this project has those nodes, so this section is the only place the kernel boundary is exercised.

Run `bash scripts/acceptance_check.sh` on a real GNOME session and complete the manual steps it prints:

- [ ] In **both** session types (Wayland and X11): type `ghbdtn`, press Space, and confirm it becomes `привет`; type `привет`, press Space, and confirm it is left alone.
- [ ] The fix hotkey (SHIFT+SHIFT) corrects a word with no Space; SHIFT+BACKSPACE undoes the last fix.
- [ ] Each mode behaves as documented: `auto` corrects on its own, `hybrid` withholds a borderline word until the hotkey, `manual` waits for the hotkey.
- [ ] A word added to the exceptions list is left alone.
- [ ] `https://example.com/a_b` and `/usr/lib/klyrshift/x.py` are never rewritten.
- [ ] The log holds no typed text: `grep -c привет ~/.local/state/klyrshift/klyrshift.log` is `0`.
- [ ] Exactly one daemon runs: `pgrep -af 'klyrshift start' | wc -l` is `1`.

Record the session type, the distribution and the result for each line. A release may not claim the input path works until this section is done.

5. Documentation and the landing page

- [ ] `CHANGELOG.md` describes every user-visible change since the last release, including the breaking rename and the config-directory migration.
- [ ] `README.md` and `docs/*` show the current version in every `apt install ./klyrshift_<version>_all.deb` example. A test enforces this.
- [ ] The landing page builds (`cd site && npm run build`) and the JSON-LD `softwareVersion`, the hero version and the download links agree.
- [ ] `python3 scripts/make_og_image.py` (after a site build — it needs the emitted fonts) regenerates `site/public/og-image.png` with no diff: the link preview is generated, and a hand-edited one shows the dead name again.
- [ ] The canonical origin, `og:url` and the `hreflang` alternates point at the live hosts (`klyrshift.layero.app`, `klyrshift.vercel.app`). The pre-rename `linguafix.*` names now return 404, so a canonical that points at them makes search engines drop the page. A test enforces this.

6. The release assets

The release workflow attaches, and `klyrshift dict install` depends on:

- [ ] `klyrshift_<version>_all.deb` — the package.
- [ ] `klyrshift-<version>.tar.gz` — the source distribution.
- [ ] `<lang>-50k.txt` for every frequency list (`ru`, `en`, `uk`, `de`, `fr`).
- [ ] `klyrshift-dict-<slug>.tar.gz` for **every** thematic category that has a `README.md` in `dictionaries/` (twelve today), each containing `<lang>-<slug>-1k.txt` for `ru` and `en`.
- [ ] `SHA256SUMS.txt`.

`python3 scripts/release_consistency.py --tag v<version>` lists the expected names; the workflow runs the same check against the staged files before it publishes.

7. Publish, then verify

- [ ] CI is green on the **exact commit** the tag will point at. Do not tag a commit whose checks have not finished.
- [ ] Tag and push: `git tag -a v<version> -m '...' && git push origin v<version>`.
- [ ] Watch the release workflow: the consistency gate, the smoke test, the zero-config test and the post-publish link check must all pass.
- [ ] `python3 scripts/release_consistency.py --latest` passes — the newest published release now matches the tree.
- [ ] The download links on the landing page resolve. Redeploy the Layero host if it still serves the previous version.

Renamed hosts

The GitHub repository rename is done: the repository is `Pavel1778/Klyrshift` and GitHub redirects the old `Pavel1778/LinguaFix` slug. The site hosts were renamed with it, so `klyrshift.layero.app` and `klyrshift.vercel.app` are the live origins and the pre-rename `linguafix.*` names return 404.

The Layero mirror is still deployed by hand (see site/README.md), so it can lag behind `main`; redeploy it with the Layero credentials before announcing a release, because the site advertises the version.

Contributing

Thanks for your interest in Klyrshift! This document describes how to set up a development environment and the standards we follow.

Development setup

```
git clone https://github.com/Pavel1778/Klyrshift.git
cd Klyrshift
bash scripts/dev_setup.sh
```

`dev_setup.sh` creates a virtual environment, installs the package in editable mode together with the development extras, and installs the git hooks if `pre-commit` is available.

Running the tests

```
make test          # pytest with coverage
make lint          # ruff + black --check
make typecheck     # mypy --strict
make check         # all of the above
```

Or directly:

```
python -m pytest tests/ -q --cov=klyrshift --cov-report=term-missing
ruff check .
```

```
black --check .
mypy --strict .
```

The test suite must stay above **80 %** coverage. The operating-system boundary (`evdev` devices, `uinput`, `g3kb-switch`, `setxkbmap`, `wtype`, `xdotool`) is replaced with lightweight fakes; the tests exercise the real daemon logic.

Code standards

- Python 3.10+, fully typed. `mypy --strict` must pass.
- Formatting is enforced by `black` (line length 100) and imports by `isort` (black profile).
- Linting is enforced by `ruff`. Note that `RUF001`–`RUF003` are disabled because the project intentionally contains Cyrillic user-facing strings.
- All public functions and classes have Google-style docstrings.
- No `print()` in production code — use the `logging` module.
- External commands are always invoked as `subprocess.run([...], check=False, capture_output=True, timeout=N)`, never with `shell=True`.
- The daemon must never crash on a recoverable error: catch, log, continue.

Adding a language

1. Add the layout map to `src/klyrshift/data/layouts.json`.
2. Add `src/klyrshift/data/ngrams_<lang>.json` with a `words` list and a `bigrams` frequency map.
3. Extend `LanguageDetector` if the new language needs special casing.
4. Add tests under `tests/`.

Commit messages

Use conventional, atomic commits:

```
feat: add German layout support
fix: avoid crash when no input devices are present
docs: clarify Wayland limitations
```

Building the .deb

```
bash scripts/build_deb.sh
```

This produces `dist/klyrshift_<version>_all.deb`. The script builds the package tree by hand (no `dh_make` required) and runs `dpkg-deb --build`.

Test the package before publishing:

```
make smoke-test                     # installs the .deb in a clean debian:12
make zero-config                    # proves no manual step is needed
SMOKE_IMAGE=ubuntu:22.04 make smoke-test   # Python 3.10 path
```

`make smoke-test` installs the real package into a clean container and checks the entry point, the data files, the systemd unit path, the `uaccess` udev rule and `collect-logs`. `make zero-config` additionally asserts the packaging contract: the rule uses `TAG+="uaccess"` (never `GROUP="input"`), the unit is enabled globally without any manual step, no user is added to a group, and `apt remove --purge` leaves no file behind. Both catch packaging bugs that the pytest suite cannot, because pytest replaces the operating-system boundary with fakes. When you add a runtime dependency, add it to the `Depends:` line in `scripts/build_deb.sh` too — the two dependency lists (apt and pip) must stay in sync.

Two files exist in both `data/` and `src/klyrshift/data/`: `klyrshift.service`, `klyrshift.desktop` and `klyrshift.svg`. Keep them identical — a test enforces this. `data/` is the source for `install.sh`; `src/klyrshift/data/` is what the installed package ships.

Cutting a release (the full checklist, including the real-keyboard test that CI cannot run) is in RELEASING.md.

Pull requests

- Keep the change focused; describe the motivation in the PR body.
- Make sure `make check` is green and CI passes.
- Update the documentation when behaviour or configuration changes.

Roadmap

Where Klyrshift is, where it is going, and in what order. Every item is marked by real status, not intent. This is a plan, not a promise: a later stage does not begin until the one before it is working and tested.

Status legend: **done** (implemented and tested), **partial** (works, with documented limits), **planned** (not started), **blocked** (needs something outside the codebase).

Where the product is today

Klyrshift is a working Debian/GNOME layout switcher: it detects wrong-layout words, rewrites them, switches the layout, and can undo the change. The core is strong (1300+ tests, ~86 % coverage, 0 false positives and 0 corruption on the detector benchmark). The gaps are system integration on real hardware and packaging beyond a manual `.deb`; the first-run wizard and tour ship, but are only exercised by hand on a real desktop.

Stage 0 — Audit and foundations — done

Item	Status
Audit of detector, modes, undo, GUI, storage, CI	done
`docs/ARCHITECTURE.md` (existing, extended)	done
`docs/PLATFORM_SUPPORT.md`	done
`docs/OFFLINE_AND_PRIVACY.md`	done
`docs/UNDO_CONTRACT.md`	done

Item	Status
`docs/TEST_STRATEGY.md`	done
`docs/ROADMAP.md` (this file)	done

Stage 1 — Reliability and undo — partial

The main goal is that a correction is always safe to reverse.

Item	Status
`Ctrl+Z` lets the application undo the rewrite	done
The undo hotkey restores the last fix	done
A fix is dropped once the caret moves or new text is typed	done
Undo history bounded by time and depth	done
Atomic replacement on `uinput`	done
Atomic replacement on `wtype` / `xdotool` fallbacks	**blocked** — those tools have no atomic primitive
Confirmed on real editors, browser, terminal	**blocked** — needs a real desktop

Stage 2 — Offline language platform — done

Item	Status
Bundled dictionaries for five languages	done
Thematic dictionaries with offline fallback	done
User dictionary add / remove / import / export	done
A versioned, checksummed language-package format	done — `langpack.py`, `dict pack/verify/install-file`
Install / remove languages from the GUI	done
Package rollback on a failed update	done — `langpack.install_package` restores on failure
Integrity check before install	done — SHA-256 per member, verified before any write

Stage 3 — Learning and text intelligence — done

Item	Status
Remember confirmed corrections	done
Remember rejected corrections (`Ctrl+Z`)	done
Session vocabulary promotion	done
Manual add / remove of learned words	done — `klyrshift learn list/promote/forget` edits the running daemon's in-memory vocabulary

Item	Status
Separate dictionaries per domain (names, code, general)	done — `names` and `code` categories ship for ru+en and install offline
Local confidence model	done — `Confidence` maps the margin onto `[0, 1]` and records its evidence; `klyrshift explain` shows it
Suggestions instead of auto-change at low confidence	done — `hybrid` withholds and offers the fix through the tray entry, the accept hotkey and an optional notification

Stage 4 — UX / UI redesign — done

Item	Status
GTK4 + libadwaita preferences window	done
First-run tour	done
Status, dictionary, history, typo, help pages	done
System tray	done
Light / dark / system theme	done
First-run wizard	done — languages, mode and autostart, then the tour
Per-application profiles	done — `exceptions_apps` skips an app, `[app_layouts]` pins its layout, `[app_modes]` pins its correction mode (auto/manual/hybrid); set from the «Дополнительно» page or `klyrshift app-mode`
Full keyboard navigation and scaling audit	done — every page is reachable with Tab, `Ctrl+Page_Down/Up` cycles the visible tabs, `Ctrl+?` opens a keyboard reference built from the live config, and the layout was checked at 360–1200 px
Interface localisation	done — strings live in `klyrshift.i18n`, an English catalogue ships in `data/locales/en.json`, `--lang` switches the CLI and `Config.ui_locale` the GUI; a missing entry falls back to the Russian source

Stage 5 — Install and maintenance — partial

Item	Status
Self-contained `.deb`	done
Install / uninstall guide	done
`doctor` diagnostics	done
Config migration between versions	done
Package integrity check	done — the `.deb` ships an `md5sums` manifest; `klyrshift verify` and `dpkg --verify klyrshift` detect a corrupted file

Item	Status
Debian Policy compliance	done — the `.deb` ships a DEP-5 `copyright` (Debian Policy 12.5), `changelog.gz` (12.7) and a compressed man page (12.1); the base frequency lists are credited to CC-BY-SA-4.0 and the code stays MIT; `lintian` reports no error-level tags
Shell completion	done — bash and zsh completions generated from the argument parser (`data/completions/`), shipped by the `.deb` and `install.sh`
Flatpak / AppImage	planned — needs a portal input path

Stage 6 — Cross-platform — planned

The detector core is platform-independent and worth reusing. The input adapters are not. Nothing here is started; see PLATFORM_SUPPORT.md for what each platform would need.

Stage 7 — Quality and competitiveness — done

Item	Status
Reproducible detector benchmark	done
Recorded baseline and thresholds	done
Mixed / ambiguous / code corpus	done — `tests/test_ambiguous_words.py` sweeps cross-layout collisions
Latency and memory budget in CI	done — `scripts/measure_latency.py --budget-ms/--budget-mib` fails the `latency-budget` CI job

Order of work

Stages 2, 3, 4 and 7 are complete. What is left is blocked on something outside the code:

1. **Stage 1** — the atomic-replacement and real-editor items need a real desktop session (Wayland/GNOME) and the `wtype`/`xdotool` tools, which have no atomic primitive. Everything testable headlessly is done.
2. **Stage 5** — Flatpak/AppImage needs a portal input path, which does not exist yet for this kind of synthetic input.
3. **Stage 6** — cross-platform needs the input adapters rewritten per platform (see PLATFORM_SUPPORT.md); the detector core is already platform-independent and would be reused as is.

Implemented features

What Klyrshift ships today, grouped by area. Every entry is implemented in `src/klyrshift/` and covered by the test suite unless the row says otherwise; the "Where" column names the module or command a reader can open to confirm it. This is a status inventory, not a plan -- what is not built yet lives in ROADMAP.md, and the design behind each area is in ARCHITECTURE.md.

The version this list describes is the one in `pyproject.toml` (0.2.10 at the time of writing). The suite that backs it is `pytest` (about 1900 tests, ≥ 80 % coverage) plus the deterministic QA corpus.

Core correction

Feature	Where	Notes
Wrong-layout detection	`detector.py`	Combines character-frequency, n-gram and context signals.
Rewrite of already-typed text	`daemon.py`, `injector.py`	`ghbdtn` → `привет`; the daemon tracks the word as scancodes, not characters.
Character-level layout conversion	`converter.py`	Same physical keys, other layout.
Three correction modes	`config.py` (`mode`)	`auto`, `manual`, `hybrid`; `manual` is the master gate for every automatic pass.
Confidence model and withholding	`detector.py`, `daemon.py`	`hybrid` withholds a low-margin fix and offers it via the tray, the accept hotkey or a notification.
Word-boundary triggers	`config.py`	Space and Enter on by default; Tab and punctuation opt-in.
Configurable hotkeys	`config.py`, `daemon.py`	Fix (`SHIFT+SHIFT`), undo (`SHIFT+BACKSPACE`), toggle layout, T9, punctuation, accept suggestion, reload.
Undo of the last fix	`daemon.py`	A bounded, caret-aware history; see UNDO_CONTRACT.md.
False-positive guards	`detector.py`	Plausibility, structural-boundary and identifier guards; a token with `. @ / \ : _ -` is never rewritten.
Mixed-script protection	`detector.py`	A token mixing Latin and Cyrillic is never converted as a unit.
Short-word fix hotkey	`detector.py`	Forces a below-`min_word_length` word on the explicit hotkey.
Layout switching	`switcher.py`	`g3kb-switch` (Wayland) and `setxkbmap` (X11).
Text injection backends	`injector.py`	`uinput` (atomic), `wtype`, `xdotool`.

Learning and text intelligence

Feature	Where	Notes
Session learning	`learning.py`	Remembers accepted, rejected and kept words; always on.
Rejection suppression	`learning.py`, `daemon.py`	`Ctrl+Z` (or the undo hotkey) suppresses a word for the session; an explicit hotkey still wins.
Vocabulary promotion	`learning.py`	A word promoted after `learning_promote_threshold` keeps is never rewritten.
Privacy-preserving profile	`learning.py`	Only counters reach disk; no typed word is ever persisted.
Typo correction (T9)	`typo.py`	Opt-in; replaces a word only within `max_distance` of exactly one vocabulary word.

Feature	Where	Notes
Punctuation cleanup	`punctuation.py`	Opt-in; dashes, ellipsis, smart quotes, spacing.
Text expansion (snippets)	`text_expander.py`	`!!email` → longer text at a word boundary.
Selection fix	`selection_fix.py`	Converts an already-selected piece of text on a hotkey.
Context analysis	`detector.py`	Neighbouring words may only add to a candidate, never suppress one.

Dictionaries and languages

Feature	Where	Notes
Bundled base dictionaries	`dictionaries/`, `dictionary.py`	Five languages (`en`, `ru`, `uk`, `de`, `fr`); `en` + `ru` loaded by default.
Thematic (professional) dictionaries	`thematic.py`	14 categories (IT, code, names, medicine, legal, finance, engineering, science, business, electronics, media, education, gaming, sport), ru+en.
User dictionary	`dictionary.py`	Consulted in two directions; edited explicitly, never auto-written from typed text.
Extended dictionary download	`dictionary.py`	Release asset → branch mirror → upstream → bundled source.
Versioned language packages	`langpack.py`	`manifest.json` + per-member SHA-256, verified before any write, with rollback on failure.
Language install/ remove from the GUI	`langstore.py`	Backs the Dictionary tab.
Technical-token protection	`data/ stop_words.txt`	Short vowel-less tokens (`http`, `pdf`, `jwt`, ...) stay in the typed layout.

Interface (GTK4 / libadwaita)

Feature	Where	Notes
Preferences window	`gui/window.py`, `gui/*_page.py`	Home, Settings, Advanced, Dictionary, Typo, History, Help, About pages.
First-run wizard and guided tour	`gui/widgets/first_run_wizard.py`, `onboarding.py`, `gui/widgets/ tour_overlay.py`	Languages, mode and autostart, then the tour.
System tray	`tray.py`	Optional AppIndicator icon.
Notifications	`daemon.py`	Fixed label, never the corrected text.
Light / dark / system theme	`gui/theme.py`	Window appearance only.
Interface localisation	`i18n.py`, `data/locales/`	Russian, English, German, French, Ukrainian; a missing entry falls back to the Russian source.

Feature	Where	Notes
Keyboard navigation and shortcuts dialog	`gui/widgets/shortcuts_dialog.py`	Tab reachable; `Ctrl+Page_Down/Up`, `Ctrl+?`.
Per-application profiles	`app_layouts.py`, `config.py`	`exceptions_apps` skips an app; `[app_layouts]` pins its layout; `[app_modes]` pins its mode.
Quiet hours	`config.py`	Automatic correction silent in a time window; the manual hotkey still works.
Password-field guard	`app_focus.py`	Skips correction in password fields.

Command line and operations

Feature	Where	Notes
Daemon control	`cli.py`, `daemon_control.py`	`start`, `stop`, `kill`, `restart`, `status`, `--dry-run`.
Configuration	`cli.py`	`mode`, `app-mode`, `config show/edit/reset/path/migrate`, `hotkeys`, `undo`.
Correction tools	`cli.py`	`fix`, `explain`, `simulate`.
Learning control	`cli.py`	`learn list/list-rejected/promote/forget`, `--on/--off/--reset`.
Dictionary management	`cli.py`	`dict list/add/remove/download/import-file/categories/install/remove-category/list-installed`, `pack/verify/install-file/uninstall-file`.
Backup and restore	`backup.py`, `cli.py`	`export` / `import` of config, user dictionary and snippets.
Self-diagnosis	`doctor.py`	`doctor` prints a □/□ table with hints.
Integrity check	`integrity.py`	`verify` re-checks the `.deb` `md5sums` manifest and honours dpkg `path-exclude`.
Logging	`logs.py`, `collect_logs.py`	`logs -n/-f/--level`, `collect-logs` diagnostic archive.
Hot config reload	`daemon.py`	`SIGHUP` rebuilds every derived cache atomically.
Autostart	`daemon_control.py`, `install.sh`	Systemd user unit plus the XDG entry, both honoured.
Update check	`update_check.py`	Opt-in daily GitHub releases check, off the input thread.

Testing and quality

Feature	Where	Notes
Unit and integration suite	`tests/`	Fakes the OS boundary; about 1900 tests.

Feature	Where	Notes
Headless simulator	`simulate.py`	Drives the real daemon with a faked keyboard and renders the screen.
Autonomous QA engine	`qa/`	Curated + seeded scenarios, structural invariants, regression store; deterministic and offline.
Detector benchmark	`scripts/measure_detector.py`	Recall, false positives, corruption and latency on fixed slices.
Latency/memory budget	`scripts/measure_latency.py`	Fails a CI job past the budget.
Package tests	`scripts/*_test.sh`	Smoke, zero-config, self-sufficiency and patch-upgrade on real containers.
Reproducible `.deb`	`scripts/build_deb.sh`	Pinned compressor, clamped mtimes, date from the changelog.
Debian Policy compliance	`data/copyright`, `scripts/build_deb.sh`	DEP-5 `copyright`, Debian changelog, man page, shell completion.
Landing page	`site/`	Astro site with a live detector demo kept in parity by a test.

Deliberate limits

These are documented, not defects; see ROADMAP.md and PLATFORM_SUPPORT.md.

- Atomic replacement is only possible on `uinput`; `wtype` and `xdotool` have no atomic primitive.
- The real `evdev` → `uinput` path and a live compositor are not exercised by CI; they need a real GNOME desktop.
- `g3kb-switch` is a `Recommends`, not a `Depends` (it is not in Debian), so under pure Wayland the layout may not switch without it.
- Flatpak / AppImage and cross-platform adapters are planned, not built.

Отчёт о выполненной работе

Документ описывает, что уже сделано в проекте Klyrshift, и какие результаты достигнуты. Это снимок состояния на версию **0.2.10** (тег `v0.2.10`, дата выпуска 2026-11-10); текущее состояние — это инвентаризация результатов, а не план. Что ещё не сделано, описано в ROADMAP.md, а список реализованных возможностей по областям — в FEATURES.md.

Краткое резюме

Klyrshift — это демон автоматического переключения раскладки клавиатуры для Debian GNOME (X11 и Wayland). Он читает нажатия клавиш через `evdev`, накапливает набранное слово и на границе слова исправляет текст, набранный не в той раскладке (`ghbdtn` → `привет`), переключая раскладку.

К версии 0.2.10 проект достиг следующих результатов:

Показатель	Значение
Версия / тег	0.2.10 / `v0.2.10`
История	333 коммита, первый — 2026-10-03
Модули пакета (`src/klyrshift/`)	75 файлов `.py`
Тестовые модули (`tests/`)	89 файлов, 1887 тестов собрано
Прогон тестов	1776 passed, 111 skipped
Покрытие	85 % (порог CI — 80 %, локальный — 85 %)
Качество кода	`ruff`, `black --check`, `mypy --strict` — чисто
Автономный QA	96/96 сценариев, 0 новых багов
Языки детектора	5 (`en`, `ru`, `uk`, `de`, `fr`)
Тематические словари	14 категорий (ru+en)
Документация	19 руководств в `docs/` + единый офлайн-справочник (Markdown и PDF)
Упаковка	воспроизводимый `.deb`, Debian Policy 12.5 (DEP-5 `copyright`)

Завершённые задачи

Ниже перечислены завершённые направления работы. Каждая строка называет модуль, команду или документ, по которому результат можно проверить.

Ядро исправления раскладки

- **Определение неверной раскладки.** Детектор объединяет частотные, n-граммные и контекстные сигналы (`detector.py`).
- **Перезапись уже набранного текста.** `ghbdtn` → `привет`; демон отслеживает слово в скан-кодах, а не в символах (`daemon.py`, `injector.py`).
- **Три режима работы** (`auto`, `manual`, `hybrid`), где `manual` — главный «шлагбаум» для всех автоматических проходов (`config.py`).
- **Модель уверенности** с удержанием исправления в `hybrid` и предложением через трей, горячую клавишу или уведомление (`detector.py`, `daemon.py`).
- **Триггеры границы слова** (Space и Enter включены по умолчанию; Tab и пунктуация — опционально), `config.py`.
- **Настраиваемые горячие клавиши:** исправление (`SHIFT+SHIFT`), отмена (`SHIFT+BACKSPACE`), переключение раскладки, T9, пунктуация, принятие предложения, перезагрузка (`config.py`, `daemon.py`).
- **Отмена последнего исправления** с ограниченной и следящей за курсором историей (UNDO_CONTRACT.md).
- **Защита от ложных срабатываний:** проверки правдоподобия, структурных границ и идентификаторов; токен с `. @ / \ : _ -` никогда не переписывается (`detector.py`).
- **Защита смешанного письма:** токен, смешивающий латиницу и кириллицу, не конвертируется целиком (`detector.py`).
- **Бэкенды ввода текста:** `uinput` (атомарно), `wtype`, `xdotool` (`injector.py`).
- **Переключение раскладки:** `g3kb-switch` (Wayland) и `setxkbmap` (X11) (`switcher.py`).

Обучение и интеллект текста

- **Обучение в рамках сессии** с запоминанием принятых, отвергнутых и оставленных слов (`learning.py`).
- **Подавление отвергнутых слов:** `Ctrl+Z` (или горячая клавиша отмены) подавляет слово на сессию; явная горячая клавиша всё равно срабатывает.
- **Повышение слова в словарь** после порога удержаний — такое слово больше не переписывается.
- **Приватный профиль:** на диск попадают только счётчики, ни одно набранное слово не сохраняется.
- **Исправление опечаток (T9)** и **чистка пунктуации** — опционально (`typo.py`, `punctuation.py`).
- **Текстовые сниппеты** (`!!email` → длинный текст) и **исправление выделения** (`text_expander.py`, `selection_fix.py`).
- **Контекстный анализ:** соседние слова могут только добавлять кандидата, но не подавлять его (`detector.py`).

Словари и языки

- **Встроенные базовые словари** для пяти языков (`en`, `ru`, `uk`, `de`, `fr`); по умолчанию загружаются `en` и `ru` (`dictionaries/`, `dictionary.py`).
- **14 тематических (профессиональных) словарей** на ru+en: IT, код, имена, медицина, юриспруденция, финансы, инженерия, наука, бизнес, электроника, медиа, образование, игры, спорт (`thematic.py`).
- **Пользовательский словарь**, учитываемый в двух направлениях; редактируется явно и никогда не дописывается из набранного текста (`dictionary.py`).
- **Версионные языковые пакеты** с `manifest.json` и SHA-256 по членам, с проверкой перед записью и откатом при ошибке (`langpack.py`).
- **Защита технических токенов:** короткие токены без гласных (`http`, `pdf`, `jwt`, …) остаются в набранной раскладке (`data/stop_words.txt`).

Интерфейс (GTK4 / libadwaita)

- **Окно настроек** со страницами Home, Settings, Advanced, Dictionary, Typo, History, Help, About (`gui/`).
- **Мастер первого запуска и обучающий тур** (`gui/widgets/first_run_wizard.py`, `onboarding.py`, `gui/widgets/tour_overlay.py`).
- **Системный трей**, уведомления (фиксированный текст, без исправленных слов).
- **Светлая / тёмная / системная тема** (`gui/theme.py`).
- **Локализация интерфейса:** русский, английский, немецкий, французский, украинский; отсутствующая запись откатывается к русскому источнику (`i18n.py`, `data/locales/`).
- **Профили по приложениям** (`exceptions_apps`, `[app_layouts]`, `[app_modes]`), **тихие часы** и **защита поля пароля** (`app_layouts.py`, `config.py`, `app_focus.py`).

Командная строка и эксплуатация

- **Управление демоном:** `start`, `stop`, `kill`, `restart`, `status` (`cli.py`, `daemon_control.py`).

- **Настройка:** `mode`, `app-mode`, `config show/edit/reset/path/migrate`, `hotkeys`, `undo`.
- **Инструменты исправления:** `fix`, `explain`, `simulate`.
- **Управление словарями:** `dict list/add/remove/download/...`, `pack/verify/...`.
- **Резервное копирование** (`backup.py`), **самодиагностика** (`doctor.py`), **проверка целостности** `verify` (`integrity.py`).
- **Логирование** (`logs.py`, `collect_logs.py`), **горячая перезагрузка** конфигурации по `SIGHUP`, **автозапуск** (systemd user unit + XDG), опциональная проверка обновлений (`update_check.py`).
- **Сгенерированные man-страница и автодополнение** bash/zsh (`data/klyrshift.1`, `data/completions/`).

Тестирование и качество

- **Модульный и интеграционный набор** `tests/` — около 1900 тестов; граница ОС подменяется фейками.
- **Безголовый симулятор** `klyrshift simulate` — ведёт реальный демон с поддельной клавиатурой и отрисовывает экран (`simulate.py`).
- **Автономный QA-движок** `klyrshift qa` — курируемые и seeded-сценарии, структурные инварианты, хранилище регрессий; детерминирован и работает офлайн (`qa/`).
- **Бенчмарк детектора** (`scripts/measure_detector.py`), **бюджет задержки/памяти** в CI (`scripts/measure_latency.py`).
- **Тесты упаковки** — smoke, zero-config, self-sufficiency и patch-upgrade в чистых контейнерах (`scripts/*_test.sh`).
- **Воспроизводимый** `.deb` — зафиксированный компрессор, выровненные mtime, дата из `CHANGELOG.md` (`scripts/build_deb.sh`).
- **Лендинг** `site/` (Astro) с живой демонстрацией детектора, удерживаемой в паритете тестом.

Достигнутые результаты

Проверки выполнены на текущем дереве (версия 0.2.10) и подтверждены выводом команд:

Проверка	Команда	Результат
Тесты	`pytest tests/ -q --cov=klyrshift`	**1776 passed, 111 skipped**, покрытие **85 %**
Сбор тестов	`pytest tests/ --collect-only -q`	1887 тестов собрано
Линтер	`ruff check .`	`All checks passed!`
Формат	`black --check .`	177 файлов без изменений
Типы	`mypy --strict .`	`Success: no issues found in 173 source files`
QA-движок	`klyrshift qa run --generate 60 --seed 12345`	96 проверено, 96 успешно, 0 провалов

Часть тестов пропущена (`skipped`) ожидаемо: GUI-тесты требуют дисплея (`xvfb-run`), часть проверок сайта и рендер скриншотов включаются отдельно, а `tesseract` в этом окружении отсутствует. Это документированное поведение, а не сбой (см. TESTING.md).

Что было исправлено в цикле 0.2.x

Версия 0.2.10 — итог серии исправлений, каждое из которых закрыто тестом:

- **Атомарная перезагрузка конфигурации.** `KlyrshiftDaemon.apply_config` пересобирает все производные кэши; при ошибке состояние откатывается к предыдущему известному рабочему (`AUDIT.md`, дефект D1).
- **Честная ошибка конфигурации.** Плохое значение в шаге `config` симулятора — это `ScenarioError`, а не трейсбек (`simulate.py`).
- **Границы слова больше не «съедают» символ.** Enter, Tab и пунктуация перепечатываются вместе со словом; живая горячая клавиша сбрасывает запомненную границу (`daemon.py`).
- **`manual` — главный шлагбаум.** В ручном режиме ни один автоматический проход (включая T9 и пунктуацию) не срабатывает без команды.
- **`Ctrl+Z` надёжно подавляет именно то слово**, которое отменили.
- **Приватность.** Строка лога об удержанной конверсии больше не содержит набранного слова; на диск не попадает ни одно слово.
- **Воспроизводимый `.deb`** между хостами и **проверка целостности `verify`**, учитывающая политику `path-exclude` dpkg.
- **Устойчивость демона.** Завершение не падает из-за управляющего сокета в селекторе; нет аварий при отсутствии шины D-Bus.

Известные ограничения

Это документированные ограничения, а не дефекты (см. ROADMAP.md и PLATFORM_SUPPORT.md):

- Атомарная замена возможна только на `uinput`; у `wtype` и `xdotool` такого примитива нет.
- Реальный путь `evdev` → `uinput` и живой композитор не проверяются в CI — нужен настоящий рабочий стол GNOME (`scripts/acceptance_check.sh`).
- `g3kb-switch` — это `Recommends`, а не `Depends` (его нет в Debian), поэтому под чистым Wayland раскладка может не переключаться.
- Flatpak / AppImage и кросс-платформенные адаптеры запланированы, но не сделаны.

Как воспроизвести проверки

```
bash scripts/dev_setup.sh     # venv + dev-зависимости
make check                    # ruff + black + mypy --strict + pytest --cov
make qa                       # автономный QA-движок (офлайн)
make build-deb && make smoke-test   # .deb в чистом контейнере
```

Единый офлайн-справочник собирается командой `make docs-pdf` из источников в `docs/`; этот отчёт входит в сборку.

Проверка существующей реализации

Этот документ фиксирует результат проверки того, что уже готово в Klyrshift, выполненной на дереве `0.2.10` (коммит-основание `b486945`). Он отвечает на вопрос «что уже готово?» **измерением**, а не пересказом: каждая строка ниже — команда, которую можно повторить, и её фактический вывод. Отчёт о выполненной работе — в WORK_REPORT.md, инвентаризация возможностей — в FEATURES.md, известные ограничения — в ROADMAP.md.

Итог: **заявленные возможности подтверждены**, все проверяемые в этом окружении контуры (тесты, линтеры, типы, QA-движок, симулятор, сборка `.deb`, лендинг) зелёные. Найдены расхождения в документации (устаревшие числа) и один известный, но не исправленный дефект QA-движка; и то и другое перечислено ниже.

1. Что проверено и с каким результатом

Окружение: Debian 13 (trixie), Python 3.13.15, `git` HEAD `b486945`.

Проверка	Команда	Результат
Тесты (headless)	`pytest tests/ -q --cov=klyrshift`	**1776 passed, 111 skipped**, покрытие **85 %**
Тесты (GUI, максимум)	`PYTHONPATH=/usr/lib/python3/dist-packages KLYRSHIFT_RENDER_TEST=1 xvfb-run -a pytest tests/ -q --cov=klyrshift --cov-config=coverage-gui.rc --cov-fail-under=85`	**1887 passed, 0 skipped**, покрытие **85.80 %**
Сбор тестов	`pytest tests/ --collect-only -q`	1887 тестов собрано
Линтер	`ruff check .`	`All checks passed!`
Формат	`black --check .`	177 файлов без изменений
Порядок импортов	`isort --check-only .`	чисто
Типы	`mypy --strict .`	`Success: no issues found in 173 source files`
Автономный QA	`klyrshift qa run --generate 60 --seed 12345 --no-record`	**96 проверено, 96 успешно, 0 ошибок стенда**, exit 0
Бенчмарк детектора	`python scripts/measure_detector.py`	ru→us recall 92.8 %, en→ru 95.1 %, **0 ложных срабатываний, 0 порчи**
Бюджет задержки/памяти	`python scripts/measure_latency.py --limit 3000 --repeats 2`	0.0068 / 0.0067 мс/слово, 4.3 MiB
Консистентность версии	`python scripts/release_consistency.py --source`	`ok`

Проверка	Команда	Результат
Воспроизводимость `.d`	`bash scripts/build_deb.sh` дважды + `cmp`	**байт-идентичен** (`6c0a55ea…`)
Манифест целостности	`dpkg-deb -e` + `md5sum -c`	покрывает ровно 108 файлов, все суммы верны
Лендинг	`cd site && npm run build`	собрано 2 страницы
Интерфейс (i18n)	`klyrshift --lang ru/en/de/fr/uk status`	все пять локалей отвечают

Уровень 1b: безголовый симулятор

Флагманский сценарий и режим «ручной» воспроизводятся как задокументировано:

```
$ klyrshift simulate 'type ghbdtn, space'
Экран: привет
Режим: auto · раскладка: ru
  замена −7 +'привет ' @ru

$ klyrshift explain --text ghbdtn
Текущая раскладка: us
Решение: исправить в ru -> привет
Оценка: уверенность 100% (перевод даёт известные слова)
```

Командная строка

Проверены `status`, `mode`, `hotkeys`, `config show`, `dict list/add/remove/categories`, `export`/`import`, `doctor`, `verify`, `logs`, `simulate`, `explain`, `fix`, `qa`. Все отвечают; `doctor` корректно возвращает ненулевой код, когда находит проблемы (в этом окружении — нет udev-правила и устройств ввода, что ожидаемо без рабочего стола).

2. Расхождения

2.1 Дефект QA-движка: список/словарь из двух и более элементов (не исправлен)

Задокументирован в AUTOPROGRESS_BREAKAGE.md и **воспроизводится на текущем дереве**. `qa/executor.build_script` склеивает шаги через `", "`, а `simulate.run_scenario` делит сценарий по `","`; запятые внутри `[a, b]`/`{k: v}` неотличимы от разделителя шагов.

```
$ klyrshift simulate 'config exceptions_apps=[kitty, code], type ghbdtn, space'
Ошибка сценария: unknown key 'code]'

$ klyrshift qa run --scenarios two-apps.json --no-record   # сценарий с двумя app
QA: проверено 37, успешно 36, провалов 0, ошибок стенда 1
$ echo $?
1
```

Одноэлементный список/словарь проходит (в курируемом корпусе ровно один такой сценарий, поэтому прогон по умолчанию зелёный). То же остриё у `type`: `klyrshift simulate 'type hello, world'` → `unknown key 'world'`.

Статус: известный дефект, не исправлен; исправление предложено в AUTOPROGRESS_BREAKAGE.md (разбор сценария с учётом вложенности скобок) и здесь не применялось, потому что шаг плана — проверка, а не исправление.

2.2 Мёртвая ручка `RunConfig.repeats`

`qa/runner.py` объявляет `repeats: int = 2` и документирует его как «сколько раз падение должно воспроизвестись», но **ничто его не читает**: `runner.run` → `analyse` → `reproduces(scenario)` с зашитым `runs=2`. Ручка, которая молча ничего не делает, — ловушка для следующего читателя (тот же вывод в AUTOPROGRESS_BREAKAGE.md, п. 4).

2.3 Устаревшие числа в документации

Документы-снимки описывают состояние на момент написания и с тех пор отстали. Числа ниже — факт на HEAD `b486945`:

Где	Написано	Факт	Комментарий
`docs/WORK_REPORT.md`	«19 руководств в `docs/`»	**20**	сразу после отчёта добавлен `AUTOPROGRESS_BREAKAGE.md`
`docs/WORK_REPORT.md`	«333 коммита»	**335**	снимок на момент написания
`docs/PROJECT_STRUCTURE.md`	«88 modules» (о тестах)	**89**	`test_*.py` в `tests/`
`docs/ROADMAP.md`	«1300+ tests, ~86 % coverage»	**1887** тестов, **85.80 %**	«1300+» формально верно, но сильно устарело

Эти строки — прозаические снимки, их не пиннит ни один тест (в отличие от версий, структуры и ссылок), поэтому дрейф молчаливый. Отдельно: строка `docs/WORK_REPORT.md` «Прогон тестов — 1776 passed, 111 skipped» **верна** для headless-прогона, но не показывает более сильный GUI-прогон (1887 passed, 0 skipped), который CI выполняет в задании `gui`.

3. Чего в этом окружении проверить нельзя

Это не дефекты, а границы стенда (совпадают с PLATFORM_SUPPORT.md и TESTING.md):

- **Реальный путь `evdev` → `uinput` и живой композитор.** Нет графической сессии и устройств ввода; проверяется только логика через симулятор.
- **`g3kb-switch`** (переключение раскладки под Wayland) — нет в Debian и не установлен; остаётся `Recommends`.
- **Пакетные тесты в контейнере** (`scripts/smoke_test.sh`, `deb_selfsufficiency_test.sh`, `upgrade_test.sh`) — нужен Docker, который в этом окружении недоступен. Вместо них проверены сборка `.deb`, воспроизводимость и полнота манифеста `md5sums` напрямую.
- **`wtype` / `xdotool`** — не установлены; атомарная замена проверяема только на `uinput` (документированное ограничение).

4. Отсутствующие возможности (по ROADMAP)

Ничего из перечисленного не заявлено как готовое; это план, а не регрессия:

- **Flatpak / AppImage** — planned (нужен путь ввода через портал).
- **Кросс-платформенные адаптеры** (Stage 6) — planned.
- **Атомарная замена на `wtype` / `xdotool`** — blocked (у этих утилит нет атомарного примитива).
- **Проверка на реальном рабочем столе** (Stage 1, редакторы/браузер/терминал) — blocked (нужна настоящая сессия GNOME).

5. Как воспроизвести

```
bash scripts/dev_setup.sh                                        # venv + dev-зависимости
make check                                                       # ruff + black + mypy +
pytest --cov
make qa                                                          # автономный QA (офлайн)
make build-deb && cmp dist/klyrshift_*.deb <второй сборкой> # воспроизводимость

## максимальный прогон (нужен дисплей и python3-gi):
sudo apt install -y xvfb python3-gi python3-gi-cairo gir1.2-gtk-4.0 gir1.2-adw-1
PYTHONPATH=/usr/lib/python3/dist-packages KLYRSHIFT_RENDER_TEST=1 \
  xvfb-run -a pytest tests/ -q --cov=klyrshift --cov-config=coverage-gui.rc --cov-fail-
under=85
```

6. Вывод

Существующая реализация **работает как заявлено** во всём, что проверяемо без рабочего стола: 1887 тестов проходят, покрытие 85.80 %, линтеры и типы чисты, QA-движок 96/96, детектор без ложных срабатываний и порчи, `.deb` воспроизводим и целостен, лендинг собирается, пять языков интерфейса живы. Расхождения — устаревшие числа в документах-снимках и один известный неисправленный дефект QA-движка на многоэлементных списках/словарях в `config`-шаге; отсутствующие возможности перечислены в ROADMAP.md и не заявлены как готовые.

Changelog

All notable changes to this project are documented in this file.

The format is based on Keep a Changelog, and this project adheres to Semantic Versioning.

[Unreleased]

[0.2.10] - 2026-11-10

Added

- **More technical tokens are protected, and the canary is independent.** A sweep of real extensions, protocols and config keys found tokens the detector still rewrote into

Cyrillic (`cjs`, `jsx`, `npz`, `pfx`, `tgz` from the first pass; `cdf`, `csh`, `etx`, `hdf`, `jks`, `jpe`, `pbm`, `pnm`, `rtx`, `vcf`, `vtt`, `xbm`, `xlb` -- thirteen more -- once the canary was widened); they are not words of any shipped language and join the technical-token block of `data/stop_words.txt`. The canary is deliberately **wider than that list**: it is generated from the stdlib `mimetypes` extension table plus a static list of source/protocol/build/config tokens (**242 tokens**), minus a documented allowlist of nine extensions that are real words (`so`, `key`, `log`, `mid`, `img`, `tmp`, `dat`, `bin`, `iso`) -- leaving a real word alone is correct, so it is not a leak. Two tests assert the widening, so the canary cannot silently collapse back into a restatement of the list it guards: a token that leaks is named by the test, and removing a token from `stop_words.txt` makes the canary fail.

- **A patch-upgrade test.** `scripts/patch_upgrade_test.sh` (and `make patch-upgrade-test`) installs the previous patch release over the current one in a clean container and checks that the user's config survives unchanged, the daemon unit stays enabled, `klyrshift verify` passes and the CLI works. The smoke test only proves a clean install; a patch release is normally an upgrade, so this closes the gap.

- **`AUDIT.md`.** A full audit of the daemon performed with the project's own tooling: the baseline numbers, the four defects found (with the measurement that proved each), the fixes and their tests, the before/after figures, and an explicit out-of-scope list.

- **Short technical tokens are no longer rewritten into Cyrillic.** A short, vowel-less Latin token (`http`, `www`, `xml`, `pdf`, `jpg`, `csv`, `tls`, `jwt`, `cfg`, `txt`, `vpn`, `xlsx`, `yml`, `dns`, `svn`, `fps`, `llm`, ...) scored as noise in the English bigram model, so the Russian model won by a small margin and the detector rewrote the token into letters the user never typed (`http` -> `реез`). These are not words of any shipped language, so they now join the bundled stop-word list (`data/stop_words.txt`); the automatic pass leaves them in the layout they were typed in. The flagship fixes (`ghbdtn` -> `привет`, `rfr` -> `как`) are unaffected, and the regression is pinned by `tests/test_false_positives.py` and a QA scenario.

- **The simulator's `config` step refreshes every cache it invalidates.** The daemon caches a lot of configuration at construction time (the parsed hotkeys, the exception set, the detector's tuning, the per-app layout manager), and `klyrshift simulate 'config ...'` used to write the field and leave the caches stale. An override such as `config exceptions_apps=[kitty]` was therefore silently ignored. The reload logic is now shared (`KlyrshiftDaemon.apply_config`), and both `reload_config` and the simulator call it, so an override takes effect immediately. Pinned by `tests/test_simulate.py` and a QA scenario.

- **An autonomous QA engine (`klyrshift qa`).** A testing layer on top of the pytest suite: it drives the real daemon through `klyrshift.simulate`, checks independent invariants and records confirmed bugs in a regression store. A scenario is JSON data (a bundled corpus plus a seeded deterministic generator), never code — only the simulator's fixed step DSL is ever run, so no step an LLM proposes can execute. `klyrshift qa run` is offline and deterministic (no API key), and a new CI job runs it on every push; `--report` writes a JSON and a Markdown report that show measured numbers only. See docs/QA_ENGINE.md.

- **A comparison section on the landing page.** The README carried a feature matrix against the other Linux layout switchers, but the page a visitor lands on did not. `Comparison.astro` renders the same matrix (automatic correction, Wayland, terminals, layout switching, the stop-word password guard, tray, Russian interface) with each competitor linked to its repository, and the header nav gains a comparison anchor. The

rows come from each project's own public documentation rather than from a marketing claim.

- **A correction mode per application.** The global mode is one setting, but the right mode depends on what the user is doing: an editor with code should not be rewritten automatically while a chat window benefits from it. The new `[app_modes]` table pins a mode (auto/manual/hybrid) to an application, and an application absent from the table keeps the global mode. The override is resolved at both policy gates, so it also decides whether a low-confidence hybrid fix is applied. Set it from «Дополнительно» → «Режим по приложениям» (the mode is chosen from a combo, so an invalid value cannot be entered) or with `klyrshift app-mode`. The focused application is now probed once per word instead of once per gate, which keeps the extra check off the hot path.

- **A keyboard reference, and keyboard tab navigation.** Every page was already reachable with Tab, but nothing listed the window shortcuts and there was no way to move between tabs without the mouse. `Ctrl+Page_Down/Up` now cycle the visible tabs (hidden pages are skipped, so the focus never lands somewhere the tab bar does not show) and `Ctrl+?` opens a reference that pairs the window accelerators with the daemon hotkeys read from the live config. The menu gains the same entry, so the dialog is reachable without knowing the chord.

- **The interface can switch language.** Every user-facing string now goes through `klyrshift.i18n`: the Russian text stays in the source as the key, and an English catalogue lives in `data/locales/en.json`. `klyrshift --lang en <command>` localises the CLI, and the settings page gains an "Interface language" combo (`auto`, Russian, English) that is stored as `Config.ui_locale` and applied at startup without a code change. A missing translation falls back to the Russian source, so a gap is a cosmetic defect, never a crash; `tests/test_i18n.py` fails the build when a string has no entry, when a catalogue entry is orphaned, or when the plural helper mis-selects a form.

- **Three more interface languages: German, French and Ukrainian.** The interface shipped in Russian and English; `de`, `fr` and `uk` catalogues now sit beside them and appear in the same "Interface language" combo, in their own script (`Deutsch`, `Français`, `Українська`). The set of languages is derived from one registry (`i18n.SUPPORTED_LOCALES`), so the config validator and the GUI selector cannot disagree with the catalogues that exist. The catalogues cover the settings window, the CLI and the first-run tour and context help; only the tray and some daemon messages are untranslated yet and fall back to English rather than to Russian, so a partial catalogue reads coherently instead of half in Cyrillic. Ukrainian joins Russian in using three plural forms; French treats zero as singular ("0 mot"), unlike German and English. `--lang de|fr|uk` localises the CLI on the same catalogues.

- **One offline handbook, in Markdown and PDF.** The documentation lived in a dozen files, which is right for a repository and awkward for anyone reading offline. `make docs-pdf` (or `scripts/build_docs.py`) concatenates `README.md`, the `docs/*.md` guides and `CHANGELOG.md` into a single document and renders it to `docs/klyrshift-handbook.pdf`, with a table of contents, page numbers and rewritten internal links. The build is offline and deterministic — no template fetch — so it can run in CI. The concatenated Markdown and the rendered PDF are both committed, so the handbook can be read or searched without a toolchain; the build regenerates them from the sources on demand. A change belongs in

the source document, never in either output. The `docs` extra (`weasyprint`, `markdown`) is development-only and ships in no package.

- **Domain dictionaries for names and code.** Two new thematic categories, `names` and `code`, ship for Russian and English. `klyrshift dict install names` (or `code`) widens the detector's vocabulary so a personal name or a programming keyword is recognised instead of being "corrected", exactly like the existing professional categories. The lists are curated in-repo and installable offline.

- **A calibrated confidence model, and `klyrshift explain`.** The detector has always returned a raw plausibility margin, which is unbounded and grows with the word count — fine as an internal gate, useless as a number a user can reason about. `LanguageDetector.analyze()` now returns a `Confidence` that maps the same decision onto `[0, 1]` (a logistic over the margin, the share of the target that is known vocabulary, and a penalty for very short buffers) and records how it was reached: a taught word, a conversion into known vocabulary, a mixed-script buffer, or the language model. The new read-only `klyrshift explain --text <word>` prints the verdict and the reason, and `klyrshift doctor` gains a "Модель детектора" row showing the loaded languages and a sanity confidence. Nothing about what the daemon corrects changes.
- **The detector reports which languages it actually loaded.** A requested language with no corpus and no extended dictionary is skipped; the new `LanguageDetector.loaded_languages` property exposes the real set so `doctor` cannot claim a model that is not there.
- **The detector's latency and memory are now budgeted in CI.** The `latency-budget` job runs `scripts/measure_latency.py` with `--budget-ms 0.5` and `--budget-mib 64`; the script exits non-zero when the slowest direction or the peak traced allocation crosses the line. The thresholds are loose on purpose — a shared runner is noisy, and the regression worth catching is an order of magnitude (a corpus reloaded per word, an accidental $O(n^2)$), not a few percent. `--budget-ms` / `--budget-mib` also make the script usable as a local check.
- **Hybrid mode now offers the fix it will not apply.** Hybrid mode has always withheld a low-confidence conversion instead of rewriting the word, but the offer was only reachable through the fix hotkey. The withheld suggestion is now kept explicitly and can be applied deliberately: the tray gains an "Apply suggestion" entry (disabled until an offer is waiting), a new `hotkey_accept_suggestion` binding does the same from the keyboard, and `suggest_on_withhold` (off by default) shows a notification when an offer appears. The offer is dropped as soon as the next word is flushed, so a stale suggestion is never applied to text the user has moved past. The typed text stays in memory: the offer is never logged and the notification body is a fixed label, preserving the "no typed text on disk or in argv" invariant.

- **The `.deb` now ships an `md5sums` manifest, and `klyrshift verify` checks it.** `dpkg-deb --build` does not generate the manifest, so a corrupted or edited file had no way to be detected after installation. `scripts/build_deb.sh` writes one entry per installed file; `klyrshift verify` (and the `doctor` "Целостность пакета" row) compares the files on disk against it and reports a missing file distinctly from a modified one, because the fix differs. `dpkg --verify klyrshift` now works as well. A source checkout or a venv install has no manifest and is reported as "not installed from a .deb", not as a pass.

- **The package ships a man page and shell completion.** `man klyrshift` and `klyrshift <Tab>` worked only for someone who had read the README. Both are generated from the

live argument parser by `scripts/build_cli_docs.py` (`make cli-docs`), so they cannot describe a command that no longer exists: the man page documents every subcommand, and the bash and zsh completions offer its options and its `choices` positionals. The `.deb` installs them under `/usr/share/man/man1` and the two completion directories; `install.sh` and `uninstall.sh` do the same under `~/.local/share`. `tests/test_cli_docs.py` fails on drift, on a missing command and on any `man --warnings` output. `docs/CLI.md` is the prose companion and joins the offline handbook.

- **`--lang` completes its locales.** Typing `klyrshift --lang <Tab>` used to fall through to the command list, so the value had to be remembered; the bash and zsh completions now offer `auto`, `ru`, `en`, `de`, `fr` and `uk`, both for the global option and for `dict pack --lang`. The list comes from `klyrshift.i18n.SUPPORTED_LOCALES`, so a locale added there appears in the completion on the next `make cli-docs`. `--lang=` is completed too. The bash test sources the generated file and drives `_klyrshift`, so a completion that never runs fails the suite rather than passing on a string match.

- **A headless scenario simulator: `klyrshift simulate`.** The daemon reads a keyboard and injects synthetic keys, so it could only be exercised end to end on a real desktop — which an automated agent does not have. `klyrshift simulate 'type ghbdtn, space'` replaces the keyboard, the layout switcher and the text injector with in-process fakes, drives the **real** daemon with a scripted scenario and prints what the user would see (the screen text, each replacement, the rejected words). Steps are comma-separated: `type <text>`, a named `key`/`press`, `space`/`enter`/`tab`/`backspace`, a chord (`ctrl+z`, `shift+f12`), `dblshift`/`dblctrl`/`dblalt`, `layout <name>`, `mode <auto|manual|hybrid>`, `config key=value` and `wait`. `--json` gives a machine-readable result. It also models the application's own `Ctrl+Z` undo, which the daemon only observes, so the "rejected word stays rejected" behaviour is reproduced faithfully. `tests/test_simulate.py` pins it.

- **`klyrshift learn list-rejected`.** The words the user undid with `Ctrl+Z` (never corrected again this session) are now listable, over the local control socket (`{"cmd": "learn.rejected"}`), alongside the learned-word list. Like the learned vocabulary they live in the daemon's memory only.

Changed

- **The interface language is now configurable.** `Config.ui_locale` (`auto`/`ru`/`en`) selects the string catalogue; `auto` follows the desktop locale.

- **The `--lang` help no longer understates the languages.** It said "ru или en" while five catalogues shipped. Both the CLI help and the GUI option now build the list from `SUPPORTED_LOCALES`, and a test fails if the hint and the registry disagree again.

Fixed

- **A config reload is now atomic.** `KlyrshiftDaemon.apply_config` replaced `self.config` and then rebuilt a dozen derived caches; a failure part-way through (a hand-edited `dictionary_custom_path` with a typo'd `~user` raises `RuntimeError`) left the daemon with the new config while the caches still held the old values -- an exception list that did not match the config, hotkeys that did not match the file. The rebuild is wrapped in a rollback that re-applies the previous config on any exception, so the daemon always describes exactly one

state. Pinned by `tests/test_daemon_robustness.py::test_apply_config_is_atomic_on_a_midway_failure`.

- **An invalid `config` step prints a one-line error, not a traceback.** `klyrshift simulate 'config mode=bogus, ...'` raised a raw `ValueError` from `Config.validate()`, which escaped the CLI's `ScenarioError` handling and printed a stack trace. The simulator now translates a validation failure into a `ScenarioError`, so the CLI prints the expected one-liner and the QA executor keeps classifying it as a harness error. Pinned by `tests/test_simulate.py::test_invalid_config_value_raises_a_scenario_error`.

- **The simulator's undo history is bounded.** The faithful per-edit undo model (so `Ctrl+Z` reverts the most recent edit) kept one full screen snapshot per keystroke, which is O(n^2) memory over a long scenario (600 characters kept 601 copies). The history is now capped at `UNDO_HISTORY_LIMIT` (100), matching a real editor's bounded history. Pinned by `tests/test_simulate.py::test_undo_stack_is_bounded_for_a_long_typing_run`.

- **The `.deb` is now byte-reproducible, across environments.** The generated changelog embedded `date -R` and dpkg copied the build tree's mtimes into the archive, so the same tree produced a different sha256 on every build -- and two more differences split a local build from CI: the date came from the newest commit (a PR's synthetic merge commit has a different time than the merge), and dpkg's default compressor changed from xz to zstd in 1.22.x, so a newer host produced `.tar.zst` where an older one produced `.tar.xz`. `scripts/build_deb.sh` now pins `-Zxz`, clamps the tree's mtimes and derives the date from the **release date in `CHANGELOG.md`** (fixed in the tree), so CI and a local build of the same commit produce the identical artifact. `SOURCE_DATE_EPOCH` still overrides for an explicit pin. Pinned by `tests/test_packaging_contract.py::test_build_script_supports_reproducible_builds`.

- **The undo cap's boundary is pinned.** Past the cap, `Ctrl+Z` still undoes the most recent edit (the cap drops the oldest snapshots), never deletes below empty, and never rolls the learned vocabulary back with the screen; a scenario with more `Ctrl+Z` than the cap still completes. The rollback in `apply_config` also restores the config object before rebuilding the caches, so a rollback that itself fails still leaves a consistent config object and logs the residual risk by name. All pinned by tests.

- **The simulator translates any config-override failure, not just `ValueError`.** A typo'd `~user` in a path raises `RuntimeError`; the guard is now by class (with the traceback logged), so a novel invalid input cannot print a traceback either.

- **Manual mode no longer poisons the learned vocabulary.** A word the working mode silenced (manual mode, quiet hours, an excepted application) was never evaluated, yet the flush counted as a "keep" and, after `learning_promote_threshold` passes, promoted the word into the detector's session vocabulary. A wrong-layout word typed three times in manual mode then stopped being fixed after the user switched to auto. A silenced word no longer feeds the keep signal; an evaluated, kept word is still learned. Pinned by `tests/test_learning.py`, `tests/test_simulate.py` and a QA scenario.

- **Enter, Tab and punctuation no longer eat the first character of a word.** Only a Space boundary consumed and retyped its key; Enter, Tab and the punctuation trigger left the boundary character on screen, so the deletion (which counts back from the caret, past that character) removed it and the word's last character: `ghbdtn` + Enter produced

`gпривет`. `on_enter` is on by default, so this hit ordinary typing. The daemon now consumes and retypes any boundary the backend can type (the `uinput` backend learned keys for Enter and Tab), and a backend that cannot type it leaves it in place, which is the safe failure.

- **A reach-back hotkey no longer eats a character after a boundary.** Double Shift, toggle-layout, the T9/punctuation hotkeys and accept-suggestion all delete the last flushed word, but used `len(word)` backspaces while the boundary character was still on screen: `manual` mode + `ghbdtn` + Space + double Shift produced `gпривет`. They now delete the word and the trailing boundary and retype both, via `_memory_word_span()`.

- **A live-buffer hotkey fix no longer leaves a stale boundary.** Toggle-layout, the T9/punctuation hotkeys act on the live buffer when one exists; that fix retypes no boundary, but it left the remembered boundary from an earlier flush, so the next reach-back fix deleted one character too many and ate the separator before the word (`abc привет` -> `abcghbdtn`). The remembered boundary is now updated by every fix, matching what it actually retyped.

- **Manual mode no longer corrects without a command.** The reported bug: with `mode = manual`, Klyrshift still rewrote words on its own. The layout pass was gated on the working mode, but the layout-neutral T9 and punctuation passes were not, so `mode = manual` with `typo_mode = "auto"` corrected every word boundary. `manual` is now the master gate for every automatic pass; the explicit hotkeys (`SHIFT+SHIFT`, `CTRL+SHIFT+F`/`P`) and a per-application override naming a correcting mode still work. Quiet hours silence the layout-neutral passes too, not only the layout pass.

- **`Ctrl+Z` reliably suppresses the word it undid.** `_last_undone_word` was sticky: once set by the first rejection it shadowed the word of every later fix, so after the first `Ctrl+Z` the next correction was never suppressed and kept coming back however often it was undone. Every fix now clears the previous undone word, so `Ctrl+Z` rejects the word actually under the caret. A rejected token that is not a plain word (a punctuation fix such as `текст--`) is suppressed too, instead of being dropped by the word-only guard.

- **The lexical and reverse-conversion hotkeys reach a live word.** Pressing Ctrl or Alt begins a chord and suspends the word being typed, so `Ctrl+C` cannot leave a stale fragment. The fix hotkey restored it, but `CTRL+SHIFT+F` (T9), `CTRL+SHIFT+P` (punctuation) and `CTRL+SHIFT+T` (reverse conversion) did not, so on a live word they acted on an empty buffer and silently did nothing. All three now restore the suspended word first.

- **The GUI CI job runs its GTK tests again.** `ubuntu-latest` moved to 26.04 (Python 3.14) while `setup-python` still installed 3.12; the distribution PyGObject the tests import is built for the distro interpreter's ABI, so `import gi` failed, every GTK test skipped and the job failed on `--cov-fail-under=85` (measured 63 %). The job pins `ubuntu-24.04`.

- **The README screenshots no longer contradict their captions.** The middle shot is committed as `advanced-settings.png` and both the README header and `docs/screenshots/README.md` describe it as the "Продвинутые" page (detector, triggers, per-application layout, exceptions), but the render script captured the basic "Настройки" page under that name -- the two pages share most widgets, so the mismatch was invisible in the thumbnail. It now reveals the advanced page (`advanced_page.set_visible(True)`, the same call the

"Показать продвинутые настройки" button makes) and captures it last, so revealing it does not add a tab to the switcher bar in the other two shots.

- **The screenshots are current again, and a stale set now fails the suite.** The committed PNGs predated the theme switch (`3f4a8cc`), which added the whole "Оформление" group to the settings page, and the first-run wizard (`718d919`), so a fresh render no longer matched the repository. Regenerated. The render script also marked only the tour as seen, but the wizard is versioned separately and `should_offer_wizard` still fired, so on a fresh onboarding state the wizard covered the window in every shot. Both offers are now suppressed. `tests/test_screenshots.py` gains a `KLYRSHIFT_RENDER_TEST=1` case that renders once and compares against `docs/screenshots/`, so a GUI change that is not followed by `scripts/make_screenshots.py` fails instead of leaving a stale image behind, plus guards that the script captures the `advanced` page and marks both first-run offers as seen.

- **The daemon no longer aborts when there is no D-Bus session bus.** The focused-application and focused-element probes tried to initialise AT-SPI unconditionally, and GLib's `dbind` responds to a missing accessibility bus by aborting the process with `SIGTRAP` -- not a Python exception, so no `except` can catch it. Both run on every word, so a TTY or `ssh` session (where `DBUS_SESSION_BUS_ADDRESS` is unset and the probes are called anyway) could take the whole daemon down. They now return early when there is no session bus, before importing `Atspi`.

- **`klyrshift verify` no longer fails on a healthy Ubuntu install.** dpkg honours `path-exclude` rules in `/etc/dpkg/dpkg.cfg.d/` and skips the matching files at unpack time; Ubuntu drops `/usr/share/man/*` and `/usr/share/doc/*` (keeping only `copyright` and `changelog.*`). The packaged man page and `CHANGELOG.md` are therefore listed in the `md5sums` manifest but never reach disk, and the integrity check reported both as missing -- `klyrshift verify` exited 1 and `doctor` showed a red "Package integrity" row on an installation that was perfectly fine. The check now parses the same policy (last matching rule wins, so a `path-include` can re-add a path) and classifies those files as excluded by design, which does not count against the result. `dpkg --verify` behaves the same way: it names the files but exits 0.

dpkg accepts the rule value after whitespace **or** `=`, and the configs that matter use whitespace -- `/etc/dpkg/dpkg.cfg.d/docker` and Ubuntu both ship `path-exclude /usr/share/man/*`. The first cut of the parser split on `=` only, so it read zero rules from those files and the check still failed on exactly the systems it was written for; it now accepts both separators, and a test pins the space form. A new `package` CI step installs the `.deb` under a forced `path-exclude` policy and asserts the check passes, so the regression cannot return unnoticed.

- **The shipped `.deb` had no licence statement.** Debian Policy 12.5 requires a `copyright` file under `/usr/share/doc/<package>`, and the package shipped none — so the base frequency lists travelled with no attribution. Worse, the attribution that did exist was wrong: FrequencyWords releases its code under MIT, but the generated word lists are content under CC-BY-SA-4.0, and the README, the landing page and the About dialog all called them MIT. The `.deb` now ships a DEP-5 `copyright` (with a `changelog.gz`, which lintian otherwise flags at error level), the lists are credited to CC-BY-SA-4.0 everywhere they appear, and the About dialog gains a dictionary-data credit. `lintian` on the built package is clean of licence and changelog errors; `tests/test_packaging_contract.py` pins the file, the attribution and the reference to a licence text Debian actually ships.

- **Shutdown no longer aborts when the control socket is in the selector.** The daemon's cleanup closed every object the selector held, but the control socket is a `ControlServer` with `stop()`, not `close()`. The resulting `AttributeError` escaped the `finally` block (only `OSError` was suppressed) and skipped the rest of the cleanup, so the tray and the socket file leaked and the learned words were not persisted on exit. The control socket is now excluded from the device loop; `tests/test_daemon_robustness.py` drives the real cleanup path and fails if it is skipped again.
- **The withheld-conversion log line leaked the typed word.** The hybrid-mode DEBUG message used `%r` on the buffer, so `log_level=DEBUG` wrote the user's text to `klyrshift.log` -- the same class of leak the T9 path had already fixed. It now logs the confidence and the word length only.

0.2.9 - 2026-10-19

Adds always-on learning, and makes a correction safe to reverse even after the caret has moved.

Added

- **Always-on learning.** Klyrshift now watches how it is used and adapts. A word whose correction the user rejects (`Ctrl+Z`, or the undo hotkey) is not corrected again for the rest of the session, so a fix the user undoes never "changes back". A word the user keeps unchanged often enough (`learning_promote_threshold`, default 3) joins the detector's vocabulary for the session and stops being "corrected" -- brands, names and jargon are picked up without a manual `klyrshift dict add`. The learned words live in the daemon's memory only; the persisted profile holds counters, never a word, so the "no typed text on disk" invariant still holds. `klyrshift learn` shows the counters, `--off` disables learning, `--reset` clears them, and the GUI's Продвинутые page has a matching group.

Changed

- **The release workflow smoke-tests Debian 13, not only Debian 12.** Debian 13 (trixie) is the current stable and the target this package is written for (the udev rule is named `71-*` to run before `73-seat-late.rules`, systemd 257), but the workflow installed the `.deb` only in a `debian:12` container, so a trixie-only regression could ship. Both images are tested.

Fixed

- **The Layero deploy failed before Astro ran.** Astro 7 refuses to build below Node `22.22.0` (`site/package.json` `engines.node`), and the build aborted with `Node.js v22.18.0 is not supported by Astro!`. The deploy log names the source it used: `[config] node=22.18.0 (engines.node)`. `layero.json` had no `nodeVersion` then and no `.nvmrc` existed, so Layero read the major `22` out of the `>=22.22.0` range and built on its own 22-line image, `22.18.0` — below the very floor `engines.node` stated — because Layero keeps one build per major and updates the patch inside it itself (the docs: name the major, we update the exact build within it). Pinning the patch does not help. The pin is therefore a major whose whole range clears the floor: `site/layero.json` sets `nodeVersion: "24"` (`24.0.0 >= 22.22.0`, whereas `22.0.0 < 22.22.0`), and `site/.nvmrc` and the CI `site` job name the same major. `layero.json` outranks the project setting, `.nvmrc`, `.node-version`

and `engines.node`, so the file's `nodeVersion` decides. `tests/test_layero_config.py` enforces the rule — a pin whose major can resolve below the `engines.node` floor fails the test — so the deploy cannot silently drop back under Astro's minimum. The CI `site` job builds on 24 too, and `site/deploy.sh` refuses to run a local build below the `engines.node` floor with a readable message instead of Astro's abort. One caveat the docs disagree on: Layero's `layero.json` page and its `layero-v2` schema both say the file wins, while its runtime-versions page lists the panel's project setting first and never mentions the file, so the panel's "Версия Node.js" must stay on "Авто"/`24`.

- **A `Ctrl`/`Alt` chord leaked its key into the buffer.** Typing a word, then pressing `Ctrl+C` (or any command chord), left the chord's own key in the buffer -- the `c` of `Ctrl+C` -- so the next Space analysed a stray one-letter fragment. A key pressed while `Ctrl`/`Alt`/`Super` is held is now dropped instead of buffered. This is also what makes `Ctrl+Z` observable as "the user undid the last fix" without the chord polluting the buffer.

- **The landing page still canonised the pre-rename hosts.** The hosts were renamed with the project, so `klyrshift.layero.app` and `klyrshift.vercel.app` are the live origins and the old `linguafix.*` names now return 404. The canonical origin, `og:url`, the `hreflang` alternates, the sitemap and `robots.txt` still named `linguafix.*`, so every page advertised a dead address that search engines drop. `astro.config.mjs`, `src/lib/site.ts`, the footer, the `README.md` badges and the dictionary link now name the live hosts, and `tests/test_repo_urls.py` pins them and fails if a `linguafix.*` host reappears in the site source, the README or `docs/`.

- **A fix could be undone onto the wrong text.** The undo history was bounded by time and depth only, so it did not notice that the user had moved on. After a correction, typing anything else and then pressing the undo hotkey deleted the recorded character count from the new text and retyped the old word over it, corrupting what had just been written. The history is now dropped as soon as the caret moves: any key that is neither a hotkey action nor the observed `Ctrl+Z` clears it (an ordinary key changes the text, a command chord such as `Alt+Tab` or `Ctrl+V` changes focus or pastes). A Space that triggers a new correction records a fresh entry, so that correction stays undoable; a refused undo is safer than a corrupted word.

- **`pyproject.toml` and `requirements.txt` disagreed on the `tomli` floor.** A dependency bump raised the floor in `requirements.txt` only, so `install.sh`, which installs from `pyproject.toml`, kept the older `tomli`. Both now require `tomli>=2.5.0` below Python 3.11, and `tests/test_version_consistency.py` fails on the next drift.

0.2.8.4 - 2026-10-18

A fixes-only patch. The guided tour no longer loses the user's place when the window is closed mid-tour.

Fixed

- **An interrupted guided tour restarted from the first step.** `OnboardingState.last_step` was documented as "the index of the last step shown, so an interrupted tour can be resumed", but nothing ever wrote it, so the value stayed `0` and closing the window mid-tour lost the user's place. The overlay now reports each step it

displays, the window persists the index, and `start_tour` resumes an unfinished tour there. Finishing or skipping the tour clears the stored position.
- **The home page claimed auto-switching was paused whenever any app had focus.** The pause banner's reason treated any running application as a reason to pause, so an ordinary auto-mode session that had nothing on the exception list still read «приостановлено (приложение «…»)» as soon as any window was focused — for example the GUI's own window, or a terminal. The reason is now built from the never-touch list the daemon actually honours: it names the application only when that app is on it, and otherwise stays quiet.
- **The GUI silently needed `python3-gi-cairo`, which no dependency list named.** The guided tour's scrim and the fallback power glyph paint through `Gtk.DrawingArea`'s `set_draw_func`, and PyGObject can only hand that callback a `cairo.Context` when the `_gi_cairo` override module is present. `python3-gi` does not pull it in, so a stock `python3-gi` install left both widgets blank, logging `Couldn't find foreign struct converter for 'cairo.Context'` for every frame. `klyrshift gui` now refuses to start with a message that names the fix instead of opening a window that draws nothing, `doctor` reports the missing module, and the `.deb`, `install.sh` and CI package lists install it.
- **The screenshot generator depended on the machine's onboarding state.** A first-run tour offer covered the home page, so the rendered images changed with whatever the developer had already seen, and the committed images carried a stale `lingvix` path. `scripts/make_screenshots.py` now renders against a fixed throwaway `HOME`, so the output is identical from run to run, and the three committed screenshots are regenerated with the current branding.

0.2.8.3 - 2026-10-17

The project is renamed to its final name, the GitHub repository follows, and the release tree, the changelog and the published release are reconciled.

Changed

- **The project is renamed Lingvix → Klyrshift.** `LinguaFix` collided with the Windows/Microsoft utility of the same name, and the intermediate `Lingvix` was never published; `Klyrshift` is the final name. It is used everywhere: the Python package (`src/klyrshift/`, import `klyrshift`), the CLI (`klyrshift`), the systemd unit (`klyrshift.service`), the udev rule (`71-klyrshift.rules`), the desktop/autostart entries, the config, state and cache directories (`~/.config/klyrshift`, `~/.local/share/klyrshift`, `~/.local/state/klyrshift`, `~/.cache/klyrshift`), the Debian package (`klyrshift`), the release assets (`klyrshift_<version>_all.deb`, `klyrshift-dict-<slug>.tar.gz`) and the landing page. The GitHub repository is renamed `Pavel1778/LinguaFix` → `Pavel1778/Klyrshift`; GitHub redirects the old slug, so existing clones keep working.
- **An upgrade from an older install keeps the user's settings.** The config directory follows the application name, so a rename silently starts from defaults. On first load the new package copies `config.toml` (and `snippets.toml`) from `~/.config/lingvix` or `~/.config/linguafix` into `~/.config/klyrshift` — once, only when the new directory has no config yet, and by copying rather than moving, so the old file stays where it is and a current config is never overwritten.

- `README.md`, `AGENTS.md`, `docs/*` and the site reflect the new name. Historical identifiers are kept only where they are load-bearing: the `Conflicts`/`Replaces` fields, the legacy config names above, and the asset-name fallbacks.

Fixed

- **A `SITE_URL` build emitted two different origins.** The sitemap and `robots.txt` take their origin from `astro.config.mjs`, but `<link rel="canonical">`, `og:url` and the `hreflang` alternates take theirs from `site.ts`. Only the config read the `SITE_URL` override, so a local check produced a Vercel sitemap beside a Layero canonical and reported a canonical the deployment would never ship. `site.ts` now honours the override too, and a test fails if either side stops reading it.
- **An upgrade from `linguafix` left two daemons installed.** dpkg treats a package rename as unrelated packages: installing the new `.deb` over `linguafix` left the old package installed, and both kept a systemd user unit enabled at login, so two daemons fought over the keyboard. The control file now declares `Conflicts: linguafix, lingvix` and `Replaces: linguafix, lingvix`, so apt removes the old package first. Both earlier names are listed because the package was renamed twice and a locally built `.deb` may carry the intermediate one. Verified in `debian:13`: before the change both `linguafix` and `klyrshift` stayed installed with both units enabled; after it, only `klyrshift` remains and only `klyrshift.service` is enabled.
- **A fresh Debian install had no layout-switching backend.** `g3kb-switch` (the Wayland backend) is not in the Debian repositories (no candidate on Debian 12 or 13), and the X11 fallback `setxkbmap` (from `x11-xkb-utils`) was not in any dependency list. A plain `apt install ./klyrshift.deb` therefore reported `backend=none` and could not switch the layout at all. `x11-xkb-utils` is now a `Recommends` in the `.deb` and part of `install.sh`'s apt list; `klyrshift doctor` reports `backend=setxkbmap` on a fresh Debian 13.
- **The source-install snippet would break at the next rename.** The landing page cloned from `${REPO}` (derived from `REPO_SLUG`) but then ran a hard-coded `cd LinguaFix`. Both spell the same name today, so it works; the moment the repository is renamed, the clone lands in the new directory and `cd` fails for everyone who copies the snippet. `consts.ts` now exposes `REPO_NAME` and the snippet derives the directory from it. A new guard pins the `git clone`/`cd` pairs in `README.md` and `docs/*` to the live repository name, so a rename cannot leave a stale one behind.
- **The source installer had no way to opt out of autostart.** `install.sh` always wrote the XDG autostart entry and always ran `systemctl --user enable --now`, so a user who did not want the daemon starting at every login had to install first and disable afterwards. It now takes `--no-autostart`, which skips the entry, undoes a previous `enable`, and still starts the daemon for the current session. Re-running with the flag also removes an entry a previous install left behind, so the opt-out holds. `tests/test_installer_autostart.py` runs the real script against stub system binaries and fails if the flag is ignored.
- **Autostart could not be turned off.** `disable_autostart()` removed only the systemd user unit, but `install.sh` also writes an XDG entry (`~/.config/autostart/klyrshift.desktop`). That entry kept launching the daemon, so the GUI switch snapped back on at the next refresh. Disabling now removes the XDG entry and masks the unit; enabling removes the mask.
- **Autostart always read as off.** `systemd_enabled()` trusted `systemctl --user is-enabled`, which reports a unit enabled globally (`/etc/systemd/user`, what the `.deb` does) as

`disabled` until the session reloads. A working autostart therefore showed as off in the GUI. The probe now also checks the global wants symlink directly.

- **The landing page told search engines to index a 404.** `astro.config.mjs` and `src/lib/site.ts` declared the canonical origin as `klyrshift.layero.app` and `klyrshift.vercel.app`, but only the `linguafix.*` hosts were ever configured, so both `klyrshift.*` names return 404. Astro writes that origin into `<link rel="canonical">`, `og:url`, the `hreflang` alternates, the sitemap and `robots.txt`, so the live page advertised a dead address and the footer linked it. `README.md` carried the same dead origin in its three status badges, its dictionary link and its "where to find us" list, so the repository advertised 404s as well. All of them now name the hosts that resolve, and `tests/test_repo_urls.py` fails if a `klyrshift.*` host reappears in the site source, the README or `docs/`.
- **`dict install` never used the release archive.** The live release predates the rename and attaches its thematic archives as `linguafix-dict-<slug>.tar.gz`, while the code requested `klyrshift-dict-<slug>.tar.gz`. Every install therefore 404'd on the first source and fell through to the branch mirror -- an extra round trip, and the archive path never exercised. All three spellings are tried now, newest first (`klyrshift-`, then `lingvix-`, then `linguafix-`), so the command works against the published release and against the next one.
- **An unrecognised modifier in a hotkey silently became a bare key.** The matcher fires when the held modifier families equal the binding's, so `hotkey_fix_last_word = "CMD+Q"` dropped the unknown `CMD`, left an empty family set, and then triggered on every press of `Q`. `normalise_hotkey()` now rejects a chord whose non-final token is not a known modifier, and the same guard runs in the daemon's parser so a config built in memory cannot slip past it. A lone modifier (`"CTRL"`) and a chord ending in one (`"CTRL+SHIFT"`) are rejected too, since neither has a key to fire on; `CTRL+CTRL` remains the valid legacy double tap.
- **The rename relationship was documented nowhere.** A user upgrading from `linguafix` had no instructions: `docs/INSTALL.md` now has an "Upgrading from the old `linguafix` package" section. It also records what a raw `dpkg -i` does differently from `apt install ./…`: the old package is removed, but dpkg keeps an empty `linguafix` record in its database (state `ic`) and reports `1 not upgraded`. That is cosmetic — verified on Debian 12 and 13 that no file, unit or daemon survives — and `apt-get -f install` clears it. `Breaks:` was measured against `Conflicts:` and rejected: `Breaks` leaves the old unit on disk and still enabled, so both daemons would run.
- **The release gate could never pass on the tag it was checking.** The workflow runs `release_consistency.py --source --tag <tag> --assets-file <staged>` before it creates the release, but the script fetched the release whenever `--tag` was present. The tag has no release yet, so the fetch 404ed and the job failed before publishing anything — the gate was added after the last tag was cut, so it had never run for real. `--assets-file` already supplies the asset names, so the release is now fetched only when the API is the only source: with `--latest`, or with `--tag` and no `--assets-file`.

Added

- **A guided tour, context help and a "Помощь и обучение" page.** A first-run offer leads into a six-step tour — home screen, languages, mode, fix hotkey, environment check, closing card — that highlights the real widget at each step and can be skipped or replayed. The tour and the help texts live in `klyrshift.onboarding` as plain data, so the scenario is validated without a display and each topic names the `Config` fields it explains — a renamed field fails the suite instead of silently orphaning its text. Settings groups carry a

"?" button that opens the matching topic, the help page repeats the tour, lists every topic, runs the environment check and states the known limitations, and finishing or skipping records the tour version so a routine start stays quiet.

- `scripts/acceptance_check.sh` **now checks detection and the modes without a keyboard.** It drives the installed package's detector directly and pins nine cases: a word typed in the wrong layout converts in both directions, an already-correct word is left alone, and a URL, an e-mail, a file path, a version string and a hyphenated identifier are protected. It then sets each mode (`auto`, `hybrid`, `manual`) through the CLI and reads it back. These are real checks, not substitutes for the manual session steps, which now also cover both session types (Wayland and X11), the per-mode behaviour, the exceptions list, and that exactly one daemon runs.
- **The JSON-LD version could drift from the page's own version.** `Base.astro` wrote `softwareVersion: "0.2.8.2"` as a literal while `consts.ts` held `VERSION`, so a bump updated the hero and left the structured data behind — two versions on one page, invisible to every other test. The layout now imports `VERSION`, and `scripts/release_consistency.py` fails if the literal returns.
- `scripts/release_consistency.py`**, a release-consistency check.** `tests/test_version_consistency.py` compared the source files only with each other, so it could not see that the site advertised `0.2.8.2` while the newest published release was `v0.2.8`. The new script splits the two questions: `--source` compares the tree with itself and with the asset names the build scripts emit (offline, run by the unit suite), while `--tag` / `--latest` compare it with the release that actually exists (network, run by the release workflow). The workflow now gates the publish on the tag, the tree version and the staged asset list agreeing, and checks the published download URLs afterwards. `tests/test_release_consistency.py` pins both halves, exercising the published-release drift with a recorded payload so the normal test run never touches the network.
- `docs/RELEASING.md`**.** The release steps were spread across `CONTRIBUTING.md`, `TESTING.md` and the workflow. They are now one checklist: the version sources, the Debian 12/13 package checks, the upgrade from the old `linguafix` package, the asset list, the documentation and canonical checks, the real-keyboard test CI cannot run, and the two blockers outside this repository.

- **The mojibake guard missed double-encoded punctuation.** It only looked for the byte signatures of double-encoded Cyrillic, so a file whose arrows or em-dashes had been re-saved through a single-byte pipeline passed the test while shipping garbage to the user. It now also round-trips each line back through the codecs such a save uses and fails on any line that decodes cleanly, with a unit test that pins the arrow and em-dash cases.

- **Hybrid mode now differs from auto.** Previously `hybrid` behaved exactly like `auto`, so the mode existed in name only. The detector gained `target_layout_with_confidence()`, which reports the plausibility margin by which a candidate beat the text as typed (and `inf` for a conversion that is certain rather than scored: a taught word, or one whose every word is already known in the target language). Hybrid applies an automatic fix only when that margin clears `hybrid_confidence` (default `3.0`, tunable in the GUI under Детектор); an uncertain guess is withheld for the manual hotkey instead of being rewritten. `auto` still applies everything the detector accepts, and a forced fix (double Shift, force-in-manual app) ignores the threshold. The default was measured over the top 2000 Russian words typed on US: ~2.6% of true fixes fall below 3.0, the median margin is ~7.6.

- **The landing-page demo no longer disagrees with the daemon.** It had no vocabulary-validated override, so it left `ltym` untouched while the app corrected it to `день`; porting the override then exposed a second bug — the demo tokenised with JS `\w`, which is ASCII-only, so a Cyrillic conversion never matched and the override was a silent no-op. `tests/test_demo_parity.py` now runs the demo's TypeScript under Node and compares it with the real detector, so the two cannot drift again.
- **Live demo on the landing page** (`site/src/components/Demo.astro`). It ports the detector's per-word plausibility test to TypeScript, so a visitor can try the correction in the browser. Its vocabulary and bigram tables are generated from the shipped corpora by `scripts/build_demo_model.py` (`tests/test_demo_model.py` fails if they drift).
- `scripts/measure_detector.py` — reproducible recall / false-positive / corruption / latency measurement on fixed slices of the shipped corpora.
- `tests/test_demo_model.py` and new autostart cases in `tests/test_daemon_control.py`, plus hybrid-mode regression tests in `tests/test_modes_hotkeys.py` and `tests/test_demo_parity.py`.

Removed

- `docs/TESTING_v0.2.0.md` — superseded by `docs/TESTING.md`; its still-useful GUI checklist is merged into the latter as section H.

[0.2.8.2] - 2026-10-08

Follow-up to 0.2.8.1 for two defects reported after the 0.2.8 live test, plus an independent-mode model for T9 and punctuation.

Fixed

- **The config-migration fix now actually ships.** The atomic write-back that persists migrations to `config.toml` was committed to `main` (`089afee`) but was not in the `v0.2.8` tag, so a user who installed that build still saw the stale file. It is part of this release; on first load the migrated values are written back and the next `lingvix config show` agrees with the daemon.
- **T9 no longer rewrites correct words ("Т9-спам").** The reported build predated the v0.2.8.1 guard (`typo_in_manual`); it is now part of the shipped code. On top of it, the typo pass is gated by its own mode (below) rather than the global layout mode, so in `manual` layout mode a correct word is never auto-rewritten and the automatic pass only runs when `typo_mode = "auto"`.

Added

- **Independent per-feature modes.** `typo_mode` and `punctuation_mode` (`off` / `manual` / `auto` / `hybrid`) are the source of truth for T9 and punctuation, decoupled from the global layout `mode`. A legacy config is migrated on load: `typo_correction = true` with `mode = "manual"` becomes `typo_mode = "manual"`, otherwise the global mode is used. The legacy booleans are kept mirrored for compatibility. New manual hotkeys: `CTRL+SHIFT+F` (T9) and `CTRL+SHIFT+P` (punctuation).
- **T9 tab on `lingvix gui`.** Both features have an independent mode combo, the full parameter set (min word length, max edit distance for short/long words, long-word

threshold, confidence margin), an expander with examples, a "Проверить сейчас" preview dialog, and a link to the documentation. The tab was previously a plain on/off toggle.
- **Expanded thematic dictionaries to 1000+ words.** The priority categories were too small for professional text (T9 could not recognise terms like `kubectl`, `фотосинтез`, `энтальпия`). Now: `it` ru 328→1010 / en 295→1263, `medicine` ru 228→1579 / en 215→2136, `science` ru 149→1077 / en 152→1692. The remaining categories keep their current size. README counts are regenerated from the files (`scripts/refresh_dict_readmes.py`).
- Expanded `README.md` and `docs/FAQ.md` with honest dictionary numbers.

Changed

- **`/dictionaries` page redesign.** A summary band (categories / words / licence), per-category accent monograms, per-language word counts and sizes, and a cleaner base-list grid. Counts are read from the shipped files at build time, never typed.
- Thematic category `README.md` word counts are now exact and regenerated by `scripts/refresh_dict_readmes.py` (with a `--check` mode); a test fails if the documented count drifts from the file.

[0.2.8.1] - 2026-10-08

Hotfix for three defects reported after the 0.2.8 live test.

Fixed

- **Config migrations now reach the file.** A migration applied on load (the `CTRL+CTRL` undo hotkey, the `300 ms` double-tap window, the `gedit` → `texteditor` exception) lived only in memory, so `lingvix config show` — which reads the file — kept displaying the stale values while the daemon logged the migrated ones. `load_config` now writes the migrated config back atomically (temp file + `os.replace`) and logs `Config: migrated and saved ...`. `lingvix config migrate [--dry-run]` previews the change and `config show` warns when file and effective values diverge. Deliberate user values (`mode`, `backspace_settle_ms`, `min_word_length`, a hand-picked double-tap window) are never migrated.
- **T9 no longer rewrites correct words ("T9-спам").** A typo pass now refuses a word that is known in any loaded language (a name or loanword the frequency corpus omitted, e.g. `саша`), a word already corrected within `typo_debounce_seconds` (default 5 s), and a "correction" that would replace a plausible current-language word with a less plausible one (`суток` → `сутак`). In `mode = "manual"` the automatic T9 pass is silent by default; set `typo_in_manual = true` to restore it. The explicit fix hotkey is unaffected.
- **A mixed-script buffer is never rewritten as one word.** Text containing both Latin and Cyrillic (`ghbdtnпривет`) is left for the separator between the two scripts to flush, so a fast typist no longer gets `ghйпривет`.

Added

- Seven new thematic dictionary categories — `science`, `business`, `electronics`, `media`, `education`, `gaming`, `sport` (ru + en, ~150 words each), on top of the original five. Each ships as a `lingvix-dict-<slug>.tar.gz` release asset and is listed on the landing page. Install with `lingvix dict install <category> --lang ru`.

Changed

- `site/layero.json` is confirmed as the Layero build config; the root `layero.json` added by the previous session was removed (Layero reads the project config, and the change was based on a misdiagnosis of a Layero infrastructure incident).

[0.2.8] - 2026-10-08

Added

- `lingvix dict download <lang>` and `lingvix dict import-file <path>` load extended frequency word lists (`ru`, `en`, `uk`, `de`, `fr`) into `$XDG_DATA_HOME/lingvix/dictionaries/<lang>.txt`. The detector reads them in addition to the bundled corpora, widening vocabulary membership only — n-gram scoring is unchanged, so already-correct words keep their verdict while rarer words are recognised. Lists mirror FrequencyWords content (CC-BY-SA-4.0; the upstream code is MIT) under the repository's `dictionaries/` folder. A new landing-page section links them.
- `dict download` now tries the stable `releases/latest/download/<lang>-50k.txt` asset first, then the branch mirror, then upstream, and finally copies a locally shipped list — so it succeeds with no network. Every list is attached to each GitHub Release and bundled in the `.deb` (`lingvix/data/dictionaries/`), and the landing page links the release assets rather than a branch path.
- A `toggle_layout` hotkey (default `CTRL+SHIFT+T`) force-retypes the last word in the other layout, the manual reverse of a correction. Configurable as `hotkey_toggle_layout_last_word`.
- **Thematic (professional) dictionaries.** `lingvix dict categories`, `dict install <category> [--lang ru]`, `dict list-installed` and `dict remove-category <category>` add domain vocabularies on top of the base frequency list: `it`, `medicine`, `legal`, `finance`, `engineering` (ru + en). Installed categories are recorded in `installed_dict_categories` (default `["base"]`) and loaded as membership on top of the corpora. Detection priority is **user dictionary → thematic → base → T9**. Thematic lists are **not** bundled in the `.deb`; each is published as a `lingvix-dict-<slug>.tar.gz` release asset, fetched first, then from the branch mirror, then from the offline repository copy. The `/dictionaries` landing page lists the sections with language, word count, size and licence (MIT).

Changed

- The undo hotkey default (`SHIFT+BACKSPACE`) and the new toggle hotkey are shown by `lingvix hotkeys` and the GUI. A stale `CTRL+CTRL` value left by an older install is migrated to the default on load and the change is logged.
- A config still on the pre-0.2.8 double-tap window (`hotkey_double_tap_ms = 300`) is widened to the current default (2000 ms): 300 ms is too short to catch a deliberate double tap reliably. Any other value is treated as a deliberate choice and kept.
- A config still carrying the old default exception `gedit` (not installed on Debian 13 / GNOME 48) is migrated to `texteditor` (GNOME Text Editor) on load, so the current default text editor is actually excepted.
- **`manual` mode semantics (contract change).** Previously `mode = "manual"` gated every automatic correction, so an opt-in T9 pass never fired (`langauge` → `language` did nothing). Now only the automatic layout switch is manual; layout-neutral passes (T9 typo,

punctuation) run whenever their own switches are on, in every mode. A user who wants a fully quiet daemon turns `typo_correction` and `punctuation_correction` off (both default off). The settings-tab subtitle says so; `tests/test_typo_daemon.py` and `tests/test_user_config_migration.py` pin the behaviour.

Fixed

- The GUI tab icons now fall back to text when a symbolic icon is missing from the theme, instead of failing to draw the tab.
- The two desktop notifications (correction and update-available) had been corrupted into mojibake by an earlier edit; they are readable again. A test now fails on double-encoded UTF-8 anywhere in the tracked sources.
- The install examples in `README.md`, `docs/INSTALL.md` and `docs/TESTING.md` named an old package version (`0.2.0`/`0.2.1`), so copying them failed with "no such file"; they match the current release and a test pins them to it.

[0.2.7] - 2026-10-08

Fixed

- The undo hotkey documented in `README.md` and `docs/TROUBLESHOOTING.md` now matches the real default, `SHIFT+BACKSPACE` (both files said `CTRL+Z`). The default has been `SHIFT+BACKSPACE` since v0.2.2 precisely because `CTRL+Z` collides with the application's own undo; the daemon does not grab the keyboard, so it cannot swallow the chord. A user who followed the docs pressed `CTRL+Z`, the daemon's undo never fired, and the application undid the corrected word — deleting it without the daemon retyping the original, leaving the cursor in an empty spot, exactly as reported. A regression test pins the documented value to `DEFAULT_UNDO_HOTKEY`.

Added

- The daemon logs the effective hotkeys at startup (`fix`, `undo`, `toggle`, `reload`) so the binding is discoverable without opening the GUI. The log holds the binding names only, never typed text. The undo path also logs `backspace count` and the layout transition at DEBUG level.

Changed

- CI: every job now sets `timeout-minutes: 15`. A stuck GitHub runner once left the Python 3.12 job pending for ~49 minutes (the same suite finishes in ~1 minute), which hung the whole check; the timeout bounds that. A regression test asserts every job keeps a timeout.

[0.2.6] - 2026-10-08

Fixed

- The `Received SIGUSR2` log spam is gone for real. The daemon now rewrites its metadata-only history snapshot itself whenever the history changes (a fix or an undo); the GUI only reads the file and no longer signals the daemon on a 2 s poll timer. The v0.2.5 fix gated one

tab's timer but the daemon's own per-fix write was never triggered without the signal, so the log stayed noisy.
- `typo_correction` (and any other GUI setting) is no longer reset to `false` after a reinstall or a mode change. The daemon's `_persist_config` used to dump its whole, possibly stale, in-memory config over the file; it now applies only the field it changed onto the current file contents.
- A double Shift now fixes 1-2 character tokens (`фт` -> `an`), which the ordinary detector skips below `min_word_length`. A conversion is applied only when it is unambiguous: the result must be a known word of the target language and the token must not already be a word of the current one; a taught word always wins. Ambiguous tokens (for example a lone letter) are left untouched.
- An invalid `config.toml` value now logs the exact key it dropped, so a silent revert to a default is traceable to the offending line.

Added

- The `.deb` `postinst` runs `systemctl --user daemon-reload` after replacing the unit, so the next `systemctl --user restart lingvix.service` no longer warns that the unit changed on disk.
- `lingvix doctor` gains a "Графический интерфейс" check that names the exact packages to install (`python3-gi gir1.2-gtk-4.0 gir1.2-adw-1`) when the GTK4/libadwaita stack is absent. The GUI stack is a `Recommends` of the `.deb` (a headless install does not need it), so a download followed by `--no-install-recommends` previously failed with a bare message; the requirement is now documented and diagnosed up front.

Changed

- The Home tab's 1.5 s status poll is paused while the tab is hidden and refreshes on open, so a background window does not keep probing `systemctl`/`g3kb-switch`/D-Bus.

Fixed

- `apt remove --purge` now leaves no empty `/usr/lib/lingvix` directories behind. The new `doctor` GUI check imports `lingvix.gui`; run as root, that writes root-owned `gui/__pycache__` under `/usr/lib`, which dpkg does not track. `postrm` now removes every nested `__pycache__` and prunes the directories it leaves empty.

[0.2.5] - 2026-10-08

Fixed

- The History tab no longer signals the daemon every 2 seconds (`Received SIGUSR2` log spam). The GUI polled the history snapshot regardless of which tab was visible; it now pauses while the page is hidden and refreshes on open.
- A layout correction can no longer rewrite text in the same layout. If the detector returned the current layout, the daemon deleted and retyped the word in place, which could deform it (the reported `Fixing buffer of length N (ru -> ru)`); such a fix is now refused and logged with `reason=`.

- One invalid value in `config.toml` no longer resets every setting to its default. `Config.from_dict` now drops only the offending keys, so a hand-edited typo cannot silently turn `typo_correction` (or anything else) back off.
- The big power button falls back to a Cairo-drawn glyph when the active icon theme cannot resolve `system-shutdown-symbolic`, so it is never a broken-image placeholder on a non-Adwaita theme.
- The default `backspace_settle_ms` is now 120 ms (was 80 ms), giving Chromium and Electron more time to apply the synthetic Backspaces before the replacement characters arrive.

Added

- `lingvix logs [-n N] [-f] [--level LEVEL]` prints (or follows) the daemon log, with an optional level filter.
- The `Fixing buffer ...` log line now carries `reason=layout|typo|punctuation`, so a correction can be told apart from a typo fix without guessing.

[0.2.4] - 2026-10-04

Fixed

- The big power button now uses `system-shutdown-symbolic`. The old name `power-symbolic` does not exist in adwaita-icon-theme 48, so the button rendered as a broken-image placeholder. A new `tests/test_icons_exist.py` scans every `icon_name` in the GUI and asserts it exists in the installed icon theme.
- `lingvix fix` now applies the opt-in typo corrector when layout detection finds nothing, matching the daemon; `lingvix fix --text "превет"` gives `привет` when `typo_correction` is on.
- `lingvix status` now uses the same "running" definition as `stop` and the GUI (the flock PID lock, or an active systemd user unit), so a service daemon without a lock file is no longer reported as not running.
- `lingvix start` delegates to the shared control module instead of spawning and polling on its own, so `start` and the next `status` can no longer disagree.
- The manual-fix hotkey now resolves a taught short token (`ы` -> `s`) even when it is below `min_word_length`; only a conversion present in the user dictionary qualifies.

0.2.3 - 2026-10-07

Added

- Two-edit typo correction for long words: a word of at least six characters may now be corrected within `typo_max_distance_long` edits (default 2), but only when the best candidate is clearly more frequent than the runner-up. Short words keep the strict one-edit rule. Config keys: `typo_max_distance_long`, `typo_long_word_threshold`, `typo_top1_ratio_strict`.
- Adaptive idle timeout (`analysis_timeout_adaptive`, on by default): the fallback that flushes a word typed without a separator now follows the user's typing speed — shorter for a fast typist, longer for a slow one — clamped to 0.3–2.0 s. Only inter-key timing is recorded, never characters.

- A "T9" tab in the GUI with a single master toggle for typo correction and the new punctuation cleanup, plus per-rule rows.
- Opt-in punctuation cleanup (`punctuation.py`): dashes, ellipsis, Cyrillic smart quotes and spacing, applied only when layout detection and typo correction both decline. A replacement the input backend cannot type is refused wholesale rather than half-applied. Off by default.
- Search-engine metadata for two indexed hosts: each host declares itself canonical (`https://lingvix.layero.app` for the Russian primary, `https://lingvix.vercel.app` for the English mirror) and cross-links the other with `hreflang` (`ru`, `en`, `x-default`), so the copies do not compete as duplicate content. A generated `robots.txt` and a sitemap are emitted per host, the default `description`/`og:locale`/JSON-LD follow the host language, and the README carries a keyword paragraph plus both site links.
- Google and Yandex site-verification files served on both hosts, an `apple-touch-icon`, and a footer "About" block in the host language.
- Packaging contract tests (`tests/test_packaging_contract.py`): the systemd user unit (no `Nice`/`CPUSchedulingPolicy`, `Restart=always`), the desktop entry, package-data coverage, bytecode stripping in `build_deb.sh`, the release workflow's tag filter, and a real `dpkg-deb` contents check.

Changed

- `lingvix fix` now loads the user dictionary before detection, so a taught brand such as `vercel` is corrected by the CLI exactly as the daemon does.

Fixed

- The GUI no longer freezes ("приложение не отвечает"): the home page ran its daemon probes (`systemctl`, `g3kb-switch`, `xprop`, the focused-app D-Bus query) synchronously on the GTK main thread on a timer and on every toggle click. They now run on a worker thread (`gui/async_utils.run_in_background`) and report back on the main thread; the toggle click and its reconcile poll are async too.
- The window tab labels are no longer truncated ("Глав…"): the fixed wide `ViewSwitcher` is replaced by a header `ViewSwitcherTitle` plus a bottom `ViewSwitcherBar` that appears only on narrow windows.
- `daemon_control._systemctl` backs off for 10 s after a timeout so a hung user D-Bus cannot re-pay the 5 s timeout on every GUI refresh.
- The desktop launcher declares `StartupWMClass` equal to the application id, so GNOME associates the running window with its installed entry.

- Both hosts publish their own sitemap and `robots.txt` instead of the preview suppressing them, so each is independently discoverable without duplicate content.

- Landing page mobile audit (8 items): removed the leaked developer note «Плейсхолдер — замените реальным скриншотом» from all four mock-up SVGs; replaced the external shields.io footer badges (broken behind a slow/blocked CDN, and not proxied by the host) with self-hosted SVGs in `site/public/badges/`; the footer now names the single canonical host (lingvix.layero.app) instead of a mirror link that hit a login wall; code blocks now wrap instead of clipping at 375 px; the install tabs stack vertically on mobile; localised the

«Privacy First» feature title to «Приватность»; softened the step-03 copy to «Атомарно, одной транзакцией».

- The opt-in update check no longer blocks the keyboard event loop: the daily network request runs on a worker thread, so a slow or offline network cannot freeze typing for the whole request timeout.

- Landing page: the power glyph in the hero and the placeholder window is centred again. The arc's sweep flag was inverted, which pushed the icon to the top of the button instead of forming a ring around its centre.
- Landing page demo leaves URL, e-mail, path and version tokens untouched, matching the daemon's structural-separator rule.
- Re-synced `black` formatting of `tests/test_data_sync.py` with the current release of the formatter.

Removed

- Stale release tag `v0.2.0-stage2` and the merged feature branches; `main` is the only branch left in `origin`.

0.2.2 - 2026-10-07

Fixed

- `lingvix stop` (and `kill`) now actually stop a packaged daemon. The CLI only sent `SIGTERM` to the lock-file PID, so under the systemd user unit (`Restart=always`) systemd respawned the daemon about 5 s later and it looked unstoppable. Both commands now go through `daemon_control`, which stops the unit first; `kill` gained a force-stop path that does the same before sending `SIGKILL`.
- GUI settings now reach a running daemon. Every `GuiState.save` asks a live daemon to reload, so a switch flipped in the GUI (T9, a trigger, a guard) is applied immediately instead of only after a restart. This is why T9 appeared "on" but never corrected: the toggle was written to `config.toml` but the daemon kept its old in-memory copy.
- The window no longer resizes itself and breaks. The bottom switcher's `reveal` had two writers (the `ViewSwitcherTitle` binding and a manual `notify::default-width` handler with a different threshold); the duplicate handler is gone. The advanced page is added to the stack once at construction (hidden until revealed) instead of being appended at runtime to a homogeneous `ViewStack`, and the plain pages scroll, so the window shrinks cleanly.

Changed

- The big on/off button looks like the GNOME quick-settings power button: a white circle with a soft outer ring and a thin `power-symbolic` glyph that turns accent-coloured when active. The `big-toggle`/`active-state`/`busy-state` classes previously had no stylesheet at all. Colours are libadwaita named colours, so light/dark themes follow automatically, and the transitions are dropped when the system asks for reduced motion.

0.2.1 - 2026-10-07

Fixed

- GUI toggle: удалены невозможные для user-service директивы Nice/CPUScheduling из systemd unit (это была причина, почему сервис не запускался).
- GUI toggle: расширено окно reconcile до 10 секунд (совпадает с START_TIMEOUT).
- GUI toggle: показ реальной причины ошибки через toast (systemctl stderr / spawn failure / daemon exited immediately).
- Icon: добавлена зависимость `librsvg2-common` — без неё GNOME не рендерит SVG-иконку.
- Icon: добавлена зависимость `hicolor-icon-theme`.
- daemon: `_start_tray()` перенесён внутрь try/finally, чтобы исключение в трее не оставляло stale lock.

0.2.0 - 2026-10-04

Added

- Full 10 000-word frequency dictionaries for all five supported languages (`en`, `ru`, `uk`, `de`, `fr`), built from the FrequencyWords lists. The detector now loads any of them; `uk`/`de`/`fr` remain opt-in through `Config.languages`.
- A Ctrl/Alt chord abandons the word being typed, so `Ctrl+C` no longer leaves a stale word for the next Space to "correct". A Ctrl-based fix hotkey (`CTRL+F12`) and the `CTRL+CTRL` double tap still see the suspended word.
- A word taught in the user dictionary is honoured as a converted form before the false-positive guards run, so a brand typed in the wrong layout (`муксуд` -> `vercel`) is always fixed.
- Opt-in typo correction (T9). When the layout is already correct, a word within one edit (Damerau-Levenshtein) of exactly one dictionary word is replaced (`langauge` -> `language`, `teh` -> `the`). Ambiguous words, taught words and tokens with a separator are left alone. Toggle it in the GUI (**Advanced** -> **Typo correction (T9)**) or with `typo_correction = true`; `typo_max_distance` (1 or 2) and `typo_min_word_length` tune it.
- Password-field guard (AT-SPI): a fix is skipped while the focused element is a password entry, even if the stop-word list does not cover it.
- Text expansion (snippets). A `[snippets]` table in `snippets.toml` maps a trigger to text (with `{date}` / `{time}` placeholders); the daemon expands it before layout analysis. Toggle with `text_expander_enabled`.
- Selection fix. `CTRL+SHIFT+L` converts the layout of already-selected text (copy, convert, paste, restore) and can be rebound in the GUI.
- Per-app default layout. `app_layouts` maps an application name to a layout that is switched to when that application gains focus (`app_layout_switch`).
- Settings export/import. `lingvix export` / `lingvix import` (and the GUI **Backup** group) bundle the config, user dictionary and snippets into one JSON file. `lingvix restart` restarts the daemon preserving whether it runs as a user service; `lingvix config path` prints the config location.
- Opt-in update check. Once a day, when `update_check_enabled` is set, the daemon asks the GitHub releases API for the latest tag and logs (and notifies) when a newer version exists. Off by default — it is the only feature that uses the network. The GUI job now runs under

xvfb and includes the GUI package in coverage (`.github/workflows/ci.yml`, `coverage-gui.rc`).
- Pre-filled exception list. `exceptions_apps` now ships with terminals, IDEs, editors and games (`gnome-terminal`, `code`, `vim`, `steam`, ...), where the text typed is a command or an identifier rather than a word. A manual fix (`SHIFT+SHIFT`) bypasses the list, so it still works inside a terminal.
- Quiet hours (`quiet_hours_enabled`, `quiet_hours_start`, `quiet_hours_end`). Inside the daily window (may wrap midnight) automatic correction is paused; the manual fix keeps working. The GUI home tab explains the pause.
- A correction-history tab in the GUI, backed by a bounded ring of recent fixes (`history_size`). Entries hold metadata only — word length, layouts, time and an `undone` flag — and the newest can be undone from the tab.

Changed

- Default undo hotkey `CTRL+Z` -> `SHIFT+BACKSPACE` (Ctrl+Z collides with the application's own undo).
- The systemd unit runs the daemon with `Nice=-10` and round-robin scheduling so a fix wins the race against the compositor's handling of the boundary key.
- `dictionary_size` truncation now keeps the most frequent words (the corpora are stored frequency-first) instead of an alphabetical slice.

Fixed

- A user-taught brand whose source text looks plausible (`муксуд`) was rejected by the plausibility guard before the user dictionary was consulted.

0.1.0 - 2026-10-04

Added

- Automatic keyboard-layout correction (`ru` <-> `en`; `uk`/`de`/`fr` opt-in).
- Word-boundary triggers: Space, Enter, Tab, punctuation, plus an idle timeout.
- Scancode-replay replacement with an atomic Backspace batch and `backspace_settle_ms` / `trigger_settle_ms` pauses for Chromium/Electron.
- GUI (GTK4 + libadwaita): big on/off toggle, mode switcher, autostart switch, user-dictionary page, per-app exceptions.
- Working modes `auto` / `manual` / `hybrid` with undo via `Ctrl+Z`.
- Manual fix hotkey bound to a double tap of Shift by default (configurable).
- Per-app exceptions, context analysis and a user dictionary.
- CLI: `start`, `stop`, `kill`, `status`, `config`, `fix`, `doctor`, `collect-logs`, `dict`, `gui`, `mode`, `undo`.
- Zero-config `.deb`: `uaccess` udev rule, systemd user service, autostart entry and an application-menu entry.

Fixed

- Truncation when a fix was triggered by Space/Enter (`руддщ` -> `phello`): a boundary flush now waits `trigger_settle_ms` before deleting, so the boundary key is processed before the Backspaces arrive.
- `Ctrl+Shift+Shift` (and other chords) no longer fire the double-tap fix hotkey; a tap counts only when no other modifier family is held.
- Application menu entry appears immediately after install (`update-desktop-database` in the `.deb` postinst and `install.sh`).
- False-positive corrections on plausible text, hyphenated identifiers and tokens with internal separators.
- Multi-character undo is reliable through the daemon (`lingvix undo`).

Known limitations

- Password-field detection is not possible on Wayland; stop words are the mitigation.
- The `wtype` and `xdotool` backends are not atomic (only `uinput` is).
- GNOME layout switching requires `g3kb-switch`.
- The `uk`/`de`/`fr` corpora are opt-in and have not been calibrated against live input.